Modeling of Data

CHAPTER 15

15.0 Introduction

Given a set of observations, one often wants to condense and summarize the data by fitting it to a *model* that depends on adjustable parameters. Sometimes the model is simply a convenient class of functions, such as polynomials or Gaussians, and the fit supplies the appropriate coefficients. Other times, the model's parameters come from some underlying theory that the data are supposed to satisfy; examples are rate coefficients in a complex network of chemical reactions or orbital elements of a binary star. Modeling can also be used as a kind of constrained interpolation, where you want to extend a few data points into a continuous function, but with some underlying idea of what that function should look like.

One very general approach has the following paradigm: You choose or design a *figure-of-merit function* (merit function, for short) that measures the agreement between the data and the model with a particular choice of parameters. In frequentist statistics, the merit function is conventionally arranged so that small values represent close agreement. Bayesians choose as their merit function the probability of the parameters given the data (or often its logarithm) so that larger values represent closer agreement.

In either case, the parameters of the model are then adjusted to find a happy extremum in the merit function, yielding *best-fit parameters*. The adjustment process is thus a problem in minimization in many dimensions. This optimization was the subject of Chapter 10; however, there exist special, more efficient, methods that are specific to modeling, and we will discuss these in this chapter.

There are important issues that go beyond the mere finding of best-fit parameters. Data are generally not exact. They are subject to *measurement errors* (called *noise* in the context of signal processing). Thus, typical data never exactly fit the model that is being used, even when that model is correct. We need the means to assess whether or not the model is appropriate, that is, we need to test the *goodness-of-fit* against some useful statistical standard.

We usually also need to know the accuracy with which parameters are determined by the data set. In frequentist terms, we need to know the standard errors of the best-fit parameters. Alternatively, in Bayesian language, we want to find not just the peak of the joint parameter probability distribution, but the whole distribution.

Or we at least want to be able to sample from that distribution, typically by Markov chain Monte Carlo, as we will discuss at length in §15.8.

It is not uncommon in fitting data to discover that the merit function is not unimodal, with a single minimum. In some cases, we may be interested in global rather than local questions. Not, "how good is this fit?" but rather, "how sure am I that there is not a *very much better* fit in some corner of parameter space?" As we have seen in Chapter 10, especially §10.12, this kind of problem is generally quite difficult to solve.

The important message is that fitting of parameters is not the end-all of model parameter estimation. To be genuinely useful, a fitting procedure should provide (i) parameters, (ii) error estimates on the parameters or a way to sample from their probability distribution, and (iii) a statistical measure of goodness-of-fit. When the third item suggests that the model is an unlikely match to the data, then items (i) and (ii) are probably worthless. Unfortunately, many practitioners of parameter estimation never proceed beyond item (i). They deem a fit acceptable if a graph of data and model "looks good." This approach is known as *chi-by-eye*. Luckily, its practitioners get what they deserve.

15.0.1 Basic Bayes

Because the discussion in this and subsequent chapters will move freely between frequentist and Bayesian methods, this is a good place to compare these two powerfully useful ways of thinking. In §14.0, when we discussed tail, or p-value, tests, we were adopting a frequentist viewpoint. The central frequentist idea is that, given the details of a null hypothesis, there is an implied population (that is, probability distribution) of possible data sets. If the assumed null hypothesis is correct, then the actual, measured, data set is drawn from that population. (We expand on this in §15.6.) It then makes sense to ask questions about how "frequently" some aspect of the measured data occurs in the population. If the answer is "very infrequently," then the hypothesis is rejected. The frequentist viewpoint avoids questions like, "what is the *probability* that this hypothesis is true?" because its focus is on the distribution of data sets, not hypotheses. Indeed, whether by dogma or merely benign neglect, it eschews the machinery needed to handle the concept of a probability distribution of hypotheses.

That machinery is Bayes' theorem, which follows from the standard axioms of probability. Bayes' theorem relates the conditional probabilities of two events, say A and B:

$$P(A|B) = P(A)\,\frac{P(B|A)}{P(B)} \tag{15.0.1}$$

Here $P(A|B)$ is the probability of A *given* that B has occurred, and similarly for $P(B|A)$, while $P(A)$ and $P(B)$ are unconditional probabilities.

Bayesians allow a broader set of uses for probabilities than frequentists. To a Bayesian, $P(A|B)$ is a measure of the degree of plausibility of A (given B) on a scale ranging from zero to one. In this broader view, A and B need not be repeatable events; they can indeed be propositions or hypotheses. In equation (15.0.1), A might be a hypothesis and B might be some data, so that $P(A|B)$ expresses the probability of a hypothesis, given the data. The equations of probability theory thus become a set of consistent rules for conducting inference [1,2]. Interestingly, this viewpoint was

universal before the 20th century. The Bernoullis (both of them), Laplace, Gauss, Legendre, and Poisson, among others, made little or no distinction between inference and probability. An opposing frequentist view, that these concepts should be kept separate, became explicit only with the work of Fisher, Box, Kendall, Neyman, and Pearson (among others), much later.

Since plausibility is itself always conditioned on some, perhaps unarticulated, set of assumptions, all Bayesian probabilities are viewed as conditional on some collective background information I. Suppose H is some hypothesis. Even before there exist any explicit data, a Bayesian can assign to H some degree of plausibility $P(H|I)$, called the "Bayesian prior." Now, when some data D_1 comes along, Bayes theorem tells how to reassess the plausibility of H,

$$P(H|D_1 I) = P(H|I)\frac{P(D_1|HI)}{P(D_1|I)} \tag{15.0.2}$$

The factor in the numerator on the right of equation (15.0.2) is calculable as the probability of a data set *given* the hypothesis (comparable to "likelihood" as we will define it in §15.1). The denominator, called the "prior predictive probability" of the data, is in this case merely a normalization constant that can be calculated by the requirement that the probability of all hypotheses should sum to unity. (In other Bayesian contexts, the prior predictive probabilities of two qualitatively different models can be used to assess their relative plausibility.)

If some additional data D_2 come along tomorrow, we can further refine our estimate of H's probability, as

$$P(H|D_2 D_1 I) = P(H|D_1 I)\frac{P(D_2|HD_1 I)}{P(D_2|D_1 I)} \tag{15.0.3}$$

Using the product rule for probabilities, $P(AB|C) = P(A|C)P(B|AC)$, we find that equations (15.0.2) and (15.0.3) imply

$$P(H|D_2 D_1 I) = P(H|I)\frac{P(D_2 D_1|HI)}{P(D_2 D_1|I)} \tag{15.0.4}$$

which shows that we would have gotten the same answer if all the data $D_1 D_2$ had been taken together.

We might wonder, before we adopt the laws of probability as our calculus of inference and thus become Bayesians, whether there are any other alternatives. The answer is, basically, no. Cox [3] showed that making a small number of very reasonable assumptions about "degree of belief" leads inevitably to the axioms of probability, and thus the application of Bayes theorem to the evaluation of hypotheses, given data. Either you become a Bayesian or else you must live in a world with no general calculus of inference.

CITED REFERENCES AND FURTHER READING:

Bevington, P.R., and Robinson, D.K. 2002, *Data Reduction and Error Analysis for the Physical Sciences*, 3rd ed. (New York: McGraw-Hill), Chapters 6–11.

Devore, J.L. 2003, *Probability and Statistics for Engineering and the Sciences*, 6th ed. (Belmont, CA: Duxbury Press), Chapters 12–13.

Brownlee, K.A. 1965, *Statistical Theory and Methodology*, 2nd ed. (New York: Wiley).

Martin, B.R. 1971, *Statistics for Physicists* (New York: Academic Press).

Gelman, A., Carlin, J.B., Stern, H.S., and Rubin, D.B. 2004, *Bayesian Data Analysis*, 2nd ed. (Boca Raton, FL: Chapman & Hall/CRC).

Sivia, D.S. 1996, *Data Analysis: A Bayesian Tutorial* (Oxford, UK: Oxford University Press).

Jaynes, E.T. 1976, in *Foundations of Probability Theory, Statistical Inference, and Statistical Theories of Science*, W.L. Harper and C.A. Hooker, eds. (Dordrecht: Reidel).[1]

Jaynes, E.T. 1985, in *Maximum-Entropy and Bayesian Methods in Inverse Problems*, C.R. Smith and W.T. Grandy, Jr., eds. (Dordrecht: Reidel).[2]

Cox, R.T. 1946, "Probability, Frequency, and Reasonable Expectation," *American Journal of Physics*, vol. 14, pp. 1–13.[3]

15.1 Least Squares as a Maximum Likelihood Estimator

Suppose that we are fitting N data points (x_i, y_i), $i = 0, \ldots, N-1$, to a model that has M adjustable parameters a_j, $j = 0, \ldots, M-1$. The model predicts a functional relationship between the measured independent and dependent variables,

$$y(x) = y(x|a_0 \ldots a_{M-1}) \tag{15.1.1}$$

where the notation indicates dependence on the parameters explicitly on the right-hand side, following the vertical bar.

What, exactly, do we want to minimize to get fitted values for the a_j's? The first thing that comes to mind is the familiar least-squares fit,

$$\text{minimize over } a_0 \ldots a_{M-1} : \quad \sum_{i=0}^{N-1} [y_i - y(x_i|a_0 \ldots a_{M-1})]^2 \tag{15.1.2}$$

But where does this come from? What general principles is it based on?

To answer these questions, let us start by asking, "*Given a particular set of parameters*, what is the probability that the observed data set should have occurred?" If the y_i's take on continuous values, the probability will always be zero unless we add the phrase, "...plus or minus some small, fixed Δy on each data point." So let's always take this phrase as understood. If the probability of obtaining the data set is too small, then we can conclude that the parameters under consideration are "unlikely" to be right. Conversely, our intuition tells us that the data set should not be too improbable for the correct choice of parameters.

To be more quantitative, suppose that each data point y_i has a measurement error that is independently random and distributed as a normal (Gaussian) distribution around the "true" model $y(x)$. And suppose that the standard deviations σ of these normal distributions are the same for all points. Then the probability of the data set is the product of the probabilities of each point:

$$P(\text{data} \mid \text{model}) \propto \prod_{i=0}^{N-1} \left\{ \exp\left[-\frac{1}{2} \left(\frac{y_i - y(x_i)}{\sigma} \right)^2 \right] \Delta y \right\} \tag{15.1.3}$$

Notice that there is a factor Δy in each term in the product.

If we are Bayesians, we proceed by invoking Bayes' theorem, in the form

$$P(\text{model} \mid \text{data}) \propto P(\text{data} \mid \text{model})\, P(\text{model}) \tag{15.1.4}$$

where $P(\text{model}) = P(a_0 \ldots a_{M-1})$ is our prior probability distribution on all models. As often as not, we take a constant, *noninformative* prior. The most probable model, then, is the one that maximizes equation (15.1.3) or, equivalently, minimizes the negative of its logarithm,

$$\left[\sum_{i=0}^{N-1} \frac{[y_i - y(x_i)]^2}{2\sigma^2} \right] - N \log \Delta y \tag{15.1.5}$$

Since N, σ, and Δy are all constants, minimizing this equation is equivalent to minimizing (15.1.2).

If we are frequentists, we must get to the same destination by a more tortuous path (as is so often the case when Bayesian and frequentist methods coincide). We are not allowed to think about the notion of probability as applied to parameter sets, because, for frequentists, there is no statistical universe of models from which the parameters are drawn. We instead substitute a dictum: We identify the probability of the data given the parameters (which is computable as above), as the *likelihood* of the parameters given the data. This identification is entirely based on intuition. It has no formal mathematical basis in and of itself. Parameters derived in this way are called *maximum likelihood estimators*.

What we see is that least-squares fitting gives an answer that is both (i) the most probable parameter set in the Bayesian sense, assuming a flat prior, and (ii) the maximum likelihood estimate of the fitted parameters, in both cases *if* the measurement errors are independent and normally distributed with constant standard deviation. Notice that we made no assumption about the linearity or nonlinearity of the model $y(x|a_0 \ldots)$ in its parameters $a_0 \ldots a_{M-1}$. Just below, we will relax our assumption of constant standard deviations and obtain the very similar formulas for what is called "chi-square fitting" or "weighted least-squares fitting." First, however, let us discuss further our very stringent assumption of a normal distribution.

For a hundred years or so, mathematical statisticians have been in love with the fact that the probability distribution of the sum of a very large number of very small random deviations almost always converges to a normal distribution. (For precise statements of this *central limit theorem*, consult [1] or other standard works on mathematical statistics.) This infatuation tended to focus interest away from the fact that, for real data, the normal distribution is often rather poorly realized, if it is realized at all. We are often taught, rather casually, that, on average, measurements will fall within $\pm\sigma$ of the true value 68% of the time, within $\pm 2\sigma$ 95% of the time, and within $\pm 3\sigma$ 99.7% of the time. Extending this, one would expect a measurement to be off by $\pm 20\sigma$ only one time out of 2×10^{88}. We all know that "glitches" are much more likely than *that*!

In some instances, the deviations from a normal distribution are easy to understand and quantify. For example, in measurements obtained by counting events, the measurement errors are usually distributed as a Poisson distribution, whose cumulative probability function was already discussed in §6.2. When the number of counts going into one data point is large, the Poisson distribution converges toward

a Gaussian. However, the convergence is not uniform when measured in fractional accuracy. The more standard deviations out on the tail of the distribution, the larger the number of counts must be before a value close to the Gaussian is realized. The sign of the effect is always the same: The Gaussian predicts that "tail" events are much less likely than they actually (by Poisson) are. This causes such events, when they occur, to skew a least-squares fit much more than they ought.

Other times, the deviations from a normal distribution are not so easy to understand in detail. Experimental points are occasionally just *way off*. Perhaps the power flickered during a point's measurement, or someone kicked the apparatus, or someone wrote down a wrong number. Points like this are called *outliers*. They can easily turn a least-squares fit on otherwise adequate data into nonsense. Their probability of occurrence in the assumed Gaussian model is so small that the maximum likelihood estimator is willing to distort the whole curve to try to bring them, mistakenly, into line.

The subject of *robust statistics* deals with cases where the normal or Gaussian model is a bad approximation, or cases where outliers are important. We will discuss robust methods briefly in §15.7. All the sections between this one and that one assume, one way or the other, a Gaussian model for the measurement errors in the data. It it quite important that you keep the limitations of that model in mind, even as you use the very useful methods that follow from assuming it.

Finally, note that our discussion of measurement errors has been limited to *statistical* errors, the kind that will average away if we only take enough data. Measurements are also susceptible to *systematic* errors that will not go away with any amount of averaging. For example, the calibration of a metal meter stick might depend on its temperature. If we take all our measurements at the same wrong temperature, then no amount of averaging or numerical processing will correct for this unrecognized systematic error.

15.1.1 Chi-Square Fitting

We considered the chi-square statistic once before, in §14.3. Here it arises in a slightly different context.

If each data point (x_i, y_i) has its own, known standard deviation σ_i, then equation (15.1.3) is modified only by putting a subscript i on the symbol σ. That subscript also propagates docilely into (15.1.5), so that the maximum likelihood estimate of the model parameters (and also the Bayesian most probable parameter set) is obtained by minimizing the quantity

$$\chi^2 \equiv \sum_{i=0}^{N-1} \left(\frac{y_i - y(x_i|a_0 \ldots a_{M-1})}{\sigma_i} \right)^2 \tag{15.1.6}$$

called the "chi-square."

To whatever extent the measurement errors actually *are* normally distributed, the quantity χ^2 is correspondingly a sum of N squares of normally distributed quantities, each normalized to unit variance. Once we have adjusted the $a_0 \ldots a_{M-1}$ to minimize the value of χ^2, the terms in the sum are not all statistically independent. For models that are linear in the a's, however, it turns out that the probability distribution for different values of χ^2 at its minimum can nevertheless be derived analytically, and is the *chi-square distribution for* $N - M$ *degrees of freedom*. We

learned how to compute this probability function using the incomplete gamma function in §6.2. In particular, equation (6.14.39) gives the probability Q that the chi-square should exceed a particular value χ^2 by chance, where $\nu = N - M$ is the *number of degrees of freedom*. The quantity Q, or its complement $P \equiv 1 - Q$, is frequently tabulated in appendices to statistics books, or it can be computed as $P = \texttt{Chisqdist}(\nu)\texttt{.invcdf}(\chi^2)$ by the routine in §6.14.8. It is quite common, and usually not too wrong, to assume that the chi-square distribution holds even for models that are not strictly linear in the a's.

This computed probability gives a quantitative measure for the goodness-of-fit of the model. If Q is a very small probability for some particular data set, then the apparent discrepancies are unlikely to be chance fluctuations. Much more probably either (i) the model is wrong — can be statistically rejected, or (ii) someone has lied to you about the size of the measurement errors σ_i — they are really larger than stated.

While above we were quick to poke fun at the frequentist's foundations for maximum likelihood estimation (or lack thereof), we must now take aim at strict Bayesians: There are no good fully Bayesian methods for assessing goodness-of-fit, that is, for comparing the probability of a best-fit model to that of a nonspecific alternative hypothesis like "the model is wrong." The problem is that the strict application of Bayes theorem requires either (i) a comparison between two well-posed hypotheses (the *odds ratio*), or (ii) a normalization of the probability of the best-fit model against an integral of such probabilities over all possible models (the *normalizing constant*). In most situations neither of these is available. Sensible Bayesians usually fall back to p-value tail statistics like chi-square probability when they really need to know if a model is wrong.

Another important point is that the chi-square probability Q does not directly measure the credibility of the assumption that the measurement errors are normally distributed. It assumes they are. In most, but not all, cases, however, the effect of nonnormal errors is to create an abundance of outlier points. These decrease the probability Q, so that we can add another possible, though less definitive, conclusion to the above list: (iii) the measurement errors may not be normally distributed.

Possibility (iii) is fairly common, and also fairly benign. It is for this reason that reasonable experimenters are often rather tolerant of low probabilities Q. It is not uncommon to deem acceptable on equal terms any models with, say, $Q > 0.001$. This is not as sloppy as it sounds: Truly *wrong* models will often be rejected with vastly smaller values of Q, 10^{-18}, say. However, if day-in and day-out you find yourself accepting models with $Q \sim 10^{-3}$, you really should track down the cause.

If you happen to know the actual distribution law of your measurement errors, then you might wish to *Monte Carlo simulate* some data sets drawn from a particular model, cf. §7.3 – §7.4. You can then subject these synthetic data sets to your actual fitting procedure, so as to determine both the probability distribution of the χ^2 statistic and also the accuracy with which your model parameters are reproduced by the fit. We discuss this further in §15.6. The technique is very general, but it can also be slow.

At the opposite extreme, it sometimes happens that the probability Q is too large, too near to 1, literally too good to be true! Nonnormal measurement errors cannot in general produce this disease, since the normal distribution is about as "compact" as a distribution can be. Almost always, the cause of too good a chi-

square fit is that the experimenter, in a "fit" of conservativism, has *overestimated* his or her measurement errors. Very rarely, too good a chi-square signals actual fraud, data that have been "fudged" to fit the model.

A rule of thumb is that a "typical" value of χ^2 for a "moderately" good fit is $\chi^2 \approx \nu$. More precise is the statement that the χ^2 statistic has a mean ν and a standard deviation $\sqrt{2\nu}$ and, asymptotically for large ν, becomes normally distributed.

In some cases the uncertainties associated with a set of measurements are not known in advance, and considerations related to χ^2 fitting are used to derive a value for σ. If we assume that all measurements have the same standard deviation, $\sigma_i = \sigma$, and that the model does fit well, then we can proceed by first assigning an arbitrary constant σ to all points, next fitting for the model parameters by minimizing χ^2, and finally recomputing

$$\sigma^2 = \sum_{i=0}^{N-1} [y_i - y(x_i)]^2/(N-M) \tag{15.1.7}$$

Obviously, this approach prohibits an independent assessment of goodness-of-fit, a fact occasionally missed by its adherents. When, however, the measurement error is not known, this approach at least allows *some* kind of error bar to be assigned to the points.

If we take the derivative of equation (15.1.6) with respect to the parameters a_k, we obtain equations that must hold at the chi-square minimum:

$$0 = \sum_{i=0}^{N-1} \left(\frac{y_i - y(x_i)}{\sigma_i^2} \right) \left(\frac{\partial y(x_i|\ldots a_k \ldots)}{\partial a_k} \right) \qquad k = 0, \ldots, M-1 \tag{15.1.8}$$

Equation (15.1.8) is, in general, a set of M nonlinear equations for the M unknown a_k. Various of the procedures described subsequently in this chapter derive from (15.1.8) and its specializations.

CITED REFERENCES AND FURTHER READING:

Lupton, R. 1993, *Statistics in Theory and Practice* (Princeton, NJ: Princeton University Press), Chapters 10–11.[1]

Devore, J.L. 2003, *Probability and Statistics for Engineering and the Sciences*, 6th ed. (Belmont, CA: Duxbury Press), Chapter 6.

Gelman, A., Carlin, J.B., Stern, H.S., and Rubin, D.B. 2004, *Bayesian Data Analysis*, 2nd ed. (Boca Raton, FL: Chapman & Hall/CRC), Chapter 8.

15.2 Fitting Data to a Straight Line

A concrete example will make the considerations of the previous section more meaningful. We consider the problem of fitting a set of N data points (x_i, y_i) to a straight-line model

$$y(x) = y(x|a,b) = a + bx \tag{15.2.1}$$

This problem is often called *linear regression*, a terminology that originated, long ago, in the social sciences. We assume that the uncertainty σ_i associated with each

measurement y_i is known, and that the x_i's (values of the dependent variable) are known exactly.

To measure how well the model agrees with the data, we use the chi-square merit function (15.1.6), which in this case is

$$\chi^2(a,b) = \sum_{i=0}^{N-1} \left(\frac{y_i - a - bx_i}{\sigma_i} \right)^2 \tag{15.2.2}$$

If the measurement errors are normally distributed, then this merit function will give maximum likelihood parameter estimations of a and b; if the errors are not normally distributed, then the estimations are not maximum likelihood but may still be useful in a practical sense. In §15.7, we will treat the case where outlier points are so numerous as to render the χ^2 merit function useless.

Equation (15.2.2) is minimized to determine a and b. At its minimum, derivatives of $\chi^2(a,b)$ with respect to a, b vanish:

$$\begin{aligned} 0 &= \frac{\partial \chi^2}{\partial a} = -2 \sum_{i=0}^{N-1} \frac{y_i - a - bx_i}{\sigma_i^2} \\ 0 &= \frac{\partial \chi^2}{\partial b} = -2 \sum_{i=0}^{N-1} \frac{x_i (y_i - a - bx_i)}{\sigma_i^2} \end{aligned} \tag{15.2.3}$$

These conditions can be rewritten in a convenient form if we define the following sums:

$$\begin{aligned} S &\equiv \sum_{i=0}^{N-1} \frac{1}{\sigma_i^2} \quad S_x \equiv \sum_{i=0}^{N-1} \frac{x_i}{\sigma_i^2} \quad S_y \equiv \sum_{i=0}^{N-1} \frac{y_i}{\sigma_i^2} \\ S_{xx} &\equiv \sum_{i=0}^{N-1} \frac{x_i^2}{\sigma_i^2} \quad S_{xy} \equiv \sum_{i=0}^{N-1} \frac{x_i y_i}{\sigma_i^2} \end{aligned} \tag{15.2.4}$$

With these definitions (15.2.3) becomes

$$\begin{aligned} aS + bS_x &= S_y \\ aS_x + bS_{xx} &= S_{xy} \end{aligned} \tag{15.2.5}$$

The solution of these two equations in two unknowns is calculated as

$$\begin{aligned} \Delta &\equiv S S_{xx} - (S_x)^2 \\ a &= \frac{S_{xx} S_y - S_x S_{xy}}{\Delta} \\ b &= \frac{S S_{xy} - S_x S_y}{\Delta} \end{aligned} \tag{15.2.6}$$

Equation (15.2.6) gives the solution for the best-fit model parameters a and b.

We are not done, however. We must estimate the probable uncertainties in the estimates of a and b, since obviously the measurement errors in the data must introduce some uncertainty in the determination of those parameters. If the data are

independent, then each contributes its own bit of uncertainty to the parameters. Consideration of propagation of errors shows that the variance σ_f^2 in the value of any function will be

$$\sigma_f^2 = \sum_{i=0}^{N-1} \sigma_i^2 \left(\frac{\partial f}{\partial y_i} \right)^2 \tag{15.2.7}$$

For the straight line, the derivatives of a and b with respect to y_i can be directly evaluated from the solution:

$$\begin{aligned} \frac{\partial a}{\partial y_i} &= \frac{S_{xx} - S_x x_i}{\sigma_i^2 \Delta} \\ \frac{\partial b}{\partial y_i} &= \frac{S x_i - S_x}{\sigma_i^2 \Delta} \end{aligned} \tag{15.2.8}$$

Summing over the points as in (15.2.7), we get

$$\begin{aligned} \sigma_a^2 &= S_{xx}/\Delta \\ \sigma_b^2 &= S/\Delta \end{aligned} \tag{15.2.9}$$

which are the variances in the estimates of a and b, respectively. We will see in §15.6 that an additional number is also needed to characterize properly the probable uncertainty of the parameter estimation. That number is the *covariance* of a and b, and (as we will see below) is given by

$$\mathrm{Cov}(a, b) = -S_x/\Delta \tag{15.2.10}$$

The coefficient of correlation between the uncertainty in a and the uncertainty in b, which is a number between -1 and 1, follows from (15.2.10) (compare equation 14.5.1),

$$r_{ab} = \frac{-S_x}{\sqrt{S S_{xx}}} \tag{15.2.11}$$

A positive value of r_{ab} indicates that the errors in a and b are likely to have the same sign, while a negative value indicates the errors are anticorrelated, likely to have opposite signs.

We are *still* not done. We must estimate the goodness-of-fit of the data to the model. Absent this estimate, we have not the slightest indication that the parameters a and b in the model have any meaning at all! The probability Q that a value of chi-square as *poor* as the value (15.2.2) should occur by chance is

$$Q = 1 - \texttt{Chisqdist}(N-2).\texttt{invcdf}\,(\chi^2) \tag{15.2.12}$$

Here `Chisqdist` is our object embodying the chi-square distribution function (see §6.14.8) and `invcdf` is its inverse cumulative distribution function. If Q is larger than, say, 0.1, then the goodness-of-fit is believable. If it is larger than, say, 0.001, then the fit *may* be acceptable if the errors are nonnormal or have been moderately underestimated. If Q is less than 0.001, then the model and/or estimation procedure can rightly be called into question. In this latter case, turn to §15.7 to proceed further.

If you do not know the individual measurement errors of the points σ_i, and are proceeding (dangerously) to use equation (15.1.7) for estimating these errors, then

here is the procedure for estimating the probable uncertainties of the parameters a and b: Set $\sigma_i \equiv 1$ in all equations through (15.2.6), and multiply σ_a and σ_b, as obtained from equation (15.2.9), by the additional factor $\sqrt{\chi^2/(N-2)}$, where χ^2 is computed by (15.2.2) using the fitted parameters a and b. As discussed above, this procedure is equivalent to *assuming* a good fit, so you get no independent goodness-of-fit probability Q.

In §14.5 we promised a relation between the linear correlation coefficient r (equation 14.5.1) and a goodness-of-fit measure, χ^2 (equation 15.2.2). For unweighted data (all $\sigma_i = 1$), that relation is

$$\chi^2 = (1-r^2)\sum_{i=0}^{N-1}(y_i - \bar{y})^2 \tag{15.2.13}$$

For data with varying errors σ_i, the above equations remain valid if the sums in equations (15.2.13) and (14.5.1) are weighted by $1/\sigma_i^2$.

The following object, `Fitab`, carries out exactly the operations that we have discussed. You call its constructor either with, or without, errors σ_i. If the σ_i's are known, the calculations exactly correspond to the formulas above. However, when σ_i's are unavailable, the routine *assumes* equal values of σ for each point and *assumes* a good fit, as discussed in §15.1.

The formulas (15.2.6) are susceptible to roundoff error. Accordingly, we rewrite them as follows: Define

$$t_i = \frac{1}{\sigma_i}\left(x_i - \frac{S_x}{S}\right), \qquad i = 0, 1, \ldots, N-1 \tag{15.2.14}$$

and

$$S_{tt} = \sum_{i=0}^{N-1} t_i^2 \tag{15.2.15}$$

Then, as you can verify by direct substitution,

$$b = \frac{1}{S_{tt}}\sum_{i=0}^{N-1}\frac{t_i y_i}{\sigma_i} \tag{15.2.16}$$

$$a = \frac{S_y - S_x b}{S} \tag{15.2.17}$$

$$\sigma_a^2 = \frac{1}{S}\left(1 + \frac{S_x^2}{S S_{tt}}\right) \tag{15.2.18}$$

$$\sigma_b^2 = \frac{1}{S_{tt}} \tag{15.2.19}$$

$$\mathrm{Cov}(a,b) = -\frac{S_x}{S S_{tt}} \tag{15.2.20}$$

$$r_{ab} = \frac{\mathrm{Cov}(a,b)}{\sigma_a \sigma_b} \tag{15.2.21}$$

fitab.h

```
struct Fitab {
```

Object for fitting a straight line $y = a + bx$ to a set of points (x_i, y_i), with or without available errors σ_i. Call one of the two constructors to calculate the fit. The answers are then available as the variables a, b, siga, sigb, chi2, and either q or sigdat.

```
    Int ndata;
    Doub a, b, siga, sigb, chi2, q, sigdat;          Answers.
    VecDoub_I &x, &y, &sig;

    Fitab(VecDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig)
    : ndata(xx.size()), x(xx), y(yy), sig(ssig), chi2(0.), q(1.), sigdat(0.)  {
```

Constructor. Given a set of data points x[0..ndata-1], y[0..ndata-1] with individual standard deviations sig[0..ndata-1], sets a,b and their respective probable uncertainties siga and sigb, the chi-square chi2, and the goodness-of-fit probability q (that the fit would have χ^2 this large or larger).

```
        Gamma gam;
        Int i;
        Doub ss=0.,sx=0.,sy=0.,st2=0.,t,wt,sxoss;
        b=0.0;                                       Accumulate sums ...
        for (i=0;i<ndata;i++) {
            wt=1.0/SQR(sig[i]);                      ...with weights
            ss += wt;
            sx += x[i]*wt;
            sy += y[i]*wt;
        }
        sxoss=sx/ss;
        for (i=0;i<ndata;i++) {
            t=(x[i]-sxoss)/sig[i];
            st2 += t*t;
            b += t*y[i]/sig[i];
        }
        b /= st2;                                    Solve for a, b, σa, and σb.
        a=(sy-sx*b)/ss;
        siga=sqrt((1.0+sx*sx/(ss*st2))/ss);
        sigb=sqrt(1.0/st2);                          Calculate χ2.
        for (i=0;i<ndata;i++) chi2 += SQR((y[i]-a-b*x[i])/sig[i]);
        if (ndata>2) q=gam.gammq(0.5*(ndata-2),0.5*chi2);    Equation (15.2.12).
    }

    Fitab(VecDoub_I &xx, VecDoub_I &yy)
    : ndata(xx.size()), x(xx), y(yy), sig(xx), chi2(0.), q(1.), sigdat(0.) {
```

Constructor. As above, but without known errors (sig is not used). The uncertainties siga and sigb are estimated by assuming equal errors for all points, and that a straight line is a good fit. q is returned as 1.0, the normalization of chi2 is to unit standard deviation on all points, and sigdat is set to the estimated error of each point.

```
        Int i;
        Doub ss,sx=0.,sy=0.,st2=0.,t,sxoss;
        b=0.0;                                       Accumulate sums ...
        for (i=0;i<ndata;i++) {
            sx += x[i];                              ...without weights.
            sy += y[i];
        }
        ss=ndata;
        sxoss=sx/ss;
        for (i=0;i<ndata;i++) {
            t=x[i]-sxoss;
            st2 += t*t;
            b += t*y[i];
        }
        b /= st2;                                    Solve for a, b, σa, and σb.
        a=(sy-sx*b)/ss;
        siga=sqrt((1.0+sx*sx/(ss*st2))/ss);
        sigb=sqrt(1.0/st2);                          Calculate χ2.
        for (i=0;i<ndata;i++) chi2 += SQR(y[i]-a-b*x[i]);
```

```
        if (ndata > 2) sigdat=sqrt(chi2/(ndata-2));
        siga *= sigdat;
        sigb *= sigdat;
    }
};
```

For unweighted data evaluate typical sig using chi2, and adjust the standard deviations.

CITED REFERENCES AND FURTHER READING:

Bevington, P.R., and Robinson, D.K. 2002, *Data Reduction and Error Analysis for the Physical Sciences*, 3rd ed. (New York: McGraw-Hill), Chapter 6.

Devore, J.L. 2003, *Probability and Statistics for Engineering and the Sciences*, 6th ed. (Belmont, CA: Duxbury Press), Chapter 12.

15.3 Straight-Line Data with Errors in Both Coordinates

If experimental data are subject to measurement error not only in the y_i's, but also in the x_i's, then the task of fitting a straight-line model

$$y(x) = a + bx \tag{15.3.1}$$

is considerably harder. It is straightforward to write down the χ^2 merit function for this case,

$$\chi^2(a,b) = \sum_{i=0}^{N-1} \frac{(y_i - a - bx_i)^2}{\sigma_{y\,i}^2 + b^2\sigma_{x\,i}^2} \tag{15.3.2}$$

where $\sigma_{x\,i}$ and $\sigma_{y\,i}$ are, respectively, the x and y standard deviations for the ith point. The weighted sum of variances in the denominator of equation (15.3.2) can be understood both as the variance in the direction of the smallest χ^2 between each data point and the line with slope b, and also as the variance of the linear combination $y_i - a - bx_i$ of two random variables x_i and y_i,

$$\mathrm{Var}(y_i - a - bx_i) = \mathrm{Var}(y_i) + b^2\mathrm{Var}(x_i) = \sigma_{y\,i}^2 + b^2\sigma_{x\,i}^2 \equiv 1/w_i \tag{15.3.3}$$

The sum of the square of N random variables, each normalized by its variance, is thus chi-square distributed.

We want to minimize equation (15.3.2) with respect to a and b. Unfortunately, the occurrence of b in the denominator of equation (15.3.2) makes the resulting equation for the slope $\partial\chi^2/\partial b = 0$ nonlinear. However, the corresponding condition for the intercept, $\partial\chi^2/\partial a = 0$, is still linear and yields

$$a = \left[\sum_i w_i(y_i - bx_i)\right] \Big/ \sum_i w_i \tag{15.3.4}$$

where the w_i's are defined by equation (15.3.3). A reasonable strategy, now, is to use the machinery of Chapter 10 (e.g., a `Brent` object) for minimizing a general one-dimensional function to minimize with respect to b while using equation (15.3.4) at each stage to ensure that the minimum with respect to b is also minimized with respect to a.

Because of the finite error bars on the x_i's, the minimum χ^2 as a function of b will be finite, though usually large, when b equals infinity (line of infinite slope). The angle $\theta \equiv$

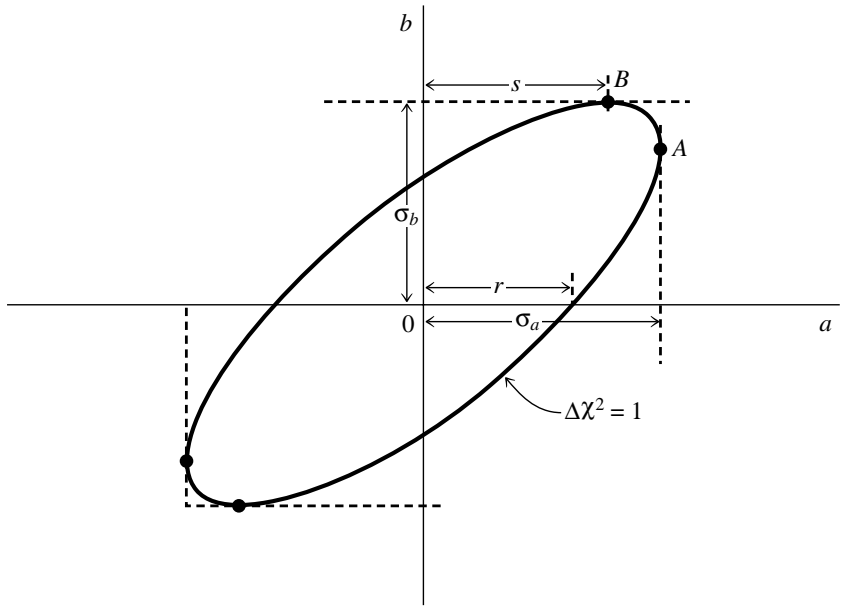


Figure 15.3.1. Standard errors for the parameters a and b. The point B can be found by varying the slope b while simultaneously minimizing the intercept a. This gives the standard error σ_b and also the value s. The standard error σ_a can then be found by the geometric relation $\sigma_a^2 = s^2 + r^2$.

arctan b is thus more suitable as a parametrization of slope than b itself. The value of χ^2 will then be periodic in θ with period π (not 2π!). If any data points have very small σ_y's but moderate or large σ_x's, then it is also possible to have a maximum in χ^2 near zero slope, $\theta \approx 0$. In that case, there can sometimes be two χ^2 minima, one at positive slope and the other at negative. Only one of these is the correct global minimum. It is therefore important to have a good starting guess for b (or θ). Our strategy, implemented below, is to scale the y_i's so as to have variance equal to the x_i's, and then to do a conventional (as in §15.2) linear fit with weights derived from the (scaled) sum $\sigma_{y\,i}^2 + \sigma_{x\,i}^2$. This yields a good starting guess for b if the data are even *plausibly* related to a straight-line model.

Finding the standard errors σ_a and σ_b on the parameters a and b is more complicated. We will see in §15.6 that, in appropriate circumstances, the standard errors in a and b are the respective projections onto the a- and b-axes of the "confidence region boundary" where χ^2 takes on a value one greater than its minimum, $\Delta\chi^2 = 1$. In the linear case of §15.2, these projections follow from the Taylor series expansion

$$\Delta\chi^2 \approx \frac{1}{2}\left[\frac{\partial^2\chi^2}{\partial a^2}(\Delta a)^2 + \frac{\partial^2\chi^2}{\partial b^2}(\Delta b)^2\right] + \frac{\partial^2\chi^2}{\partial a\,\partial b}\Delta a\,\Delta b \tag{15.3.5}$$

Because of the present nonlinearity in b, however, analytic formulas for the second derivatives are quite unwieldy; more important, the lowest-order term frequently gives a poor approximation to $\Delta\chi^2$. Our strategy is therefore to find the roots of $\Delta\chi^2 = 1$ numerically, by adjusting the value of the slope b away from the minimum. In the program below, the general root finder `zbrent` is used. It may occur that there are no roots at all — for example, if all error bars are so large that all the data points are compatible with each other. It is important, therefore, to make some effort at bracketing a putative root before refining it (cf. §9.1).

Because a is minimized at each stage of varying b, successful numerical root finding leads to a value of Δa that minimizes χ^2 for the value of Δb that gives $\Delta\chi^2 = 1$. This (see Figure 15.3.1) directly gives the tangent projection of the confidence region onto the b-axis, and thus σ_b. It does not, however, give the tangent projection of the confidence region onto

the a-axis. In the figure, we have found the point labeled B; to find σ_a we need to find the point A. Geometry to the rescue: To the extent that the confidence region is approximated by an ellipse, you can prove (see figure) that $\sigma_a^2 = r^2 + s^2$. The value of s is known from having found the point B. The value of r follows from equations (15.3.2) and (15.3.3) applied at the χ^2 minimum (point O in the figure), giving

$$r^2 = 1 \Big/ \sum_i w_i \tag{15.3.6}$$

Actually, since b can go through infinity, this whole procedure makes more sense in (a, θ) space than in (a, b) space. That is, in fact, how the following program works. Since it is conventional, however, to return standard errors for a and b, not a and θ, we finally use the relation

$$\sigma_b = \sigma_\theta / \cos^2 \theta \tag{15.3.7}$$

We caution that if b and its standard error are both large, so that the confidence region actually includes infinite slope, then the standard error σ_b is not very meaningful. The functor `Chixy` is normally called only by the routine `Fitexy`. However, if you want, you can yourself explore the confidence region by making repeated calls to `Chixy` (whose argument is an angle θ, not a slope b), after a single initializing call to `Fitexy`.

Be aware that the literature on the seemingly straightforward subject of this section is generally confusing and sometimes plain wrong. Deming's [1] early treatment is sound, but its reliance on Taylor expansions gives inaccurate error estimates. References [2-4] are reliable, more recent, general treatments with critiques of earlier work. York [5] and Reed [6] usefully discuss the simple case of a straight line as treated here, but the latter paper has some errors, corrected in [7]. All this commotion has attracted the Bayesians [8-10], who have still different points of view.

A final caution, repeated from §15.0, is that if the goodness-of-fit is not acceptable (returned probability is too small), the standard errors σ_a and σ_b are surely not believable. In dire circumstances, you might try scaling all your x and y error bars by a constant factor until the probability is acceptable (0.5, say), to get more plausible values for σ_a and σ_b.

Implementing code is given in a Webnote [11].

CITED REFERENCES AND FURTHER READING:

Deming, W.E. 1943, *Statistical Adjustment of Data* (New York: Wiley), reprinted 1964 (New York: Dover).[1]

Jefferys, W.H. 1980, "On the Method of Least Squares," *Astronomical Journal*, vol. 85, pp. 177–181; see also vol. 95, p. 1299 (1988).[2]

Jefferys, W.H. 1981, "On the Method of Least Squares — Part Two," *Astronomical Journal*, vol. 86, pp. 149–155; see also vol. 95, p. 1300 (1988).[3]

Lybanon, M. 1984, "A Better Least-Squares Method When Both Variables Have Uncertainties," *American Journal of Physics*, vol. 52, pp. 22–26.[4]

York, D. 1966, "Least-Squares Fitting of a Straight Line," *Canadian Journal of Physics*, vol. 44, pp. 1079–1086.[5]

Reed, B.C. 1989, "Linear Least-Squares Fits with Error in Both Coordinates," *American Journal of Physics*, vol. 57, pp. 642–646; see also vol. 58, p. 189, and vol. 58, p. 1209.[6]

Reed, B.C. 1992, "Linear Least-squares Fits with Errors in Both Coordinates. II: Comments on Parameter Variances," *American Journal of Physics*, vol. 60, pp. 59–62.[7]

Zellner, A. 1971, *An Introduction to Bayesian Inference in Econometrics* (New York: Wiley); reprinted 1987 (Malabar, FL: R. E. Krieger).[8]

Gull, S.F. 1989, in *Maximum Entropy and Bayesian Methods*, J. Skilling, ed. (Boston: Kluwer).[9]

Jaynes, E.T. 1991, in *Maximum-Entropy and Bayesian Methods, Proceedings of the 10th International Workshop*, W.T. Grandy, Jr., and L.H. Schick, eds. (Boston: Kluwer).[10]

Macdonald, J.R., and Thompson, W.J. 1992, "Least-Squares Fitting When Both Variables Contain Errors: Pitfalls and Possibilities," *American Journal of Physics*, vol. 60, pp. 66–73.

Numerical Recipes Software 2007, "Code Implementation for Fitexy," *Numerical Recipes Webnote No. 19*, at http://www.nr.com/webnotes?19 [11]

15.4 General Linear Least Squares

An immediate generalization of §15.2 is to fit a set of data points (x_i, y_i) to a model that is not just a linear combination of 1 and x (namely $a + bx$), but rather a linear combination of *any* M specified functions of x. For example, the functions could be $1, x, x^2, \ldots, x^{M-1}$, in which case their general linear combination,

$$y(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{M-1} x^{M-1} \tag{15.4.1}$$

is a polynomial of degree $M - 1$. Or, the functions could be sines and cosines, in which case their general linear combination is a Fourier series. The general form of this kind of model is

$$y(x) = \sum_{k=0}^{M-1} a_k X_k(x) \tag{15.4.2}$$

where the quantities $X_0(x), \ldots, X_{M-1}(x)$ are arbitrary fixed functions of x, called the *basis functions*.

Note that the functions $X_k(x)$ can be wildly nonlinear functions of x. In this discussion, "linear" refers only to the model's dependence on its *parameters* a_k.

For these linear models we generalize the discussion of the previous section by defining a merit function

$$\chi^2 = \sum_{i=0}^{N-1} \left[\frac{y_i - \sum_{k=0}^{M-1} a_k X_k(x_i)}{\sigma_i} \right]^2 \tag{15.4.3}$$

As before, σ_i is the measurement error (standard deviation) of the ith data point, presumed to be known. If the measurement errors are not known, they may all (as discussed at the end of §15.1) be set to the constant value $\sigma = 1$.

Once again, we will pick as best parameters those that minimize χ^2. There are several different techniques available for finding this minimum. Two are particularly useful, and we will discuss both in this section. To introduce them and elucidate their relationship, we need some notation.

Let $\mathbf{A}$ be a matrix whose $N \times M$ components are constructed from the M basis functions evaluated at the N abscissas x_i, and from the N measurement errors σ_i, by the prescription

$$A_{ij} = \frac{X_j(x_i)}{\sigma_i} \tag{15.4.4}$$

The matrix $\mathbf{A}$ is called the *design matrix* of the fitting problem. Notice that in general $\mathbf{A}$ has more rows than columns, $N \geq M$, since there must be more data points than model parameters to be solved for. (You can fit a straight line to two points, but not a very meaningful quintic!) The design matrix is shown schematically in Figure 15.4.1.

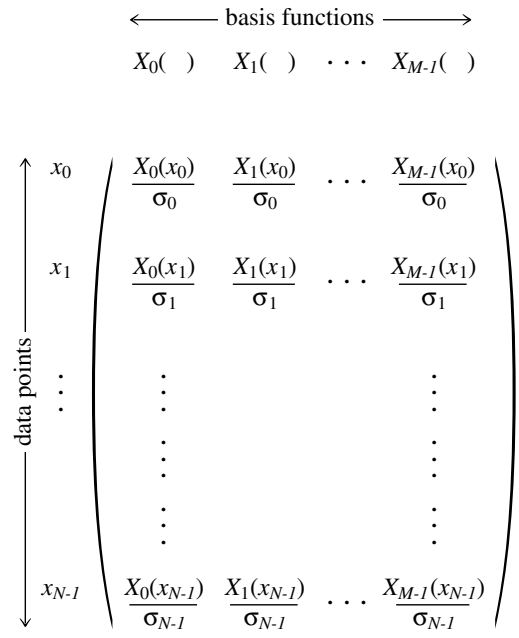


Figure 15.4.1. Design matrix **A** for the least-squares fit of a linear combination of M basis functions to N data points. The matrix elements involve the basis functions evaluated at the values of the independent variable at which measurements are made and the standard deviations of the measured dependent variable. The measured values of the dependent variable do not enter the design matrix.

Also define a vector **b** of length N by

$$b_i = \frac{y_i}{\sigma_i} \tag{15.4.5}$$

and denote the M vector whose components are the parameters to be fitted, $a_0, \ldots, a_{M-1}$, by **a**.

15.4.1 Solution by Use of the Normal Equations

The minimum of (15.4.3) occurs where the derivative of χ^2 with respect to all M parameters a_k vanishes. Specializing equation (15.1.8) to the case of the model (15.4.2), this condition yields the M equations

$$0 = \sum_{i=0}^{N-1} \frac{1}{\sigma_i^2}\left[y_i - \sum_{j=0}^{M-1} a_j X_j(x_i)\right] X_k(x_i) \qquad k = 0, \ldots, M-1 \tag{15.4.6}$$

Interchanging the order of summations, we can write (15.4.6) as the matrix equation

$$\sum_{j=0}^{M-1} \alpha_{kj} a_j = \beta_k \tag{15.4.7}$$

where

$$\alpha_{kj} = \sum_{i=0}^{N-1} \frac{X_j(x_i)X_k(x_i)}{\sigma_i^2} \qquad \text{or, equivalently,} \qquad \boldsymbol{\alpha} = \mathbf{A}^T \cdot \mathbf{A} \tag{15.4.8}$$

an $M \times M$ matrix, and

$$\beta_k = \sum_{i=0}^{N-1} \frac{y_i X_k(x_i)}{\sigma_i^2} \qquad \text{or, equivalently,} \qquad \boldsymbol{\beta} = \mathbf{A}^T \cdot \mathbf{b} \tag{15.4.9}$$

a vector of length M.

The equations (15.4.6) or (15.4.7) are called the *normal equations* of the least-squares problem. They can be solved for the vector of parameters $\mathbf{a}$ by the standard methods of Chapter 2, notably LU decomposition and backsubstitution, Choleksy decomposition, or Gauss-Jordan elimination. In matrix form, the normal equations can be written as either

$$\boldsymbol{\alpha} \cdot \mathbf{a} = \boldsymbol{\beta} \qquad \text{or as} \qquad \left(\mathbf{A}^T \cdot \mathbf{A}\right) \cdot \mathbf{a} = \mathbf{A}^T \cdot \mathbf{b} \tag{15.4.10}$$

The inverse matrix $\mathbf{C} \equiv \boldsymbol{\alpha}^{-1}$, called the covariance matrix, is closely related to the probable (or, more precisely, *standard*) uncertainties of the estimated parameters $\mathbf{a}$. To estimate these uncertainties, consider that

$$a_j = \sum_{k=0}^{M-1} \boldsymbol{\alpha}_{jk}^{-1} \beta_k = \sum_{k=0}^{M-1} C_{jk} \left[\sum_{i=0}^{N-1} \frac{y_i X_k(x_i)}{\sigma_i^2} \right] \tag{15.4.11}$$

and that the variance associated with the estimate a_j can be found as in (15.2.7) from

$$\sigma^2(a_j) = \sum_{i=0}^{N-1} \sigma_i^2 \left(\frac{\partial a_j}{\partial y_i} \right)^2 \tag{15.4.12}$$

Note that α_{jk} is independent of y_i, so that

$$\frac{\partial a_j}{\partial y_i} = \sum_{k=0}^{M-1} C_{jk} X_k(x_i)/\sigma_i^2 \tag{15.4.13}$$

Consequently, we find that

$$\sigma^2(a_j) = \sum_{k=0}^{M-1} \sum_{l=0}^{M-1} C_{jk} C_{jl} \left[\sum_{i=0}^{N-1} \frac{X_k(x_i) X_l(x_i)}{\sigma_i^2} \right] \tag{15.4.14}$$

The final term in brackets is just the matrix $\boldsymbol{\alpha}$. Since this is the matrix inverse of $\mathbf{C}$, (15.4.14) reduces immediately to

$$\sigma^2(a_j) = C_{jj} \tag{15.4.15}$$

In other words, the diagonal elements of $\mathbf{C}$ are the variances (squared uncertainties) of the fitted parameters $\mathbf{a}$. It should not surprise you to learn that the off-diagonal elements C_{jk} are the covariances between a_j and a_k (cf. 15.2.10); but we shall defer discussion of these to §15.6.

We will now give a routine that implements the above formulas for the general linear least-squares problem, by the method of normal equations. Since we wish to compute not only the solution vector **a** but also the covariance matrix **C**, it is most convenient to use Gauss-Jordan elimination (routine `gaussj` of §2.1) to perform the linear algebra. The operation count in this application is no larger than that for LU decomposition. If you have no need for the covariance matrix, however, you can save a factor of 3 on the linear algebra by switching to LU decomposition, without computation of the matrix inverse. In theory, since $\mathbf{A}^T \cdot \mathbf{A}$ is positive-definite, Cholesky decomposition is the most efficient way to solve the normal equations. However, in practice, most of the computing time is spent in looping over the data to form the equations, and Gauss-Jordan is quite adequate.

We need to warn you that the solution of a least-squares problem directly from the normal equations is rather susceptible to roundoff error, because the condition number of the matrix $\boldsymbol{\alpha}$ is the square of the condition number of **A**. An alternative, and preferred, technique involves QR decomposition (§2.10, §11.4, and §11.7) of the design matrix **A**. This is essentially what we did at the end of §15.2 for fitting data to a straight line, but without invoking all the machinery of QR to derive the necessary formulas. Later in this section, we will discuss other difficulties in the least-squares problem, for which the cure is *singular value decomposition* (SVD), of which we give an implementation. It turns out that SVD also fixes the roundoff problem, so it is our recommended technique for all but "easy" least-squares problems. It is for these easy problems that the following routine, which solves the normal equations, is intended.

The object `Fitlin`, below, has one "value-added feature" that can be quite useful in practical work: Frequently it is a matter of art to decide which parameters a_k in a model should be fit from the data set, and which should be held constant at fixed values, for example values predicted by a theory or measured in a previous experiment. One wants, therefore, to have a convenient means for "freezing" and "unfreezing" the parameters a_k. In the following code, the total number of parameters a_k is denoted `ma` (called M above) and is deduced from the size of the vector that is returned by the user-supplied fitting function routine. The `Fitlin` object maintains a boolean array `ia[0..ma-1]`. Components that are `false` indicate that you want the corresponding elements of the parameter vector `a[0..ma-1]` to be held fixed at their input values. Components that are `true` indicate parameters that should be fitted for. On output, any frozen parameters will have their variances, and all their covariances, set to zero in the covariance matrix.

fitlin.h

```
struct Fitlin {
```

Object for general linear least-squares fitting by solving the normal equations, also including the ability to hold specified parameters at fixed, specified values. Call constructor to bind data vectors and fitting functions. Then call any combination of `hold`, `free`, and `fit` as often as desired. `fit` sets the output quantities `a`, `covar`, and `chisq`.

```
    Int ndat, ma;
    VecDoub_I &x,&y,&sig;
    VecDoub (*funcs)(const Doub);
    VecBool ia;

    VecDoub a;
    MatDoub covar;
    Doub chisq;
```

Output values. `a` is the vector of fitted coefficients, `covar` is its covariance matrix, and `chisq` is the value of χ^2 for the fit.

```
    Fitlin(VecDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig, VecDoub funks(const Doub))
```

```
: ndat(xx.size()), x(xx), y(yy), sig(ssig), funcs(funks) {
```
Constructor. Binds references to the data arrays `xx`, `yy`, and `ssig`, and to a user-supplied function `funks(x)` that returns a `VecDoub` containing `ma` basis functions evaluated at $x =$ `x`. Initializes all parameters as free (not held).
```
    ma = funcs(x[0]).size();
    a.resize(ma);
    covar.resize(ma,ma);
    ia.resize(ma);
    for (Int i=0;i<ma;i++) ia[i] = true;
}

void hold(const Int i, const Doub val) {ia[i]=false; a[i]=val;}
void free(const Int i) {ia[i]=true;}
```
Optional functions for holding a parameter, identified by a value `i` in the range $0, \ldots,$ `ma-1`, fixed at the value `val`, or for freeing a parameter that was previously held fixed. `hold` and `free` may be called for any number of parameters before calling `fit` to calculate best-fit values for the remaining (not held) parameters, and the process may be repeated multiple times. Alternatively, you can set the boolean vector `ia` directly, before calling `fit`.

```
void fit() {
```
Solve the normal equations for χ^2 minimization to fit for some or all of the coefficients `a[0..ma-1]` of a function that depends linearly on `a`, $y = \sum_i$ `a`$_i \times$ `funks`$_i(x)$. Set answer values for `a[0..ma-1]`, $\chi^2 =$ `chisq`, and the covariance matrix `covar[0..ma-1][0..ma-1]`. (Parameters held fixed by calls to `hold` will return zero covariances.)
```
    Int i,j,k,l,m,mfit=0;
    Doub ym,wt,sum,sig2i;
    VecDoub afunc(ma);
    for (j=0;j<ma;j++) if (ia[j]) mfit++;
    if (mfit == 0) throw("lfit: no parameters to be fitted");
    MatDoub temp(mfit,mfit,0.),beta(mfit,1,0.);
    for (i=0;i<ndat;i++) {                    Loop over data to accumulate coefficients of
        afunc = funcs(x[i]);                      the normal equations.
        ym=y[i];
        if (mfit < ma) {                      Subtract off dependences on known pieces
            for (j=0;j<ma;j++)                    of the fitting function.
                if (!ia[j]) ym -= a[j]*afunc[j];
        }
        sig2i=1.0/SQR(sig[i]);
        for (j=0,l=0;l<ma;l++) {              Set up matrix and r.h.s. for matrix inversion.
            if (ia[l]) {
                wt=afunc[l]*sig2i;
                for (k=0,m=0;m<=l;m++)
                    if (ia[m]) temp[j][k++] += wt*afunc[m];
                beta[j++][0] += ym*wt;
            }
        }
    }
    for (j=1;j<mfit;j++) for (k=0;k<j;k++) temp[k][j]=temp[j][k];
    gaussj(temp,beta);                        Matrix solution.
    for (j=0,l=0;l<ma;l++) if (ia[l]) a[l]=beta[j++][0];
```
Spread the solution to appropriate positions in `a`, and evaluate χ^2 of the fit.
```
    chisq=0.0;
    for (i=0;i<ndat;i++) {
        afunc = funcs(x[i]);
        sum=0.0;
        for (j=0;j<ma;j++) sum += a[j]*afunc[j];
        chisq += SQR((y[i]-sum)/sig[i]);
    }
    for (j=0;j<mfit;j++) for (k=0;k<mfit;k++) covar[j][k]=temp[j][k];
    for (i=mfit;i<ma;i++)                     Rearrange covariance matrix into the correct
        for (j=0;j<i+1;j++) covar[i][j]=covar[j][i]=0.0;        order.
    k=mfit-1;
    for (j=ma-1;j>=0;j--) {
```

```
            if (ia[j]) {
                for (i=0;i<ma;i++) SWAP(covar[i][k],covar[i][j]);
                for (i=0;i<ma;i++) SWAP(covar[k][i],covar[j][i]);
                k--;
            }
        }
    }
};
```

Typical use of `Fitlin` will look something like this:

```
const Int npts=...
VecDoub xx(npts),yy(npts),ssig(npts);
...
Fitlin myfit(xx,yy,ssig,cubicfit);
myfit.fit();
```

where (in this example) `cubicfit` is a user-supplied function that might look like this:

```
VecDoub cubicfit(const Doub x) {
    VecDoub ans(4);
    ans[0] = 1.;
    for (Int i=1;i<4;i++) ans[i] = x*ans[i-1];
    return ans;
}
```

15.4.2 Solution by Use of Singular Value Decomposition

In some applications, the normal equations are perfectly adequate for linear least-squares problems. However, in many other cases the normal equations are very close to singular. A zero pivot element may be encountered during the solution of the linear equations (e.g., in `gaussj`), in which case you get no solution at all. Or a very small pivot may occur, in which case you typically get fitted parameters a_k with very large magnitudes that are delicately (and unstably) balanced to cancel out almost precisely when the fitted function is evaluated.

Why does this commonly occur? A mathematical reason is that the condition number of the matrix $\boldsymbol{\alpha}$ is the square of the condition number of $\mathbf{A}$. But an additional physical reason is that, more often than experimenters would like to admit, data do not clearly distinguish between two or more of the basis functions provided. If two such functions, or two different combinations of functions, happen to fit the data about equally well — or equally badly — then the matrix $\boldsymbol{\alpha}$, unable to distinguish between them, neatly folds up its tent and becomes singular. There is a certain mathematical irony in the fact that least-squares problems are *both* overdetermined (number of data points greater than number of parameters) *and* underdetermined (ambiguous combinations of parameters exist); but that is how it frequently is. The ambiguities can be extremely hard to notice a priori in complicated problems.

Enter singular value decomposition (SVD). This would be a good time for you to review the material in §2.6, which we will not repeat here. In the case of an overdetermined system, SVD produces a solution that is the best approximation in the least-squares sense, cf. equation (2.6.10). That is exactly what we want. In the case of an underdetermined system, SVD produces a solution whose values (for us, the a_k's) are smallest in the least-squares sense, cf. equation (2.6.8). That is also what we want: When some combination of basis functions is irrelevant to the fit, that

combination will be driven down to a small, innocuous, value, rather than pushed up to delicately canceling infinities.

In terms of the design matrix **A** (equation 15.4.4) and the vector **b** (equation 15.4.5), minimization of χ^2 in (15.4.3) can be written as

$$\text{find} \quad \mathbf{a} \quad \text{that minimizes} \quad \chi^2 = |\mathbf{A}\cdot\mathbf{a} - \mathbf{b}|^2 \tag{15.4.16}$$

Comparing to equation (2.6.9), we see that this is precisely the problem that routines in the SVD object are designed to solve. The solution, which is given by equation (2.6.12), can be rewritten as follows: If **U** and **V** enter the SVD decomposition of **A** according to equation (2.6.1), as computed by SVD, then let the vectors $\mathbf{U}_{(i)}$ $i = 0, \ldots, M-1$ denote the *columns* of **U** (each one a vector of length N), and let the vectors $\mathbf{V}_{(i)}$ $i = 0, \ldots, M-1$ denote the *columns* of **V** (each one a vector of length M). Then the solution (2.6.12) of the least-squares problem (15.4.16) can be written as

$$\mathbf{a} = \sum_{i=0}^{M-1} \left(\frac{\mathbf{U}_{(i)}\cdot\mathbf{b}}{w_i} \right) \mathbf{V}_{(i)} \tag{15.4.17}$$

where the w_i are, as in §2.6, the singular values calculated by SVD.

Equation (15.4.17) says that the fitted parameters **a** are linear combinations of the columns of **V**, with coefficients obtained by forming dot products of the columns of **U** with the weighted data vector (15.4.5). Though it is beyond our scope to prove here, it turns out that the standard (loosely, "probable") errors in the fitted parameters are also linear combinations of the columns of **V**. In fact, equation (15.4.17) can be written in a form displaying these errors as

$$\mathbf{a} = \left[\sum_{i=0}^{M-1} \left(\frac{\mathbf{U}_{(i)}\cdot\mathbf{b}}{w_i} \right) \mathbf{V}_{(i)} \right] \pm \frac{1}{w_0}\mathbf{V}_{(0)} \pm \cdots \pm \frac{1}{w_{M-1}}\mathbf{V}_{(M-1)} \tag{15.4.18}$$

Here each $\pm$ is followed by a standard deviation. The amazing fact is that, decomposed in this fashion, the standard deviations are all mutually independent (uncorrelated). Therefore they can be added together in root-mean-square fashion. What is going on is that the vectors $\mathbf{V}_{(i)}$ are the principal axes of the error ellipsoid of the fitted parameters **a** (see §15.6).

It follows that the variance in the estimate of a parameter a_j is given by

$$\sigma^2(a_j) = \sum_{i=0}^{M-1} \frac{1}{w_i^2} [\mathbf{V}_{(i)}]_j^2 = \sum_{i=0}^{M-1} \left(\frac{V_{ji}}{w_i} \right)^2 \tag{15.4.19}$$

whose result should be identical to (15.4.14). As before, you should not be surprised at the formula for the covariances, here given without proof,

$$\mathrm{Cov}(a_j, a_k) = \sum_{i=0}^{M-1} \left(\frac{V_{ji} V_{ki}}{w_i^2} \right) \tag{15.4.20}$$

We introduced this subsection by noting that the normal equations can fail by encountering a zero pivot. We have not yet, however, mentioned how SVD overcomes this problem. The answer is: If any singular value w_i is zero, its reciprocal

in equation (15.4.18) should be set to zero, not infinity. (Compare the discussion preceding equation 2.6.7.) This corresponds to adding to the fitted parameters **a** a *zero* multiple, rather than some random large multiple, of any linear combination of basis functions that are degenerate in the fit. It is a good thing to do!

Moreover, if a singular value w_i is nonzero but very small, you should also define *its* reciprocal to be zero, since its apparent value is probably an artifact of roundoff error, not a meaningful number. A plausible answer to the question "how small is small?" is to edit in this fashion all singular values whose ratio to the largest singular value is less than N times the machine precision ϵ. (This is a more conservative recommendation than the default in §2.6, which scales as $N^{1/2}$.)

There is another reason for editing even *additional* singular values, ones large enough that roundoff error is not a question. Singular value decomposition allows you to identify linear combinations of variables that just happen not to contribute much to reducing the χ^2 of your data set. Editing these can sometimes reduce the probable error errors on your coefficients quite significantly, while increasing the minimum χ^2 only negligibly. We will learn more about identifying and treating such cases in §15.6.

Generally speaking, we recommend that you always use SVD techniques instead of using the normal equations. SVD's only significant disadvantage is that it requires extra storage of order $N \times M$ for the design matrix and its decomposition. Storage is also required for the $M \times M$ matrix $\mathbf{V}$, but this is instead of the same-sized matrix for the coefficients of the normal equations. SVD can be significantly slower than solving the normal equations; however, its great advantage, that it (theoretically) *cannot fail*, more than makes up for the speed disadvantage.

The following object, `Fitsvd`, has an interface almost identical to `Fitlin`, above. An additional optional parameter in the constructor sets the threshold for editing singular values.

fitsvd.h

```
struct Fitsvd {
```

Object for general linear least-squares fitting using singular value decomposition. Call constructor to bind data vectors and fitting functions. Then call `fit`, which sets the output quantities `a`, `covar`, and `chisq`.

```
    Int ndat, ma;
    Doub tol;
    VecDoub_I *x,&y,&sig;
    VecDoub (*funcs)(const Doub);
    VecDoub a;
    MatDoub covar;
    Doub chisq;
```

(Why is `x` a pointer? Explained in §15.4.4.)

Output values. `a` is the vector of fitted coefficients, `covar` is its covariance matrix, and `chisq` is the value of χ^2 for the fit.

```
    Fitsvd(VecDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig,
    VecDoub funks(const Doub), const Doub TOL=1.e-12)
    : ndat(yy.size()), x(&xx), xmd(NULL), y(yy), sig(ssig),
    funcs(funks), tol(TOL) {}
```

Constructor. Binds references to the data arrays `xx`, `yy`, and `ssig`, and to a user-supplied function `funks(x)` that returns a `VecDoub` containing `ma` basis functions evaluated at $x =$ `x`. If `TOL` is positive, it is the threshold (relative to the largest singular value) for discarding small singular values. If it is ≤ 0, the default value in `SVD` is used.

```
    void fit() {
```

Solve by singular value decomposition the χ^2 minimization that fits for the coefficients `a[0..ma-1]` of a function that depends linearly on `a`, $y = \sum_i$ `a`$_i \times$ `funks`$_i(x)$. Set answer values for `a[0..ma-1]`, `chisq` $= \chi^2$, and the covariance matrix `covar[0..ma-1][0..ma-1]`.

```
        Int i,j,k;
        Doub tmp,thresh,sum;
        if (x) ma = funcs((*x)[0]).size();
        else ma = funcsmd(row(*xmd,0)).size();          (Discussed in §15.4.4.)
        a.resize(ma);
        covar.resize(ma,ma);
        MatDoub aa(ndat,ma);
        VecDoub b(ndat),afunc(ma);
        for (i=0;i<ndat;i++) {                          Accumulate coefficients of the
            if (x) afunc=funcs((*x)[i]);                    design matrix.
            else afunc=funcsmd(row(*xmd,i));            (Discussed in §15.4.4.)
            tmp=1.0/sig[i];
            for (j=0;j<ma;j++) aa[i][j]=afunc[j]*tmp;
            b[i]=y[i]*tmp;
        }
        SVD svd(aa);                                    Singular value decomposition.
        thresh = (tol > 0. ? tol*svd.w[0] : -1.);
        svd.solve(b,a,thresh);                          Solve for the coefficients.
        chisq=0.0;                                      Evaluate chi-square.
        for (i=0;i<ndat;i++) {
            sum=0.;
            for (j=0;j<ma;j++) sum += aa[i][j]*a[j];
            chisq += SQR(sum-b[i]);
        }
        for (i=0;i<ma;i++) {                            Sum contributions to covariance
            for (j=0;j<i+1;j++) {                           matrix (15.4.20).
                sum=0.0;
                for (k=0;k<ma;k++) if (svd.w[k] > svd.tsh)
                    sum += svd.v[i][k]*svd.v[j][k]/SQR(svd.w[k]);
                covar[j][i]=covar[i][j]=sum;
            }
        }

    }

    From here on, code for multidimensional fits, to be discussed in §15.4.4.
    MatDoub_I *xmd;
    VecDoub (*funcsmd)(VecDoub_I &);

    Fitsvd(MatDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig,
    VecDoub funks(VecDoub_I &), const Doub TOL=1.e-12)
    : ndat(yy.size()), x(NULL), xmd(&xx), y(yy), sig(ssig),
    funcsmd(funks), tol(TOL) {}
    Constructor for multidimensional fits. Exactly the same as the previous constructor, except
    that xx is now a matrix whose rows are the multidimensional data points and funks is now
    a function of a multidimensional data point (as a VecDoub).

    VecDoub row(MatDoub_I &a, const Int i) {
    Utility. Returns the row of a MatDoub as a VecDoub.
        Int j,n=a.ncols();
        VecDoub ans(n);
        for (j=0;j<n;j++) ans[j] = a[i][j];
        return ans;
    }
};
```

For degenerate or nearly degenerate problems, if you want to try different singular value thresholds, you call the `Fitsvd` constructor once. Then, as many times as you want, "reach in" and increase `tol`, then call `fit` again and examine the resulting value of `chisq` (and optionally also the covariance matrix). Keep going as long as `chisq` does not increase by too much. To learn what is "too much," see §15.6; but a few $\times 0.1$ is almost always OK.

15.4.3 Examples

Be aware that some apparently nonlinear problems can be expressed so that they are linear. For example, an exponential model with two parameters a and b,

$$y(x) = a\exp(-bx) \tag{15.4.21}$$

can be rewritten as

$$\log[y(x)] = c - bx \tag{15.4.22}$$

which is linear in its parameters c and b. (Of course you must be aware that such transformations do not exactly take Gaussian errors into Gaussian errors.)

Also watch out for "nonparameters," as in

$$y(x) = a\exp(-bx + d) \tag{15.4.23}$$

Here the parameters a and d are, in fact, indistinguishable. This is a good example of where the normal equations will be exactly singular, and where SVD will find a zero singular value. SVD will then make a least-squares choice for setting a balance between a and d (or, rather, their equivalents in the linear model derived by taking the logarithms). However — and this is true whenever SVD gives back a zero singular value — you are better advised to figure out analytically where the degeneracy is among your basis functions, and then make appropriate deletions in the basis set.

We already gave an example of a user-supplied fitting-function routine, `cubicfit`, above. Here are two further examples. First, we trivially generalize `cubicfit` for polynomials of an arbitrary, preset, degree:

```
Int fpoly_np = 10;                    Global variable for the degree plus one.        fit_examples.h

VecDoub fpoly(const Doub x) {
Fitting routine for a polynomial of degree fpoly_np-1.
    Int j;
    VecDoub p(fpoly_np);
    p[0]=1.0;
    for (j=1;j<fpoly_np;j++) p[j]=p[j-1]*x;
    return p;
}
```

The second example is slightly less trivial. It is used to fit Legendre polynomials up to some order `fleg_nl` to a data set. (Note that, for most uses, the data should satisfy $-1 \le x \le 1$.)

```
Int fleg_nl = 10;                     Global variable for the degree plus one.        fit_examples.h

VecDoub fleg(const Doub x) {
Fitting routine for an expansion with nl Legendre polynomials, evaluated using the recurrence
relation as in §5.4.
    Int j;
    Doub twox,f2,f1,d;
    VecDoub pl(fleg_nl);
    pl[0]=1.;
    pl[1]=x;
    if (fleg_nl > 2) {
        twox=2.*x;
        f2=x;
        d=1.;
        for (j=2;j<fleg_nl;j++) {
```

```
            f1=d++;
            f2+=twox;
            pl[j]=(f2*pl[j-1]-f1*pl[j-2])/d;
        }
    }
    return pl;
}
```

15.4.4 Multidimensional Fits

If you are measuring a single variable y as a function of more than one variable — say, a *vector* of variables $\mathbf{x}$ — then your basis functions will be functions of a vector, $X_0(\mathbf{x}), \ldots, X_{M-1}(\mathbf{x})$. The χ^2 merit function is now

$$\chi^2 = \sum_{i=0}^{N-1} \left[\frac{y_i - \sum_{k=0}^{M-1} a_k X_k(\mathbf{x}_i)}{\sigma_i} \right]^2 \tag{15.4.24}$$

All of the preceding discussion goes through unchanged, with x replaced by $\mathbf{x}$. In fact, we anticipated this in the coding of `Fitsvd`, above, which can do multidimensional general linear fits as easily as one-dimensional. Here is how:

A second, overloaded, constructor in `Fitsvd` substitutes a matrix `xx` for what was previously a vector. The rows of the matrix are the `ndat` data points. The number of columns is the dimensionality of the space (that is, of $\mathbf{x}$). Similarly, the user-supplied function `funks` now takes a vector argument, an $\mathbf{x}$. A simple example (fitting a quadratic function to data in two dimensions) might be

```
VecDoub quadratic2d(VecDoub_I &xx) {
    VecDoub ans(6);
    Doub x=xx[0], y=xx[1];
    ans[0] = 1;
    ans[1] = x; ans[2] = y;
    ans[3] = x*x; ans[4] = x*y; ans[5] = y*y;
    return ans;
}
```

Be sure that the argument of your user function has exactly the type "`VecDoub_I &`" (and not, for example, "`VecDoub &`" or "`VecDoub_I`"), since strict C++ compilers are picky about this.

The two constructors in `Fitsvd` communicate to `fit` whether data points are one-dimensional or multidimensional by setting either `xmd` or `x` to `NULL`. This explains the oddity that `x` was bound to the user data as a pointer, while `y` and `sig` were bound as references. (Yes, we know this is a bit of a hack!)

CITED REFERENCES AND FURTHER READING:

Bevington, P.R., and Robinson, D.K. 2002, *Data Reduction and Error Analysis for the Physical Sciences*, 3rd ed. (New York: McGraw-Hill), Chapter 7.

Lupton, R. 1993, *Statistics in Theory and Practice* (Princeton, NJ: Princeton University Press), Chapter 11.

Lawson, C.L., and Hanson, R. 1974, *Solving Least Squares Problems* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 1995 (Philadelphia: S.I.A.M.).

Monahan, J.F. 2001, *Numerical Methods of Statistics* (Cambridge, UK: Cambridge University Press), Chapter 5.

Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 9.

Gelman, A., Carlin, J.B., Stern, H.S., and Rubin, D.B. 2004, *Bayesian Data Analysis*, 2nd ed. (Boca Raton, FL: Chapman & Hall/CRC), Chapter 14.

15.5 Nonlinear Models

We now consider fitting when the model depends *nonlinearly* on the set of M unknown parameters $a_k, k = 0, 1, \ldots, M-1$. We use the same approach as in previous sections, namely to define a χ^2 merit function and determine best-fit parameters by its minimization. With nonlinear dependences, however, the minimization must proceed iteratively. Given trial values for the parameters, we develop a procedure that improves the trial solution. The procedure is then repeated until χ^2 stops (or effectively stops) decreasing.

How is this problem different from the general nonlinear function minimization problem already dealt with in Chapter 10? Superficially, not at all. Sufficiently close to the minimum, we expect the χ^2 function to be well approximated by a quadratic form, which we can write as

$$\chi^2(\mathbf{a}) \approx \gamma - \mathbf{d} \cdot \mathbf{a} + \tfrac{1}{2}\mathbf{a} \cdot \mathbf{D} \cdot \mathbf{a} \tag{15.5.1}$$

where $\mathbf{d}$ is an M-vector and $\mathbf{D}$ is an $M \times M$ matrix. (Compare equation 10.8.1.) If the approximation is a good one, we know how to jump from the current trial parameters $\mathbf{a}_{\rm cur}$ to the minimizing ones $\mathbf{a}_{\rm min}$ in a single leap, namely

$$\mathbf{a}_{\rm min} = \mathbf{a}_{\rm cur} + \mathbf{D}^{-1} \cdot \left[-\nabla \chi^2(\mathbf{a}_{\rm cur})\right] \tag{15.5.2}$$

(Compare equation 10.9.4.)

On the other hand, (15.5.1) might be a poor local approximation to the shape of the function that we are trying to minimize at $\mathbf{a}_{\rm cur}$. In that case, about all we can do is take a step down the gradient, as in the steepest descent method (§10.8). In other words,

$$\mathbf{a}_{\rm next} = \mathbf{a}_{\rm cur} - \text{constant} \times \nabla \chi^2(\mathbf{a}_{\rm cur}) \tag{15.5.3}$$

where the constant is small enough not to exhaust the downhill direction.

To use (15.5.2) or (15.5.3), we must be able to compute the gradient of the χ^2 function at any set of parameters $\mathbf{a}$. To use (15.5.2) we also need the matrix $\mathbf{D}$, which is the second derivative matrix (Hessian matrix) of the χ^2 merit function, at any $\mathbf{a}$.

Now, this is the crucial difference from Chapter 10: There, we had no way of directly evaluating the Hessian matrix. We were given only the ability to evaluate the function to be minimized and (in some cases) its gradient. Therefore, we had to resort to iterative methods *not just* because our function was nonlinear, *but also* in order to build up information about the Hessian matrix. Sections 10.9 and 10.8 concerned themselves with two different techniques for building up this information.

Here, life is much simpler. We *know* exactly the form of χ^2, since it is based on a model function that we ourselves have specified. Therefore, the Hessian matrix is known to us. Thus we are free to use (15.5.2) whenever we care to do so. The only reason to use (15.5.3) will be failure of (15.5.2) to improve the fit, signaling failure of (15.5.1) as a good local approximation.

15.5.1 Calculation of the Gradient and Hessian

The model to be fitted is

$$y = y(x|\mathbf{a}) \tag{15.5.4}$$

and the χ^2 merit function is

$$\chi^2(\mathbf{a}) = \sum_{i=0}^{N-1} \left[\frac{y_i - y(x_i|\mathbf{a})}{\sigma_i} \right]^2 \tag{15.5.5}$$

The gradient of χ^2 with respect to the parameters $\mathbf{a}$, which will be zero at the χ^2 minimum, has components

$$\frac{\partial \chi^2}{\partial a_k} = -2 \sum_{i=0}^{N-1} \frac{[y_i - y(x_i|\mathbf{a})]}{\sigma_i^2} \frac{\partial y(x_i|\mathbf{a})}{\partial a_k} \qquad k = 0, 1, \ldots, M-1 \tag{15.5.6}$$

Taking an additional partial derivative gives

$$\frac{\partial^2 \chi^2}{\partial a_k \partial a_l} = 2 \sum_{i=0}^{N-1} \frac{1}{\sigma_i^2} \left[\frac{\partial y(x_i|\mathbf{a})}{\partial a_k} \frac{\partial y(x_i|\mathbf{a})}{\partial a_l} - [y_i - y(x_i|\mathbf{a})] \frac{\partial^2 y(x_i|\mathbf{a})}{\partial a_l \partial a_k} \right] \tag{15.5.7}$$

It is conventional to remove the factors of 2 by defining

$$\beta_k \equiv -\frac{1}{2} \frac{\partial \chi^2}{\partial a_k} \qquad \alpha_{kl} \equiv \frac{1}{2} \frac{\partial^2 \chi^2}{\partial a_k \partial a_l} \tag{15.5.8}$$

making $\boldsymbol{\alpha} = \frac{1}{2}\mathbf{D}$ in equation (15.5.2), in terms of which that equation can be rewritten as the set of linear equations:

$$\sum_{l=0}^{M-1} \alpha_{kl}\, \delta a_l = \beta_k \tag{15.5.9}$$

This set is solved for the increments δa_l that, added to the current approximation, give the next approximation. In the context of least squares, the matrix $\boldsymbol{\alpha}$, equal to one-half times the Hessian matrix, is usually called the *curvature matrix*.

Equation (15.5.3), the steepest descent formula, translates to

$$\delta a_l = \text{constant} \times \beta_l \tag{15.5.10}$$

Note that the components α_{kl} of the Hessian matrix (15.5.7) depend both on the first derivatives and on the second derivatives of the basis functions with respect to their parameters. Some treatments proceed to ignore the second derivative without comment. We will ignore it also, but only *after* a few comments.

Second derivatives occur because the gradient (15.5.6) already has a dependence on $\partial y/\partial a_k$, and so the next derivative simply must contain terms involving $\partial^2 y/\partial a_l \partial a_k$. The second derivative term can be dismissed when it is zero (as in the linear case of equation 15.4.8) or small enough to be negligible when compared to the term involving the first derivative. It also has an additional possibility of being ignorably small in practice: The term multiplying the second derivative in equation

(15.5.7) is $[y_i - y(x_i|\mathbf{a})]$. For a successful model, this term should just be the random measurement error of each point. This error can have either sign, and should in general be uncorrelated with the model. Therefore, the second derivative terms tend to cancel out when summed over i.

Inclusion of the second derivative term can in fact be destabilizing if the model fits badly or is contaminated by outlier points that are unlikely to be offset by compensating points of opposite sign. From this point on, we will always use as the definition of α_{kl} the formula

$$\alpha_{kl} = \sum_{i=0}^{N-1} \frac{1}{\sigma_i^2} \left[\frac{\partial y(x_i|\mathbf{a})}{\partial a_k} \frac{\partial y(x_i|\mathbf{a})}{\partial a_l} \right] \tag{15.5.11}$$

This expression more closely resembles its linear cousin (15.4.8). You should understand that minor (or even major) fiddling with $\boldsymbol{\alpha}$ has no effect at all on what final set of parameters $\mathbf{a}$ is reached, but affects only the iterative route that is taken in getting there. The condition at the χ^2 minimum, that $\beta_k = 0$ for all k, is independent of how $\boldsymbol{\alpha}$ is defined.

15.5.2 Levenberg-Marquardt Method

Marquardt [1] put forth an elegant method, related to an earlier suggestion of Levenberg, for varying smoothly between the extremes of the inverse-Hessian method (15.5.9) and the steepest descent method (15.5.10). The latter method is used far from the minimum, switching continuously to the former as the minimum is approached. This *Levenberg-Marquardt method* (also called the *Marquardt method*) works very well in practice if you can guess plausible starting guesses for your parameters. It has become a standard nonlinear least-squares routine.

The method is based on two elementary, but important, insights. Consider the "constant" in equation (15.5.10). What should it be, even in order of magnitude? What sets its scale? There is no information about the answer in the gradient. That tells only the slope, not how far that slope extends. Marquardt's first insight is that the components of the Hessian matrix, even if they are not usable in any precise fashion, give *some* information about the order-of-magnitude scale of the problem.

The quantity χ^2 is nondimensional, i.e., is a pure number; this is evident from its definition (15.5.5). On the other hand, β_k has the dimensions of $1/a_k$, which may well be dimensional, i.e., have units like cm^{-1}, or kilowatt-hours, or whatever. (In fact, each component of β_k can have different dimensions!) The constant of proportionality between β_k and δa_k must therefore have the dimensions of a_k^2. Scan the components of $\boldsymbol{\alpha}$ and you see that there is only one obvious quantity with these dimensions, and that is $1/\alpha_{kk}$, the reciprocal of the diagonal element. So that must set the scale of the constant. But that scale might itself be too big. So let's divide the constant by some (nondimensional) fudge factor λ, with the possibility of setting $\lambda \gg 1$ to cut down the step. In other words, replace equation (15.5.10) by

$$\delta a_l = \frac{1}{\lambda \alpha_{ll}} \beta_l \qquad \text{or} \qquad \lambda\, \alpha_{ll}\, \delta a_l = \beta_l \tag{15.5.12}$$

It is necessary that α_{ll} be positive, but this is guaranteed by definition (15.5.11) — another reason for adopting that equation.

Marquardt's second insight is that equations (15.5.12) and (15.5.9) can be combined if we define a new matrix α' by the following prescription:

$$\begin{aligned} \alpha'_{jj} &\equiv \alpha_{jj}(1+\lambda) \\ \alpha'_{jk} &\equiv \alpha_{jk} \qquad (j \neq k) \end{aligned} \tag{15.5.13}$$

and then replace both (15.5.12) and (15.5.9) by

$$\sum_{l=0}^{M-1} \alpha'_{kl}\,\delta a_l = \beta_k \tag{15.5.14}$$

When λ is very large, the matrix α' is forced into being *diagonally dominant*, so equation (15.5.14) goes over to be identical to (15.5.12). On the other hand, as λ approaches zero, equation (15.5.14) goes over to (15.5.9).

Given an initial guess for the set of fitted parameters $\mathbf{a}$, the recommended Marquardt recipe is as follows:

- Compute $\chi^2(\mathbf{a})$.
- Pick a modest value for λ, say $\lambda = 0.001$.
- (†) Solve the linear equations (15.5.14) for $\delta\mathbf{a}$ and evaluate $\chi^2(\mathbf{a}+\delta\mathbf{a})$.
- If $\chi^2(\mathbf{a}+\delta\mathbf{a}) \geq \chi^2(\mathbf{a})$, *increase* λ by a factor of 10 (or any other substantial factor) and go back to (†).
- If $\chi^2(\mathbf{a}+\delta\mathbf{a}) < \chi^2(\mathbf{a})$, *decrease* λ by a factor of 10, update the trial solution $\mathbf{a} \leftarrow \mathbf{a}+\delta\mathbf{a}$, and go back to (†).

Also necessary is a condition for stopping. Iterating to convergence (to machine accuracy or to the roundoff limit) is generally wasteful and unnecessary since the minimum is at best only a statistical estimate of the parameters $\mathbf{a}$. As we will see in §15.6, a change in the parameters that changes χ^2 by an amount $\ll 1$ is *never* statistically meaningful.

Furthermore, it is not uncommon to find the parameters wandering around near the minimum in a flat valley of complicated topography. The reason is that Marquardt's method generalizes the method of normal equations (§15.4); hence it has the same problem as that method with regard to near-degeneracy of the minimum. Outright failure by a zero pivot is possible, but unlikely. More often, a small pivot will generate a large correction that is then rejected, the value of λ being then increased. For sufficiently large λ, the matrix $\boldsymbol{\alpha}'$ is positive-definite and can have no small pivots. Thus the method does tend to stay away from zero pivots, but at the cost of a tendency to wander around doing steepest descent in very unsteep degenerate valleys.

These considerations suggest that, in practice, one might as well stop iterating after a few occurrences of χ^2 decreasing by a negligible amount, say either less than 0.001 absolutely or (in case roundoff prevents that being reached) fractionally. Don't stop after a step where χ^2 *increases* more than trivially: That only shows that λ has not yet adjusted itself optimally.

Once the acceptable minimum has been found, one wants to set $\lambda = 0$ and compute the matrix

$$\mathbf{C} \equiv \boldsymbol{\alpha}^{-1} \tag{15.5.15}$$

which, as before, is the estimated covariance matrix of the standard errors in the fitted parameters $\mathbf{a}$ (see next section).

The following object, Fitmrq, implements Marquardt's method for nonlinear parameter estimation. The user interface is intentionally very close to that of Fitlin in §15.4. In particular, the feature of being able to freeze or unfreeze chosen parameters is available here, too.

One difference from Fitlin is that you have to supply an initial guess for the parameters **a**. Now *that* is a can of worms! When you are fitting for parameters that enter highly nonlinearly, there is no reason in the world that the χ^2 surface should have only a single minimum. Marquardt's method embodies no magical insight into finding the global minimum; it's just a downhill search. Often, it should be the endgame strategy for fitting parameters, preceded by perhaps cruder, and likely problem-specific, methods for getting into the right general basin of convergence.

Another difference between Fitmrq and Fitlin is the format of the user-supplied function funks. Since Fitmrq needs both function and gradient values, funks is now coded as a void function returning answers through arguments passed by reference. An example is given below. You call Fitmrq's constructor once, to bind your data vectors and function. Then (after any optional calls to hold or free) you call fit, which sets values for a, chisq, and covar. The curvature matrix alpha is also available. Note that the original vector of parameter guesses that you send to the constructor is not modified; rather, the answer is returned in a.

fitmrq.h

```
struct Fitmrq {
```

Object for nonlinear least-squares fitting by the Levenberg-Marquardt method, also including the ability to hold specified parameters at fixed, specified values. Call constructor to bind data vectors and fitting functions and to input an initial parameter guess. Then call any combination of hold, free, and fit as often as desired. fit sets the output quantities a, covar, alpha, and chisq.

```
    static const Int NDONE=4, ITMAX=1000;
```

Convergence parameters.

```
    Int ndat, ma, mfit;
    VecDoub_I &x,&y,&sig;
    Doub tol;
    void (*funcs)(const Doub, VecDoub_I &, Doub &, VecDoub_O &);
    VecBool ia;
    VecDoub a;
    MatDoub covar;
    MatDoub alpha;
    Doub chisq;
```

Output values. a is the vector of fitted coefficients, covar is its covariance matrix, alpha is the curvature matrix, and chisq is the value of χ^2 for the fit.

```
    Fitmrq(VecDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig, VecDoub_I &aa,
    void funks(const Doub, VecDoub_I &, Doub &, VecDoub_O &), const Doub
    TOL=1.e-3) : ndat(xx.size()), ma(aa.size()), x(xx), y(yy), sig(ssig),
    tol(TOL), funcs(funks), ia(ma), alpha(ma,ma), a(aa), covar(ma,ma) {
```

Constructor. Binds references to the data arrays xx, yy, and ssig, and to a user-supplied function funks that calculates the nonlinear fitting function and its derivatives. Also inputs the initial parameters guess aa (which is copied, not modified) and an optional convergence tolerance TOL. Initializes all parameters as free (not held).

```
        for (Int i=0;i<ma;i++) ia[i] = true;
    }

    void hold(const Int i, const Doub val) {ia[i]=false; a[i]=val;}
    void free(const Int i) {ia[i]=true;}
```

Optional functions for holding a parameter, identified by a value i in the range 0,...,ma-1, fixed at the value val, or for freeing a parameter that was previously held fixed. hold and free may be called for any number of parameters before calling fit to calculate best-fit values for the remaining (not held) parameters, and the process may be repeated multiple times.

```
    void fit() {
```

Iterate to reduce the χ^2 of a fit between a set of data points `x[0..ndat-1]`, `y[0..ndat-1]` with individual standard deviations `sig[0..ndat-1]`, and a nonlinear function that depends on `ma` coefficients `a[0..ma-1]`. When χ^2 is no longer decreasing, set best-fit values for the parameters `a[0..ma-1]`, and `chisq` $= \chi^2$, `covar[0..ma-1][0..ma-1]`, and `alpha[0..ma-1][0..ma-1]`. (Parameters held fixed will return zero covariances.)

```
    Int j,k,l,iter,done=0;
    Doub alamda=.001,ochisq;
    VecDoub atry(ma),beta(ma),da(ma);
    mfit=0;
    for (j=0;j<ma;j++) if (ia[j]) mfit++;
    MatDoub oneda(mfit,1), temp(mfit,mfit);
    mrqcof(a,alpha,beta);                    Initialization.
    for (j=0;j<ma;j++) atry[j]=a[j];
    ochisq=chisq;
    for (iter=0;iter<ITMAX;iter++) {
        if (done==NDONE) alamda=0.;          Last pass. Use zero alamda.
        for (j=0;j<mfit;j++) {               Alter linearized fitting matrix, by augmenting di-
            for (k=0;k<mfit;k++) covar[j][k]=alpha[j][k];    agonal elements.
            covar[j][j]=alpha[j][j]*(1.0+alamda);
            for (k=0;k<mfit;k++) temp[j][k]=covar[j][k];
            oneda[j][0]=beta[j];
        }
        gaussj(temp,oneda);                  Matrix solution.
        for (j=0;j<mfit;j++) {
            for (k=0;k<mfit;k++) covar[j][k]=temp[j][k];
            da[j]=oneda[j][0];
        }
        if (done==NDONE) {                   Converged. Clean up and return.
            covsrt(covar);
            covsrt(alpha);
            return;
        }
        for (j=0,l=0;l<ma;l++)               Did the trial succeed?
            if (ia[l]) atry[l]=a[l]+da[j++];
        mrqcof(atry,covar,da);
        if (abs(chisq-ochisq) < MAX(tol,tol*chisq)) done++;
        if (chisq < ochisq) {                Success, accept the new solution.
            alamda *= 0.1;
            ochisq=chisq;
            for (j=0;j<mfit;j++) {
                for (k=0;k<mfit;k++) alpha[j][k]=covar[j][k];
                    beta[j]=da[j];
            }
            for (l=0;l<ma;l++) a[l]=atry[l];
        } else {                             Failure, increase alamda.
            alamda *= 10.0;
            chisq=ochisq;
        }
    }
    throw("Fitmrq too many iterations");
}


void mrqcof(VecDoub_I &a, MatDoub_O &alpha, VecDoub_O &beta) {
```

Used by `fit` to evaluate the linearized fitting matrix `alpha`, and vector `beta` as in (15.5.8), and to calculate χ^2.

```
    Int i,j,k,l,m;
    Doub ymod,wt,sig2i,dy;
    VecDoub dyda(ma);
    for (j=0;j<mfit;j++) {                   Initialize (symmetric) alpha, beta.
        for (k=0;k<=j;k++) alpha[j][k]=0.0;
        beta[j]=0.;
    }
```

```
            chisq=0.;
            for (i=0;i<ndat;i++) {          Summation loop over all data.
                funcs(x[i],a,ymod,dyda);
                sig2i=1.0/(sig[i]*sig[i]);
                dy=y[i]-ymod;
                for (j=0,l=0;l<ma;l++) {
                    if (ia[l]) {
                        wt=dyda[l]*sig2i;
                        for (k=0,m=0;m<l+1;m++)
                            if (ia[m]) alpha[j][k++] += wt*dyda[m];
                        beta[j++] += dy*wt;
                    }
                }
                chisq += dy*dy*sig2i;       And find \chi^2.
            }
            for (j=1;j<mfit;j++)            Fill in the symmetric side.
                for (k=0;k<j;k++) alpha[k][j]=alpha[j][k];
        }

        void covsrt(MatDoub_IO &covar) {
        Expand in storage the covariance matrix covar, so as to take into account parameters that
        are being held fixed. (For the latter, return zero covariances.)
            Int i,j,k;
            for (i=mfit;i<ma;i++)
                for (j=0;j<i+1;j++) covar[i][j]=covar[j][i]=0.0;
            k=mfit-1;
            for (j=ma-1;j>=0;j--) {
                if (ia[j]) {
                    for (i=0;i<ma;i++) SWAP(covar[i][k],covar[i][j]);
                    for (i=0;i<ma;i++) SWAP(covar[k][i],covar[j][i]);
                    k--;
                }
            }
        }

};
```

15.5.3 Example

The following function `fgauss` is an example of a user-supplied function `funks`. Used with `Fitmrq`, it fits for the model

$$y(x) = \sum_{k=0}^{K-1} B_k \exp\left[-\left(\frac{x - E_k}{G_k}\right)^2\right] \tag{15.5.16}$$

which is a sum of K Gaussians, each with a variable position, amplitude, and width. We store the parameters in the order $B_0, E_0, G_0, B_1, E_1, G_1, \ldots, B_{K-1}, E_{K-1}, G_{K-1}$.

fit_examples.h

```
void fgauss(const Doub x, VecDoub_I &a, Doub &y, VecDoub_O &dyda) {
```

$y(x;a)$ is the sum of na/3 Gaussians (15.5.16). The amplitude, center, and width of the Gaussians are stored in consecutive locations of a: a[3k] = B_k, a[3k+1] = E_k, a[3k+2] = G_k, $k = 0, \ldots,$ na/3 $-$ 1. The dimensions of the arrays are a[0..na-1], dyda[0..na-1].

```
    Int i,na=a.size();
    Doub fac,ex,arg;
    y=0.;
    for (i=0;i<na-1;i+=3) {
        arg=(x-a[i+1])/a[i+2];
        ex=exp(-SQR(arg));
```

```
            fac=a[i]*ex*2.*arg;
            y += a[i]*ex;
            dyda[i]=ex;
            dyda[i+1]=fac/a[i+2];
            dyda[i+2]=fac*arg/a[i+2];
        }
}
```

15.5.4 More Advanced Methods for Nonlinear Least Squares

You will need more capability than `Fitmrq` can supply if either (i) it is converging too slowly, or (ii) it is converging to a local minimum that is not the one you want. Several options are available.

NL2SOL [3] is a highly regarded nonlinear least-squares implementation with many advanced features. For example, it keeps the second-derivative term we dropped in the Levenberg-Marquardt method whenever it would be better to do so, a so-called *full Newton-type* method.

A different variant on the Levenberg-Marquardt algorithm is to implement it as a model-trust region method for minimization (see §9.7 and ref. [2]) applied to the special case of a least-squares function. A code of this kind due to Moré [4] can be found in MINPACK [5].

CITED REFERENCES AND FURTHER READING:

Bevington, P.R., and Robinson, D.K. 2002, *Data Reduction and Error Analysis for the Physical Sciences*, 3rd ed. (New York: McGraw-Hill), Chapter 8.

Monahan, J.F. 2001, *Numerical Methods of Statistics* (Cambridge, UK: Cambridge University Press), Chapters 5–9.

Seber, G.A.F., and Wild, C.J. 2003, *Nonlinear Regression* (Hoboken, NJ: Wiley).

Gelman, A., Carlin, J.B., Stern, H.S., and Rubin, D.B. 2004, *Bayesian Data Analysis*, 2nd ed. (Boca Raton, FL: Chapman & Hall/CRC).

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.2 (by J.E. Dennis).

Marquardt, D.W. 1963, *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, pp. 431–441.[1]

Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).[2]

Dennis, J.E., Gay, D.M, and Welsch, R.E. 1981, "An Adaptive Nonlinear Least-Squares Algorithm," *ACM Transactions on Mathematical Software*, vol. 7, pp. 348–368; *op. cit.*, pp. 369–383.[3].

Moré, J.J. 1977, in *Numerical Analysis*, Lecture Notes in Mathematics, vol. 630, G.A. Watson, ed. (Berlin: Springer), pp. 105–116.[4]

Moré, J.J., Garbow, B.S., and Hillstrom, K.E. 1980, *User Guide for MINPACK-1*, Argonne National Laboratory Report ANL-80-74.[5]

15.6 Confidence Limits on Estimated Model Parameters

Several times already in this chapter we have made statements about the standard errors, or uncertainties, in a set of M estimated parameters $\mathbf{a}$. We have given some formulas for computing standard deviations or variances of individual parameters (equations 15.2.9, 15.4.15, and 15.4.19), as well as some formulas for covariances between pairs of parameters (equation 15.2.10; remark following equation 15.4.15; equation 15.4.20; equation 15.5.15).

In this section, we want to be more explicit regarding the precise meaning of these quantitative uncertainties, and to give further information about how quantitative confidence limits on fitted parameters can be estimated. The subject can get somewhat technical, and even somewhat confusing, so we will try to make precise statements, even when they must be offered without proof.

Figure 15.6.1 shows the conceptual scheme of an experiment that "measures" a set of parameters. There is some underlying true set of parameters $\mathbf{a}_{\text{true}}$ that are known to Mother Nature but hidden from the experimenter. These true parameters are statistically realized, along with random measurement errors, as a measured data set, which we will symbolize as $\mathcal{D}_{(0)}$. The data set $\mathcal{D}_{(0)}$ *is* known to the experimenter. He or she fits the data to a model by χ^2 minimization or some other technique and obtains measured, i.e., fitted, values for the parameters, which we here denote $\mathbf{a}_{(0)}$.

Because measurement errors have a random component, $\mathcal{D}_{(0)}$ is not a unique realization of the true parameters $\mathbf{a}_{\text{true}}$. Rather, there are infinitely many other realizations of the true parameters as "hypothetical data sets" each of which *could* have been the one measured, but happened not to be. Let us symbolize these by $\mathcal{D}_{(1)}, \mathcal{D}_{(2)}, \ldots$. Each one, had it been realized, would have given a slightly different set of fitted parameters, $\mathbf{a}_{(1)}, \mathbf{a}_{(2)}, \ldots$, respectively. These parameter sets $\mathbf{a}_{(i)}$ therefore occur with some probability distribution in the M-dimensional space of all possible parameter sets $\mathbf{a}$. The actual measured set $\mathbf{a}_{(0)}$ is one member drawn from this distribution.

Even more interesting than the probability distribution of $\mathbf{a}_{(i)}$ would be the distribution of the difference $\mathbf{a}_{(i)} - \mathbf{a}_{\text{true}}$. This distribution differs from the former one by a translation that puts Mother Nature's true value at the origin. If we knew *this* distribution, we would know everything that there is to know about the quantitative uncertainties in our experimental measurement $\mathbf{a}_{(0)}$.

So the name of the game is to find some way of estimating or approximating the probability distribution of $\mathbf{a}_{(i)} - \mathbf{a}_{\text{true}}$ without knowing $\mathbf{a}_{\text{true}}$ and without having available to us an infinite universe of hypothetical data sets.

15.6.1 Monte Carlo Simulation of Synthetic Data Sets

Although the measured parameter set $\mathbf{a}_{(0)}$ is not the true one, let us consider a fictitious world in which it *was* the true one. Since we hope that our measured parameters are not *too* wrong, we hope that that fictitious world is not too different from the actual world with parameters $\mathbf{a}_{\text{true}}$. In particular, let us hope — no, let us *assume* — that the shape of the probability distribution $\mathbf{a}_{(i)} - \mathbf{a}_{(0)}$ in the fictitious world is the same, or very nearly the same, as the shape of the probability distribution

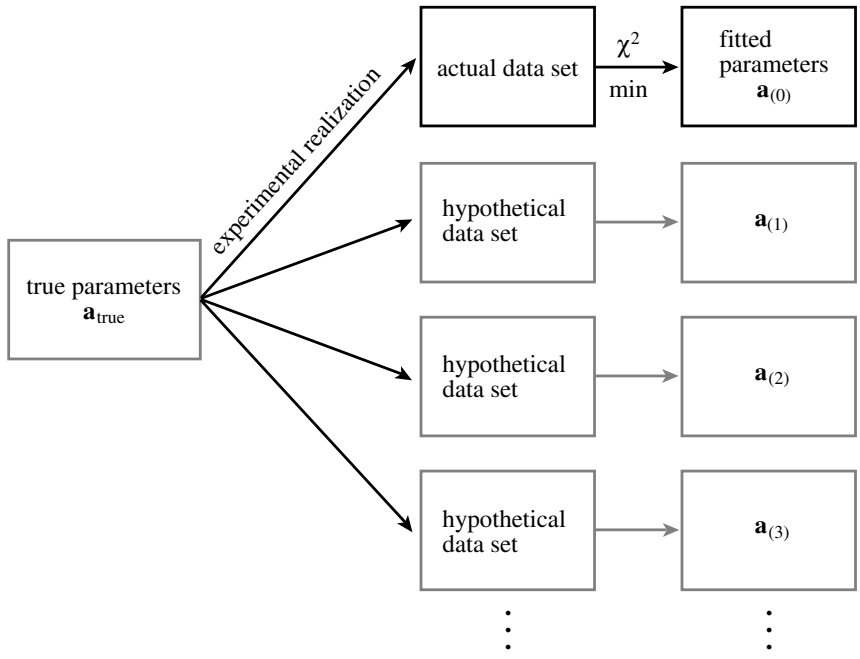


Figure 15.6.1. A statistical universe of data sets from an underlying model. True parameters $\mathbf{a}_{\text{true}}$ are realized in a data set, from which fitted (observed) parameters $\mathbf{a}_{(0)}$ are obtained. If the experiment were repeated many times, new data sets and new values of the fitted parameters would be obtained.

$\mathbf{a}_{(i)} - \mathbf{a}_{\text{true}}$ in the real world. Notice that we are not assuming that $\mathbf{a}_{(0)}$ and $\mathbf{a}_{\text{true}}$ are equal; they are certainly not. We are only assuming that the way in which random errors enter the experiment and data analysis does not vary rapidly as a function of $\mathbf{a}_{\text{true}}$, so that $\mathbf{a}_{(0)}$ can serve as a reasonable surrogate.

Now, often, the distribution of $\mathbf{a}_{(i)} - \mathbf{a}_{(0)}$ in the fictitious world *is* within our power to calculate (see Figure 15.6.2). If we know something about the process that generated our data, given an assumed set of parameters $\mathbf{a}_{(0)}$, then we can usually figure out how to *simulate* our own sets of "synthetic" realizations of these parameters as "synthetic data sets." The procedure is to draw random numbers from appropriate distributions (cf. §7.3 – §7.4) so as to mimic our best understanding of the underlying process and measurement errors in our apparatus. With such random draws, we construct data sets with exactly the same numbers of measured points, and precisely the same values of all control (independent) variables, as our actual data set $\mathcal{D}_{(0)}$. Let us call these simulated data sets $\mathcal{D}^S_{(1)}, \mathcal{D}^S_{(2)}, \ldots$. By construction, these are supposed to have exactly the same statistical relationship to $\mathbf{a}_{(0)}$ as the $\mathcal{D}_{(i)}$'s have to $\mathbf{a}_{\text{true}}$. (For the case where you don't know enough about what you are measuring to do a credible job of simulating it, see below.)

Next, for each $\mathcal{D}^S_{(j)}$, perform exactly the same procedure for estimation of parameters, e.g., χ^2 minimization, as was performed on the actual data to get the parameters $\mathbf{a}_{(0)}$, giving simulated measured parameters $\mathbf{a}^S_{(1)}, \mathbf{a}^S_{(2)}, \ldots$. Each simulated measured parameter set yields a point $\mathbf{a}^S_{(i)} - \mathbf{a}_{(0)}$. Simulate enough data sets and enough derived simulated measured parameters, and you map out the desired probability distribution in M dimensions.

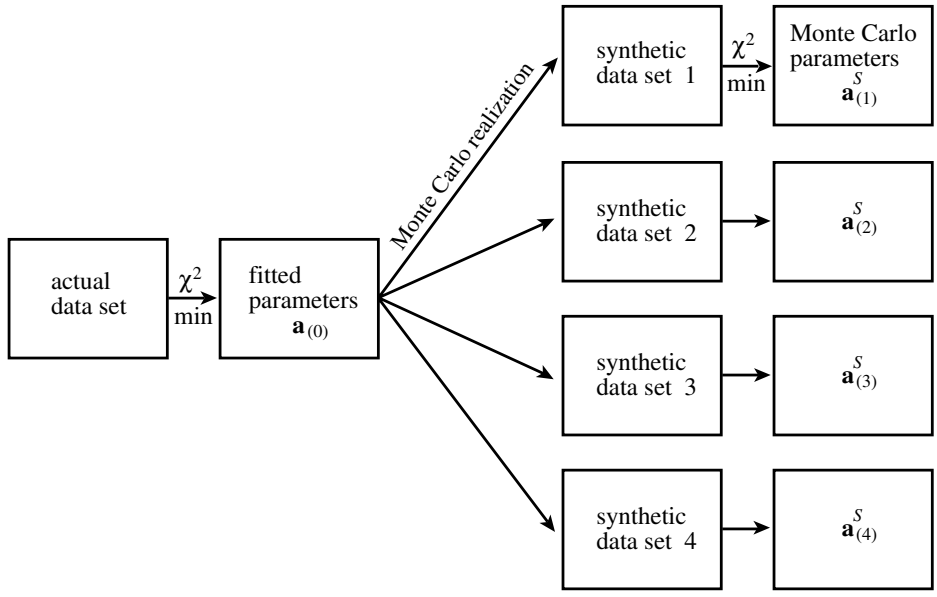


Figure 15.6.2. Monte Carlo simulation of an experiment. The fitted parameters from an actual experiment are used as surrogates for the true parameters. Computer-generated random numbers are used to simulate many synthetic data sets. Each of these is analyzed to obtain its fitted parameters. The distribution of these fitted parameters around the (known) surrogate true parameters is thus studied.

In fact, the ability to do *Monte Carlo simulations* in this fashion has revolutionized many fields of modern experimental science. Not only is one able to characterize the errors of parameter estimation in a very precise way; one can also try out on the computer different methods of parameter estimation, or different data reduction techniques, and seek to minimize the uncertainty of the result according to any desired criteria. Offered the choice between mastery of a five-foot shelf of analytical statistics books and middling ability at performing statistical Monte Carlo simulations, we would surely choose to have the latter skill.

15.6.2 Quick-and-Dirty Monte Carlo: The Bootstrap Method

Here is a powerful technique that can often be used when you don't know enough about the underlying process, or the nature of your measurement errors, to do a credible Monte Carlo simulation. Suppose that your data set consists of N *independent and identically distributed* (or *iid*) "data points." Each data point probably consists of several numbers, e.g., one or more control variables (uniformly distributed, say, in the range that you have decided to measure) and one or more associated measured values (each distributed however Mother Nature chooses). "Iid" means that the sequential order of the data points is not of consequence to the process that you are using to get the fitted parameters $\mathbf{a}$. For example, a χ^2 sum like (15.5.5) does not care in what order the points are added. Even simpler examples are the mean value of a measured quantity and the mean of some function of the measured quantities.

The *bootstrap method* [1] uses the actual data set $\mathcal{D}^S_{(0)}$, with its N data points, to generate any number of synthetic data sets $\mathcal{D}^S_{(1)}, \mathcal{D}^S_{(2)}, \ldots$, also with N data points. The procedure is simply to draw N data points at a time *with replacement* from the set $\mathcal{D}^S_{(0)}$. Because of the replacement, you do not simply get back your original data set each time. You get sets in which a random fraction of the original points, typically $\sim 1/e \approx 37\%$, are replaced by *duplicated* original points. Now, exactly as in the previous discussion, you subject these data sets to the same estimation procedure as was performed on the actual data, giving a set of simulated measured parameters $\mathbf{a}^S_{(1)}, \mathbf{a}^S_{(2)}, \ldots$. These will be distributed around $\mathbf{a}_{(0)}$ in close to the same way that $\mathbf{a}_{(0)}$ is distributed around $\mathbf{a}_{\text{true}}$.

Sounds like getting something for nothing, doesn't it? In fact, it took a while for the bootstrap method to become accepted by statisticians. By now, however, enough theorems have been proved to render the bootstrap reputable (see [2] for references). The basic idea behind the bootstrap is that the actual data set, viewed as a probability distribution consisting of delta functions at the measured values, is in most cases the best — or only — available estimator of the underlying probability distribution. It takes courage, but one can often simply use *that* distribution as the basis for Monte Carlo simulations.

Watch out for cases where the bootstrap's iid assumption is violated. For example, if you have made measurements at evenly spaced intervals of some control variable, then you can *usually* get away with pretending that these are iid uniformly distributed over the measured range. However, some estimators of $\mathbf{a}$ (e.g., ones involving Fourier methods) might be particularly sensitive to all the points on a grid being present. In that case, the bootstrap is going to give a wrong distribution. Also watch out for estimators that look at anything like small-scale clumpiness within the N data points, or estimators that sort the data and look at sequential differences. Obviously the bootstrap will fail on these, too. (The theorems justifying the method are still true, but some of their technical assumptions are violated by these examples.)

For a large class of problems, however, the bootstrap does yield easy, *very quick*, Monte Carlo estimates of the errors in an estimated parameter set.

15.6.3 Confidence Limits

Rather than present all details of the probability distribution of errors in parameter estimation, it is common practice to summarize the distribution in the form of *confidence limits*. The full probability distribution is a function defined on the M-dimensional space of parameters $\mathbf{a}$. A *confidence region* (or *confidence interval*) is just a region of that M-dimensional space (hopefully a small region) that contains a certain (hopefully large) percentage of the total probability distribution. You point to a confidence region and say, e.g., "there is a 99% chance that the true parameter values fall within this region around the measured value."

It is worth emphasizing that you, the experimenter, get to pick both the *confidence level* (99% in the above example) and the shape of the confidence region. The only requirement is that your region does include the stated percentage of probability. Certain percentages are, however, customary in scientific usage: 68.3% (the lowest confidence worthy of quoting), 90%, 95.4%, 99%, and 99.73%. Higher confidence levels are conventionally "ninety-nine point nine ... nine." As for shape, obviously you want a region that is compact and reasonably centered on your mea-

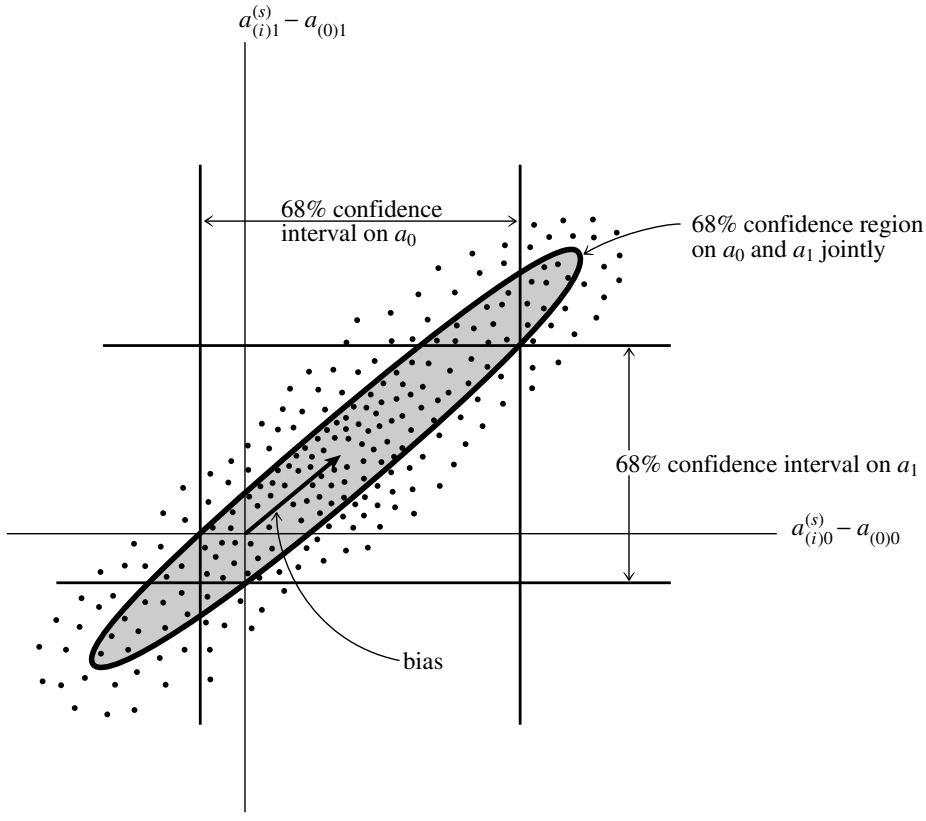


Figure 15.6.3. Confidence intervals in one and two dimensions. The same fraction of measured points (here 68%) lies (i) between the two vertical lines, (ii) between the two horizontal lines, (iii) within the ellipse.

surement $\mathbf{a}_{(0)}$, since the whole purpose of a confidence limit is to inspire confidence in that measured value. In one dimension, the convention is to use a line segment centered on the measured value; in higher dimensions, ellipses or ellipsoids are most frequently used.

You might suspect, correctly, that the numbers 68.3%, 95.4%, and 99.73%, and the use of ellipsoids, have some connection with a normal distribution. That is true historically, but not always relevant nowadays. In general, the probability distribution of the parameters will not be normal, and the above numbers, used as levels of confidence, are purely matters of convention.

Figure 15.6.3 sketches a possible probability distribution for the case $M = 2$. Shown are three different confidence regions that might usefully be given, all at the same confidence level. The two vertical lines enclose a band (horizontal interval) that represents the 68% confidence interval for the variable a_0 without regard to the value of a_1. Similarly the horizontal lines enclose a 68% confidence interval for a_1. The ellipse shows a 68% confidence interval for a_0 and a_1 jointly. Notice that to enclose the same probability as the two bands, the ellipse must necessarily extend outside of both of them (a point we will return to below).

15.6.4 Constant Chi-Square Boundaries as Confidence Limits

When the method used to estimate the parameters $\mathbf{a}_{(0)}$ is chi-square minimization, as in the previous sections of this chapter, then there is a natural choice for the shape of confidence intervals, whose use is almost universal. For the observed data set $\mathcal{D}_{(0)}$, the value of χ^2 is a minimum at $\mathbf{a}_{(0)}$. Call this minimum value $\chi^2_{\min}$. If the vector $\mathbf{a}$ of parameter values is perturbed away from $\mathbf{a}_{(0)}$, then χ^2 increases. The region within which χ^2 increases by no more than a set amount $\Delta\chi^2$ defines some M-dimensional confidence region around $\mathbf{a}_{(0)}$. If $\Delta\chi^2$ is set to be a large number, this will be a big region; if it is small, it will be small. Somewhere in between there will be choices of $\Delta\chi^2$ that cause the region to contain, variously, 68%, 90%, etc., of probability distribution for $\mathbf{a}$'s, as defined above. These regions are taken as the confidence regions for the parameters $\mathbf{a}_{(0)}$.

Very frequently one is interested not in the full M-dimensional confidence region, but in individual confidence regions for some smaller number ν of parameters. For example, one might be interested in the confidence interval of each parameter taken separately (the bands in Figure 15.6.3), in which case $\nu = 1$. In that case, the natural confidence regions in the ν-dimensional subspace of the M-dimensional parameter space are the *projections* of the M-dimensional regions defined by fixed $\Delta\chi^2$ into the ν-dimensional spaces of interest. In Figure 15.6.4, for the case $M = 2$, we show regions corresponding to several values of $\Delta\chi^2$. The one-dimensional confidence interval in a_1 corresponding to the region bounded by $\Delta\chi^2 = 1$ lies between the lines A and A'.

Note that it is the projection of the higher-dimensional region on the lower-dimension space that is used, not the intersection. The intersection would be the band between Z and Z'. It is *never* used. It is shown in the figure only for the purpose of making this cautionary point, that it should not be confused with the projection.

15.6.5 Probability Distribution of Parameters in the Normal Case

You may be wondering why we have, in this section up to now, made no connection at all with the error estimates that come out of the χ^2 fitting procedure, most notably the covariance matrix C_{ij}. The reason is this: χ^2 minimization is a useful means for estimating parameters even if the measurement errors are not normally distributed. While normally distributed errors are required if the χ^2 parameter estimate is to be a maximum likelihood estimator (§15.1), one is often willing to give up that property in return for the relative convenience of the χ^2 procedure. Only in extreme cases, i.e., measurement error distributions with very large "tails," is χ^2 minimization abandoned in favor of more robust techniques, as will be discussed in §15.7.

However, the formal covariance matrix that comes out of a χ^2 minimization has a clear quantitative interpretation only if (or to the extent that) the measurement errors actually are normally distributed. In the case of *non*normal errors, you are "allowed"

- to fit for parameters by minimizing χ^2
- to use a contour of constant $\Delta\chi^2$ as the boundary of your confidence region
- to use Monte Carlo simulation or detailed analytic calculation in determining

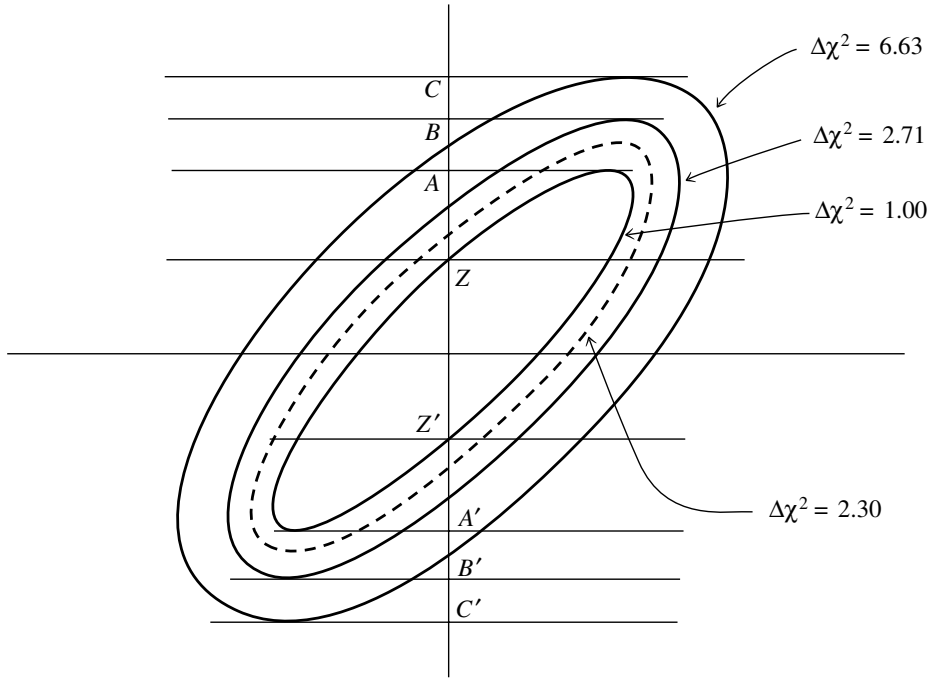


Figure 15.6.4. Confidence region ellipses corresponding to values of chi-square larger than the fitted minimum. The solid curves, with $\Delta\chi^2 = 1.00, 2.71, 6.63$, project onto one-dimensional intervals AA', BB', CC'. These intervals — not the ellipses themselves — contain 68.3%, 90%, and 99% of normally distributed data. The ellipse that contains 68.3% of normally distributed data is shown dashed and has $\Delta\chi^2 = 2.30$. For additional numerical values, see the table on p. 815.

which contour $\Delta\chi^2$ is the correct one for your desired confidence level

- to give the covariance matrix C_{ij} as the "formal covariance matrix of the fit."

You are *not* allowed

- to use formulas that we now give for the case of normal errors, which establish quantitative relationships among $\Delta\chi^2$, C_{ij}, and the confidence level.

Here are the key theorems that hold when (i) the measurement errors are normally distributed, and either (ii) the model is linear in its parameters or (iii) the sample size is large enough that the uncertainties in the fitted parameters $\mathbf{a}$ do not extend outside a region in which the model could be replaced by a suitable linearized model. [Note that condition (iii) does not preclude your use of a nonlinear routine like `Fitmrq` to *find* the fitted parameters.]

Theorem A. $\chi^2_{\min}$ is distributed as a chi-square distribution with $N - M$ degrees of freedom, where N is the number of data points and M is the number of fitted parameters. This is the basic theorem that lets you evaluate the goodness-of-fit of the model, as discussed above in §15.1. We list it first to remind you that unless the goodness-of-fit is credible, the whole estimation of parameters is suspect.

Theorem B. If $\mathbf{a}^S_{(j)}$ is drawn from the universe of simulated data sets with actual parameters $\mathbf{a}_{(0)}$, then the probability distribution of $\delta\mathbf{a} \equiv \mathbf{a}^S_{(j)} - \mathbf{a}_{(0)}$ is the multivariate normal distribution

$$P(\delta\mathbf{a})\, da_0 \ldots da_{M-1} = \text{const.} \times \exp\left(-\tfrac{1}{2}\delta\mathbf{a}\cdot\boldsymbol{\alpha}\cdot\delta\mathbf{a}\right)\, da_0 \ldots da_{M-1}$$

where $\boldsymbol{\alpha}$ is the curvature matrix defined in equation (15.5.8).

Theorem C. If $\mathbf{a}^S_{(j)}$ is drawn from the universe of simulated data sets with actual parameters $\mathbf{a}_{(0)}$, then the quantity $\Delta\chi^2 \equiv \chi^2(\mathbf{a}_{(j)}) - \chi^2(\mathbf{a}_{(0)})$ is distributed as a chi-square distribution with M degrees of freedom. Here the χ^2's are all evaluated using the fixed (actual) data set $\mathcal{D}_{(0)}$. This theorem makes the connection between particular values of $\Delta\chi^2$ and the fraction of the probability distribution that they enclose as an M-dimensional region, i.e., the confidence level of the M-dimensional confidence region.

Theorem D. Suppose that $\mathbf{a}^S_{(j)}$ is drawn from the universe of simulated data sets (as above); that its first ν components $a_0, \ldots, a_{\nu-1}$ are held fixed; and that its remaining $M - \nu$ components are varied so as to minimize χ^2. Call this minimum value χ^2_ν. Then $\Delta\chi^2_\nu \equiv \chi^2_\nu - \chi^2_{\min}$ is distributed as a chi-square distribution with ν degrees of freedom. If you consult Figure 15.6.4, you will see that this theorem connects the *projected* $\Delta\chi^2$ region with a confidence level. In the figure, a point that is held fixed in a_1 and allowed to vary in a_0 minimizing χ^2 will seek out the ellipse whose top or bottom edge is tangent to the line of constant a_1, and is therefore the line that projects it onto the smaller-dimensional space.

As a first example, let us consider the case $\nu = 1$, where we want to find the confidence interval of a single parameter, say a_0. Notice that the chi-square distribution with $\nu = 1$ degree of freedom is the same distribution as that of the square of a single normally distributed quantity. Thus $\Delta\chi^2_\nu < 1$ occurs 68.3% of the time (1-σ for the normal distribution), $\Delta\chi^2_\nu < 4$ occurs 95.4% of the time (2-σ for the normal distribution), $\Delta\chi^2_\nu < 9$ occurs 99.73% of the time (3-σ for the normal distribution), etc. In this manner you find the $\Delta\chi^2_\nu$ that corresponds to your desired confidence level. (Additional values are given in the table on the next page.)

Let $\delta\mathbf{a}$ be a change in the parameters whose first component is arbitrary, δa_0, but the rest of whose components are chosen to minimize the $\Delta\chi^2$. Then Theorem D applies. The value of $\Delta\chi^2$ is given in general by

$$\Delta\chi^2 = \delta\mathbf{a}\cdot\boldsymbol{\alpha}\cdot\delta\mathbf{a} \tag{15.6.1}$$

which follows from equation (15.5.8) applied at $\chi^2_{\min}$ where $\beta_k = 0$. Since $\delta\mathbf{a}$ by hypothesis minimizes χ^2 in all but its zeroth component, components 1 through $M - 1$ of the normal equations (15.5.9) continue to hold. Therefore, the solution of (15.5.9) is

$$\delta\mathbf{a} = \boldsymbol{\alpha}^{-1}\cdot\begin{pmatrix} c \\ 0 \\ \vdots \\ 0 \end{pmatrix} = \mathbf{C}\cdot\begin{pmatrix} c \\ 0 \\ \vdots \\ 0 \end{pmatrix} \tag{15.6.2}$$

where c is one arbitrary constant that we get to adjust to make (15.6.1) give the desired left-hand value. Plugging (15.6.2) into (15.6.1) and using the fact that $\mathbf{C}$ and $\boldsymbol{\alpha}$ are inverse matrices of one another, we get

$$c = \delta a_0 / C_{00} \qquad \text{and} \qquad \Delta\chi^2_\nu = (\delta a_0)^2 / C_{00} \tag{15.6.3}$$

or

$\Delta\chi^2$ as a Function of Confidence Level p and Number of Parameters of Interest ν						
	ν					
p	1	2	3	4	5	6
68.27%	1.00	2.30	3.53	4.72	5.89	7.04
90%	2.71	4.61	6.25	7.78	9.24	10.6
95.45%	4.00	6.18	8.02	9.72	11.3	12.8
99%	6.63	9.21	11.3	13.3	15.1	16.8
99.73%	9.00	11.8	14.2	16.3	18.2	20.1
99.99%	15.1	18.4	21.1	23.5	25.7	27.9

$$\delta a_0 = \pm\sqrt{\Delta\chi^2_\nu}\,\sqrt{C_{00}} \tag{15.6.4}$$

At last! A relation between the confidence interval $\pm\delta a_0$ and the formal standard error $\sigma_0 \equiv \sqrt{C_{00}}$. Not unreasonably, we find that the 68% confidence interval is $\pm\sigma_0$, the 95% confidence interval is $\pm 2\sigma_0$, etc.

These considerations hold not just for the individual parameters a_i, but also for any linear combination of them: If

$$b \equiv \sum_{k=0}^{M-1} c_i a_i = \mathbf{c}\cdot\mathbf{a} \tag{15.6.5}$$

then the 68% confidence interval on b is

$$\delta b = \pm\sqrt{\mathbf{c}\cdot\mathbf{C}\cdot\mathbf{c}} \tag{15.6.6}$$

However, these simple, normal-sounding numerical relationships do *not* hold in the case $\nu > 1$ [3]. In particular, $\Delta\chi^2 = 1$ is not the boundary, nor does it project onto the boundary, of a 68.3% confidence region when $\nu > 1$. If you want to calculate not confidence intervals in one parameter, but confidence ellipses in two parameters jointly, or ellipsoids in three, or higher, then you must follow the following prescription for implementing Theorems C and D above:

- Let ν be the number of fitted parameters whose joint confidence region you wish to display, $\nu \le M$. Call these parameters the "parameters of interest."
- Let p be the confidence limit desired, e.g., $p = 0.68$ or $p = 0.95$.
- Find Δ (i.e., $\Delta\chi^2$) such that the probability of a chi-square variable with ν degrees of freedom being less than Δ is p. For some useful values of p and ν, Δ is given in the table above. For other values, you can use the `invcdf` method of the `Chisqdist` object in §6.14.8 with p as the argument.
- Take the $M \times M$ covariance matrix $\mathbf{C} = \boldsymbol{\alpha}^{-1}$ of the chi-square fit. Copy the intersection of the ν rows and columns corresponding to the parameters of interest into a $\nu \times \nu$ matrix denoted $\mathbf{C}_{\rm proj}$.
- Invert the matrix $\mathbf{C}_{\rm proj}$. (In the one-dimensional case this was just taking the reciprocal of the element C_{00}.)

- The equation for the elliptical boundary of your desired confidence region in the ν-dimensional subspace of interest is

$$\Delta = \delta\mathbf{a}' \cdot \mathbf{C}_{\text{proj}}^{-1} \cdot \delta\mathbf{a}' \tag{15.6.7}$$

  where $\delta\mathbf{a}'$ is the ν-dimensional vector of parameters of interest.

If you are confused at this point, you may find it helpful to compare Figure 15.6.4 and the table on the previous page considering the case $M = 2$ with $\nu = 1$ and $\nu = 2$. You should be able to verify the following statements: (i) The horizontal band between C and C' contains 99% of the probability distribution, so it is a confidence limit on a_1 alone at this level of confidence. (ii) Ditto the band between B and B' at the 90% confidence level. (iii) The dashed ellipse, labeled by $\Delta\chi^2 = 2.30$, contains 68.3% of the probability distribution, so it is a confidence region for a_0 and a_1 jointly, at this level of confidence.

Another point of possible confusion might also be worth airing here. In §15.1.1, when we discussed the use of χ^2 as a goodness-of-fit statistic, we mentioned that a "moderately good" fit could have a χ^2 value that differed by as much as $\pm\sqrt{2\nu}$ from its expected value ν (now the total number of degrees of freedom $N - M$, not ν as used above). Indeed, the suggested tail probability that embodies this advice is $Q = 1 - \texttt{Chisqdist}(\nu)\texttt{.invcdf}\,(\chi^2)$. Yet, in the discussion above, we seem to be saying that small changes in χ^2, as little as ± 1 or ± 2.71 (see table on the previous page), are significant. Can both statements be true?

Yes. In §15.1.1 we were considering the variation in χ^2 over a population of hypothetical data sets with the same parameter values, $\mathbf{a}_{\text{true}}$ (cf. Figure 15.6.1). These values vary by typically $\pm\sqrt{2\nu}$. By contrast, in the discussion above, we took a single data set and held it fixed. We then asked, essentially as an exercise in propagation of errors, how much uncertainty in the fitted parameter values $\mathbf{a}_0$ was generated by the uncertainties in the data. One way to see that these are quite different concepts is to think about how they should each scale with N, the number of data points. As N gets large, χ^2 scales as N, while its variation over hypothetical data sets scales as $N^{1/2}$, essentially a random walk. Now imagine $\mathbf{a}$ varying around its fitted value $\mathbf{a}_0$ by a small amount, $\mathbf{a} = \mathbf{a}_0 + \delta\mathbf{a}$. The change in χ^2 scales with the number of terms in the sum, N, and quadratically with distance from the minimum,

$$\delta\chi^2 \propto N(\delta\mathbf{a})^2 \tag{15.6.8}$$

As the number of data points increases, we reasonably expect the parameters to become more accurately determined, scaling as

$$\delta\mathbf{a} \propto N^{-1/2} \tag{15.6.9}$$

Combining these two equations, we find that $\delta\chi^2$ for the minimum significant change in parameters $\delta\mathbf{a}$ scales as N^0, that is, as a constant. In fact, Theorems B and C above tell us that this is not just reasonable expectation on our part; it is actually true.

15.6.6 Confidence Limits from Singular Value Decomposition

When you have obtained your χ^2 fit by singular value decomposition (§15.4), the information about the fit's formal errors comes packaged in a somewhat different,

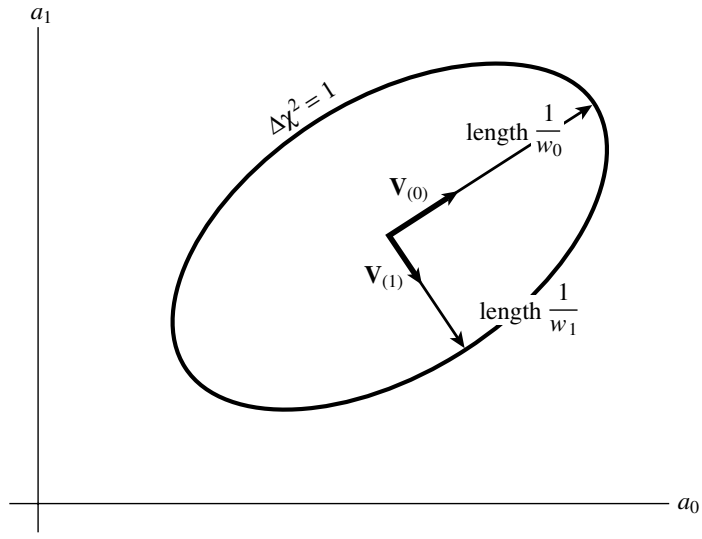


Figure 15.6.5. Relation of the confidence region ellipse $\Delta\chi^2 = 1$ to quantities computed by singular value decomposition. The vectors $\mathbf{V}_{(i)}$ are unit vectors along the principal axes of the confidence region. The semi-axes have lengths equal to the reciprocal of the singular values w_i. If the axes are all scaled by some constant factor α, $\Delta\chi^2$ is scaled by the factor α^2.

but generally more convenient, form. The columns of the matrix $\mathbf{V}$ are an orthonormal set of M vectors that are the principal axes of the $\Delta\chi^2 =$ constant ellipsoids. We denote the columns as $\mathbf{V}_{(0)} \dots \mathbf{V}_{(M-1)}$. The lengths of those axes are inversely proportional to the corresponding singular values $w_0 \dots w_{M-1}$; see Figure 15.6.5. The boundaries of the ellipsoids are thus given by

$$\Delta\chi^2 = w_0^2(\mathbf{V}_{(0)} \cdot \delta\mathbf{a})^2 + \cdots + w_{M-1}^2(\mathbf{V}_{(M-1)} \cdot \delta\mathbf{a})^2 \tag{15.6.10}$$

which is the justification for writing equation (15.4.18) above. Keep in mind that it is *much* easier to plot an ellipsoid given a list of its vector principal axes than given its matrix quadratic form: Loop over points $\mathbf{z}$ on a unit sphere in any desired way (e.g., by latitude and longitude) and plot the mapped points

$$\delta\mathbf{a} = \sqrt{\Delta\chi^2} \sum_i \frac{1}{w_i}(\mathbf{z} \cdot \mathbf{V}_{(i)})\mathbf{V}_{(i)} \tag{15.6.11}$$

The formula for the covariance matrix $\mathbf{C}$ in terms of the columns $\mathbf{V}_{(i)}$ is

$$\mathbf{C} = \sum_{i=0}^{M-1} \frac{1}{w_i^2}\mathbf{V}_{(i)} \otimes \mathbf{V}_{(i)} \tag{15.6.12}$$

or, in components,

$$C_{jk} = \sum_{i=0}^{M-1} \frac{1}{w_i^2} V_{ji} V_{ki} \tag{15.6.13}$$

A method for plotting error ellipses (2-dimensions) or ellipsoids (3-dimensions) from the covariance matrix $\mathbf{C}$ directly, not using its principal axes, is described in §16.1.1.

CITED REFERENCES AND FURTHER READING:

Davison, A.C., and Hinkley, D.V. 1997, *Bootstrap Methods and Their Application* (New York: Cambridge University Press).

Efron, B. 1982, *The Jackknife, the Bootstrap, and Other Resampling Plans* (Philadelphia: S.I.A.M.).[1]

Efron, B., and Tibshirani, R. 1993, *An Introduction to the Bootstrap* (Boca Raton, FL: CRC Press).[2]

Lupton, R. 1993, *Statistics in Theory and Practice* (Princeton, NJ: Princeton University Press), Chapters 10–11.

Avni, Y. 1976, "Energy Spectra of X-Ray Clusters of Galaxies," *Astrophysical Journal*, vol. 210, pp. 642–646.[3]

Lampton, M., Margon, M., and Bowyer, S. 1976, "Parameter Estimation in X-ray Astronomy," *Astrophysical Journal*, vol. 208, pp. 177–190.

Brownlee, K.A. 1965, *Statistical Theory and Methodology*, 2nd ed. (New York: Wiley).

Martin, B.R. 1971, *Statistics for Physicists* (New York: Academic Press).

15.7 Robust Estimation

The concept of *robustness* has been mentioned in passing several times already. In §14.1 we noted that the median was a more robust estimator of central value than the mean; in §14.6 it was mentioned that rank correlation is more robust than linear correlation. The concept of outlier points as exceptions to a Gaussian model for experimental error was discussed in §15.1.

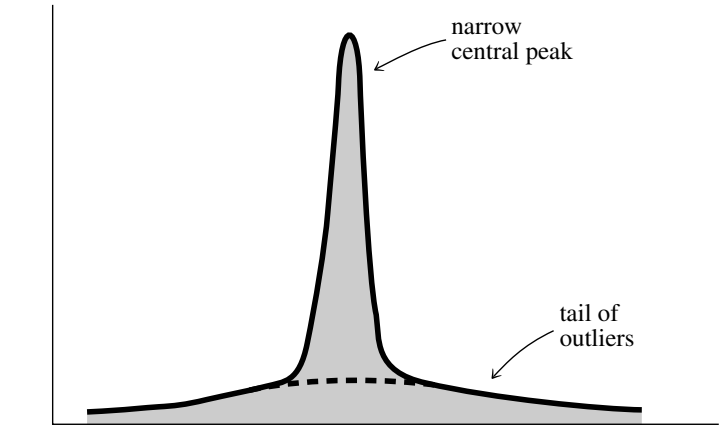
The term "robust" was coined in statistics by G.E.P. Box in 1953. Various definitions of greater or lesser mathematical rigor are possible for the term, but in general, referring to a statistical estimator, it means "insensitive to small departures from the idealized assumptions for which the estimator is optimized" [1,2,3]. The word "small" can have two different interpretations, both important: either fractionally small departures for all data points, or else fractionally large departures for a small number of data points. It is the latter interpretation, leading to the notion of outlier points, that is generally the most stressful for statistical procedures.

Statisticians have developed various sorts of robust statistical estimators. Many, if not most, can be grouped into one of three categories.

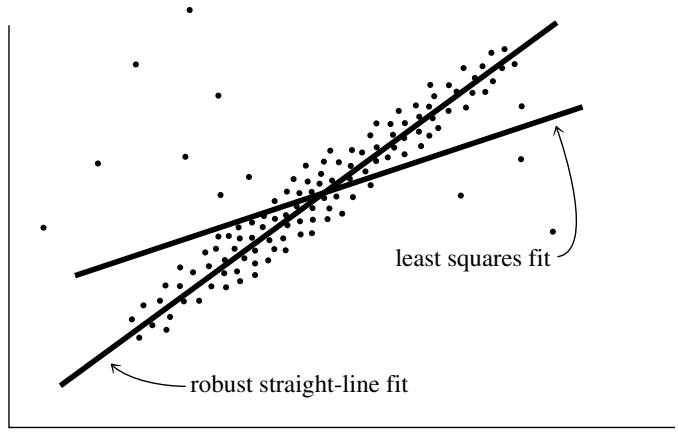
M-estimates follow from maximum likelihood arguments very much as equations (15.1.6) and (15.1.8) followed from equation (15.1.3). M-estimates are usually the most relevant class for model fitting, that is, estimation of parameters. We therefore consider these estimates in some detail below.

L-estimates are "linear combinations of order statistics." These are most applicable to estimations of central value and central tendency, though they can occasionally be applied to some problems in estimation of parameters. Two "typical" L-estimates will give you the general idea. They are (i) the median, and (ii) *Tukey's trimean*, defined as the weighted average of the first, second, and third quartile points in a distribution, with weights 1/4, 1/2, and 1/4, respectively.

R-estimates are estimates based on rank tests. For example, the equality or inequality of two distributions can be estimated by the *Wilcoxon test* of computing the mean rank of one distribution in a combined sample of both distribu-



(a)



(b)

Figure 15.7.1. Examples where robust statistical methods are desirable: (a) A one-dimensional distribution with a tail of outliers; statistical fluctuations in these outliers can prevent accurate determination of the position of the central peak. (b) A distribution in two dimensions fitted to a straight line; nonrobust techniques such as least-squares fitting can have undesired sensitivity to outlying points.

tions. The Kolmogorov-Smirnov statistic (equation 14.3.17) and the Spearman rank-order correlation coefficient (14.6.1) are R-estimates in essence, if not always by formal definition.

Some other kinds of robust techniques, coming from the fields of optimal control and filtering rather than from the field of mathematical statistics, are mentioned at the end of this section. Some examples where robust statistical methods are desirable are shown in Figure 15.7.1.

15.7.1 Estimation of Parameters by Local M-Estimates

Suppose we know that our measurement errors are not normally distributed. Then, in deriving a maximum likelihood formula for the estimated parameters $\mathbf{a}$ in a model $y(x|\mathbf{a})$, we would write instead of equation (15.1.3)

$$P = \prod_{i=0}^{N-1} \{\exp[-\rho(y_i, y\{x_i|\mathbf{a}\})]\,\Delta y\} \tag{15.7.1}$$

where the function ρ is the negative logarithm of the probability density. Taking the logarithm of (15.7.1) analogously with (15.1.5), we find that we want to minimize the expression

$$\sum_{i=0}^{N-1} \rho(y_i, y\{x_i|\mathbf{a}\}) \tag{15.7.2}$$

Very often, it is the case that the function ρ depends not independently on its two arguments, measured y_i and predicted $y(x_i)$, but only on their difference, at least if scaled by some weight factors σ_i that we are able to assign to each point. In this case the M-estimate is said to be *local*, and we can replace (15.7.2) by the prescription

$$\text{minimize over } \mathbf{a} \qquad \sum_{i=0}^{N-1} \rho\left(\frac{y_i - y(x_i|\mathbf{a})}{\sigma_i}\right) \tag{15.7.3}$$

where the function $\rho(z)$ is a function of a single variable $z \equiv [y_i - y(x_i)]/\sigma_i$.

If we now define the derivative of $\rho(z)$ to be a function $\psi(z)$,

$$\psi(z) \equiv \frac{d\rho(z)}{dz} \tag{15.7.4}$$

then the generalization of (15.1.8) to the case of a general M-estimate is

$$0 = \sum_{i=0}^{N-1} \frac{1}{\sigma_i}\psi\left(\frac{y_i - y(x_i)}{\sigma_i}\right)\left(\frac{\partial y(x_i|\mathbf{a})}{\partial a_k}\right) \qquad k = 0, \ldots, M-1 \tag{15.7.5}$$

If you compare (15.7.3) to (15.1.3) and (15.7.5) to (15.1.8), you see at once that the specialization for normally distributed errors is

$$\rho(z) = \tfrac{1}{2}z^2 \qquad \psi(z) = z \qquad \text{(normal)} \tag{15.7.6}$$

If the errors are distributed as a *double* or *two-sided exponential*, namely

$$\text{Prob}\,\{y_i - y(x_i)\} \sim \exp\left(-\left|\frac{y_i - y(x_i)}{\sigma_i}\right|\right) \tag{15.7.7}$$

then, by contrast,

$$\rho(x) = |z| \qquad \psi(z) = \text{sgn}(z) \qquad \text{(double exponential)} \tag{15.7.8}$$

Comparing to equation (15.7.3), we see that in this case the maximum likelihood estimator is obtained by minimizing the *mean absolute deviation*, rather than the mean square deviation. Here the tails of the distribution, although exponentially decreasing, are asymptotically much larger than any corresponding Gaussian.

A distribution with even more extensive — and therefore sometimes even more realistic — tails is the *Cauchy* or *Lorentzian* distribution,

$$\text{Prob}\,\{y_i - y(x_i)\} \sim \frac{1}{1 + \frac{1}{2}\left(\frac{y_i - y(x_i)}{\sigma_i}\right)^2} \tag{15.7.9}$$

This implies

$$\rho(z) = \log\left(1 + \frac{1}{2}z^2\right) \qquad \psi(z) = \frac{z}{1 + \frac{1}{2}z^2} \qquad \text{(Lorentzian)} \qquad (15.7.10)$$

Notice that the ψ function occurs as a weighting function in the generalized normal equations (15.7.5). For normally distributed errors, equation (15.7.6) says that the more deviant the points, the greater the weight. By contrast, when tails are somewhat more prominent, as in (15.7.7), then (15.7.8) says that all deviant points get the same relative weight, with only the sign information used. Finally, when the tails are even larger, (15.7.10) says the ψ increases with deviation, then starts *decreasing*, so that very deviant points — the true outliers — are not counted at all in the estimation of the parameters.

This general idea, that the weight given individual points should first increase with deviation, then decrease, motivates some additional prescriptions for ψ that do not especially correspond to standard, textbook probability distributions. Two examples are

Andrew's sine

$$\psi(z) = \begin{cases} \sin(z/c) & |z| < c\pi \\ 0 & |z| > c\pi \end{cases} \qquad (15.7.11)$$

If the measurement errors happen to be normal after all, with standard deviations σ_i, then it can be shown that the optimal value for the constant c is $c = 2.1$.

Tukey's biweight

$$\psi(z) = \begin{cases} z(1 - z^2/c^2)^2 & |z| < c \\ 0 & |z| > c \end{cases} \qquad (15.7.12)$$

where the optimal value of c for normal errors is $c = 6.0$.

15.7.2 Numerical Calculation of M-Estimates

To fit a model by means of an M-estimate, you first decide which M-estimate you want, that is, which matching pair ρ, ψ you want to use. We rather like (15.7.8) or (15.7.10).

You then have to make an unpleasant choice between two fairly difficult problems. Either find the solution of the nonlinear set of M equations (15.7.5), or else minimize the single function in M variables (15.7.3).

Notice that the function (15.7.8) has a discontinuous ψ and a discontinuous derivative for ρ. Such discontinuities frequently wreak havoc on both general nonlinear equation solvers and general function minimizing routines. You might now think of rejecting (15.7.8) in favor of (15.7.10), which is smoother. However, you will find that the latter choice is also bad news for many general equation solving or minimization routines: Small changes in the fitted parameters can drive $\psi(z)$ off its peak into one or the other of its asymptotically small regimes. Therefore, different terms in the equation spring into or out of action (almost as bad as analytic discontinuities).

Don't despair. If your computer is fast enough, or if your patience is great enough, this is an excellent application for the downhill simplex minimization algorithm exemplified in `Amoeba` (§10.5) or `Amebsa` (§10.12). Those algorithms make

no assumptions about continuity; they just ooze downhill and will work for virtually any sane choice of the function ρ.

It is very much to your (patience) advantage to find good starting values, however. Often this is done by first fitting the model by the standard χ^2 (nonrobust) techniques, e.g., as described in §15.4 or §15.5. The fitted parameters thus obtained are then used as starting values in `Amoeba`, now using the robust choice of ρ and minimizing the expression (15.7.3).

15.7.3 Fitting a Line by Minimizing Absolute Deviation

Occasionally there is a special case that happens to be much easier than is suggested by the general strategy outlined above. The case of equations (15.7.7) – (15.7.8), when the model is a simple straight line,

$$y(x|a,b) = a + bx \tag{15.7.13}$$

and where the weights σ_i are all equal, happens to be such a case. The problem is precisely the robust version of the problem posed in equation (15.2.1) above, namely fit a straight line through a set of data points. The merit function to be minimized is

$$\sum_{i=0}^{N-1} |y_i - a - bx_i| \tag{15.7.14}$$

rather than the χ^2 given by equation (15.2.2).

The key simplification is based on the following fact: The median c_M of a set of numbers c_i is also the value that minimizes the sum of the absolute deviations

$$\sum_i |c_i - c_M|$$

(Proof: Differentiate the above expression with respect to c_M and set it to zero.)

It follows that, for fixed b, the value of a that minimizes (15.7.14) is

$$a = \text{median}\{y_i - bx_i\} \tag{15.7.15}$$

Equation (15.7.5) for the parameter b is

$$0 = \sum_{i=0}^{N-1} x_i \, \text{sgn}(y_i - a - bx_i) \tag{15.7.16}$$

(where sgn(0) is to be interpreted as zero). If we replace a in this equation by the implied function $a(b)$ of (15.7.15), then we are left with an equation in a single variable that can be solved by bracketing and bisection, as described in §9.1. (In fact, it is dangerous to use any fancier method of root finding, because of the discontinuities in equation 15.7.16.)

Here is an object that does all this. It calls `select` (§8.5) to find the median. The bracketing and bisection are built into the routine, as is the linear fit that generates the initial guesses for a and b.

fitmed.h

```
struct Fitmed {
```

Object for fitting a straight line $y = a + bx$ to a set of points (x_i, y_i), by the criterion of least absolute deviations. Call the constructor to calculate the fit. The answers are then available as the variables a, b, and abdev (the mean absolute deviation of the points from the line).

```
    Int ndata;
    Doub a, b, abdev;                          Answers.
    VecDoub_I &x, &y;

    Fitmed(VecDoub_I &xx, VecDoub_I &yy) : ndata(xx.size()), x(xx), y(yy) {
```

Constructor. Given a set of data points xx[0..ndata-1], yy[0..ndata-1], sets a, b, and abdev.

```
        Int j;
        Doub b1,b2,del,f,f1,f2,sigb,temp;
        Doub sx=0.0,sy=0.0,sxy=0.0,sxx=0.0,chisq=0.0;
        for (j=0;j<ndata;j++) {                As a first guess for a and b, we will find the
            sx += x[j];                            least-squares fitting line.
            sy += y[j];
            sxy += x[j]*y[j];
            sxx += SQR(x[j]);
        }
        del=ndata*sxx-sx*sx;
        a=(sxx*sy-sx*sxy)/del;                 Least-squares solutions.
        b=(ndata*sxy-sx*sy)/del;
        for (j=0;j<ndata;j++)
            chisq += (temp=y[j]-(a+b*x[j]),temp*temp);
        sigb=sqrt(chisq/del);                  The standard deviation will give some idea of
        b1=b;                                      how big an iteration step to take.
        f1=rofunc(b1);
        if (sigb > 0.0) {
            b2=b+SIGN(3.0*sigb,f1);            Guess bracket as 3-σ away, in the downhill di-
            f2=rofunc(b2);                         rection known from f1.
            if (b2 == b1) {
                abdev /= ndata;
                return;
            }
            while (f1*f2 > 0.0) {              Bracketing.
                b=b2+1.6*(b2-b1);
                b1=b2;
                f1=f2;
                b2=b;
                f2=rofunc(b2);
            }
            sigb=0.01*sigb;
            while (abs(b2-b1) > sigb) {
                b=b1+0.5*(b2-b1);              Bisection.
                if (b == b1 || b == b2) break;
                f=rofunc(b);
                if (f*f1 >= 0.0) {
                    f1=f;
                    b1=b;
                } else {
                    f2=f;
                    b2=b;
                }
            }
        }
        abdev /= ndata;
    }

    Doub rofunc(const Doub b) {
```

Evaluates the right-hand side of equation (15.7.16) for a given value of b.

```
        const Doub EPS=numeric_limits<Doub>::epsilon();
        Int j;
        Doub d,sum=0.0;
```

```
        VecDoub arr(ndata);
        for (j=0;j<ndata;j++) arr[j]=y[j]-b*x[j];
        if ((ndata & 1) == 1) {
            a=select((ndata-1)>>1,arr);
        } else {
            j=ndata >> 1;
            a=0.5*(select(j-1,arr)+select(j,arr));
        }
        abdev=0.0;
        for (j=0;j<ndata;j++) {
            d=y[j]-(b*x[j]+a);
            abdev += abs(d);
            if (y[j] != 0.0) d /= abs(y[j]);
            if (abs(d) > EPS) sum += (d >= 0.0 ? x[j] : -x[j]);
        }
        return sum;
    }
};
```

15.7.4 Other Robust Techniques

Sometimes you may have a priori knowledge about the probable values and probable uncertainties of some parameters that you are trying to estimate from a data set. In such cases you may want to perform a fit that takes this advance information properly into account, neither completely freezing a parameter at a predetermined value (as in `Fitlin` §15.4) nor completely leaving it to be determined by the data set. The formalism for doing this is called "use of a priori covariances."

A related problem occurs in signal processing and control theory, where it is sometimes desired to "track" (i.e., maintain an estimate of) a time-varying signal in the presence of noise. If the signal is known to be characterized by some number of parameters that vary only slowly, then the formalism of *Kalman filtering* tells how the incoming raw measurements of the signal should be processed to produce best parameter estimates as a function of time. For example, if the signal is a frequency-modulated sine wave, then the slowly varying parameter might be the instantaneous frequency. The Kalman filter for this case is called a *phase-locked loop* and is implemented in the circuitry of modern radio receivers [4,5].

CITED REFERENCES AND FURTHER READING:

Huber, P.J. 1981, *Robust Statistics* (New York: Wiley).[1]

Maronna, R., Martin, D., and Yohai, V. 2006, *Robust Statistics: Theory and Methods* (Hoboken, NJ: Wiley).[2]

Launer, R.L., and Wilkinson, G.N. (eds.) 1979, *Robustness in Statistics* (New York: Academic Press).[3]

Sayed, A.H. 2003, *Fundamentals of Adaptive Filtering* (New York: Wiley-IEEE).[4]

Harvey, A.C. 1989, *Forecasting, Structural Time Series Models and the Kalman Filter* (Cambridge, UK: Cambridge University Press).[5]

15.8 Markov Chain Monte Carlo

In this section and the next we redress somewhat the imbalance, at this point, between frequentist and Bayesian methods of modeling. Like Monte Carlo integra-

tion, *Markov chain Monte Carlo* or *MCMC* is a random sampling method. Unlike Monte Carlo integration, however, the goal of MCMC is not to sample a multidimensional region uniformly. Rather, the goal is to visit a point $\mathbf{x}$ with a probability proportional to some given distribution function $\pi(\mathbf{x})$. The distribution $\pi(\mathbf{x})$ is not quite a probability, because it is not necessarily normalized to have unity integral over the sampled region; but it is proportional to a probability.

Why would we want to sample a distribution in this way? The answer is that Bayesian methods, often implemented using MCMC, provide a powerful way of estimating the parameters of a model and their degree of uncertainty. A typical case is that there is a given set of data $\mathbf{D}$, and that we are able to calculate the probability of the data set *given* the values of the model parameters $\mathbf{x}$, that is, $P(\mathbf{D}|\mathbf{x})$. If we assume a prior $P(\mathbf{x})$, then Bayes' theorem says that the (posterior) probability of the model is proportional to $\pi(\mathbf{x}) \equiv P(\mathbf{D}|\mathbf{x})P(\mathbf{x})$, but with an unknown normalizing constant. Because of this unknown constant, $\pi(\mathbf{x})$ is not a normalized probability density. But if we can sample from it, we can estimate any quantity of interest, for example its mean or variance. Indeed, we can readily recover a normalized probability density by observing how often we sample a given volume $d\mathbf{x}$. Often even more useful, we can observe the distribution of any single component or set of components of the vector $\mathbf{x}$, equivalent to *marginalizing* (i.e., integrating over) the other components.

We could in principle obtain all the same information by ordinary Monte Carlo integration over the region of interest, computing the value of $\pi(\mathbf{x}_i)$ at every (uniformly) sampled point $\mathbf{x}_i$. The huge advantage of MCMC is that it "automatically" puts its sample points preferentially where $\pi(\mathbf{x})$ is large (in fact, in direct proportion). In a high-dimensional space, or where $\pi(\mathbf{x})$ is expensive to compute, this can be advantageous by many orders of magnitude.

Two insights, originally due to Metropolis and colleagues in the early 1950s, lead to feasible MCMC methods. The first is the idea that we should try to sample $\pi(\mathbf{x})$ not via unrelated, independent points, but rather by a *Markov chain*, a sequence of points $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \ldots$ that, while locally correlated, can be shown to eventually visit every point $\mathbf{x}$ in proportion to $\pi(\mathbf{x})$, the *ergodic* property. Here the word "Markov" means that each point $\mathbf{x}_i$ is chosen from a distribution that depends only on the value of the immediately preceding point $\mathbf{x}_{i-1}$. In other words, the chain has memory extending only to one previous point and is completely defined by a transition probability function of two variables $p(\mathbf{x}_i|\mathbf{x}_{i-1})$, the probability with which $\mathbf{x}_i$ is picked given a previous point $\mathbf{x}_{i-1}$.

The second insight is that if $p(\mathbf{x}_i|\mathbf{x}_{i-1})$ is chosen to satisfy the *detailed balance equation*,

$$\pi(\mathbf{x}_1)p(\mathbf{x}_2|\mathbf{x}_1) = \pi(\mathbf{x}_2)p(\mathbf{x}_1|\mathbf{x}_2) \tag{15.8.1}$$

then (up to some technical conditions) the Markov chain will in fact sample $\pi(\mathbf{x})$ ergodically. This amazing fact is worthy of some contemplation. Equation (15.8.1) expresses the idea of *physical equilibrium* in the reversible transition

$$\mathbf{x}_1 \longleftrightarrow \mathbf{x}_2 \tag{15.8.2}$$

That is, if $\mathbf{x}_1$ and $\mathbf{x}_2$ occur in proportion to $\pi(\mathbf{x}_1)$ and $\pi(\mathbf{x}_2)$, respectively, then the overall transition rates in each direction, each the product of a population density and a transition probability, are the same. To see that this might have something to do with the Markov chain being ergodic, integrate both sides of equation (15.8.1) with

respect to $\mathbf{x}_1$:

$$\int p(\mathbf{x}_2|\mathbf{x}_1)\pi(\mathbf{x}_1)\,d\mathbf{x}_1 = \pi(\mathbf{x}_2)\int p(\mathbf{x}_1|\mathbf{x}_2)\,d\mathbf{x}_1 = \pi(\mathbf{x}_2) \tag{15.8.3}$$

The left-hand side of equation (15.8.3) is the probability of $\mathbf{x}_2$, computed by integrating over all possible values of $\mathbf{x}_1$ with the corresponding transition probability. The right-hand side is seen to be the desired $\pi(\mathbf{x}_2)$. So equation (15.8.3) says that if $\mathbf{x}_1$ is drawn from π, then *so is its successor* in the Markov chain, $\mathbf{x}_2$.

We also need to show that the equilibrium distribution is rapidly approached from any starting point $\mathbf{x}_0$. While the formal proof is beyond our scope, a heuristic proof is to recognize that, because of ergodicity, even very unlikely values $\mathbf{x}_0$ will be visited by the equilibrium Markov chain once in a great while. Since the chain has no past memory, choosing any such point as a starting point $\mathbf{x}_0$ is equivalent to just picking up the equilibrium distribution chain at that particular point in time, q.e.d. In practice we need to recognize that when we start from a very unlikely point, successor points will themselves be quite unlikely until we rejoin a more probable part of the distribution. There is thus a need to *burn-in* an MCMC chain by stepping through, and discarding, a certain number of points $\mathbf{x}_i$. Below, we discuss how to determine the length of the burn-in.

We can gain a better understanding the nature of the approach to π using concepts from §11.0 and (in the next chapter) §16.3. Heuristically, let us pretend that the states x_i are discrete. Then $p(x_j|x_i) \equiv P_{ij}$ is a transition matrix satisfying equation (16.3.1). The discussion following equation (16.3.4) shows that the matrix $\mathbf{P}^T$ must have at least one unity eigenvalue. In fact, the vector $\boldsymbol{\pi}$ (the discrete form of the distribution $\pi(\mathbf{x})$) is an eigenvector of $\mathbf{P}^T$ with unity eigenvalue, by equation (15.8.3).

Can there be eigenvalues with magnitude greater than unity? No. Suppose to the contrary that $\lambda > 1$ is the largest eigenvalue, with eigenvector $\mathbf{v}$. Then, repeatedly applying $\mathbf{P}^T$,

$$\lim_{n\to\infty} \quad (\mathbf{P}^T)^n \cdot \mathbf{v} = \lambda^n \mathbf{v} \to \infty \times \mathbf{v} \tag{15.8.4}$$

Any starting distribution that contains even a tiny piece of $\mathbf{v}$ (always possible to arrange) will be driven to have values either < 0 or > 1, which is impossible. Hence it must be that $\lambda \leq 1$.

From an arbitrary starting distribution $\mathbf{u}$, repeated steps of $\mathbf{P}^T$ must thus converge to $\boldsymbol{\pi}$ geometrically, with a rate that is asymptotically the magnitude of the second-largest eigenvalue, which will be < 1 if $\boldsymbol{\pi}$ is the unique equilibrium distribution. If the second eigenvalue is small, the distribution $p(x_j|x_i)$ is said to be *rapidly mixing*.

Obviously missing from this discussion, and beyond our scope, is a discussion of degenerate eigenvalues (related to the question of uniqueness) and a continuous, rather than discrete, treatment. In practice, one rarely knows enough about $\mathbf{P}$ to compute useful bounds on the second eigenvalue a priori.

15.8.1 Metropolis-Hastings Algorithm

Unless we can find a transition probability function $p(\mathbf{x}_2|\mathbf{x}_1)$ that satisfies the detailed balance equation (15.8.1), we have no way to proceed. Luckily, Hastings [1], generalizing Metropolis' work, has given a very general prescription:

Pick a *proposal distribution* $q(\mathbf{x}_2|\mathbf{x}_1)$. This can be pretty much anything you want, as long as a succession of steps generated by it can, in principle, reach everywhere in the region of interest. For example, $q(\mathbf{x}_2|\mathbf{x}_1)$ might be a multivariate normal distribution centered on $\mathbf{x}_1$.

Now, to generate a step starting at $\mathbf{x}_1$, first generate a *candidate point* $\mathbf{x}_{2c}$ by drawing from the proposal distribution. Second, calculate an *acceptance probability* $\alpha(\mathbf{x}_1, \mathbf{x}_{2c})$ by the formula

$$\alpha(\mathbf{x}_1, \mathbf{x}_{2c}) = \min\left(1, \frac{\pi(\mathbf{x}_{2c})\, q(\mathbf{x}_1|\mathbf{x}_{2c})}{\pi(\mathbf{x}_1)\, q(\mathbf{x}_{2c}|\mathbf{x}_1)}\right) \tag{15.8.5}$$

Finally, with probability $\alpha(\mathbf{x}_1, \mathbf{x}_{2c})$, accept the candidate point and set $\mathbf{x}_2 = \mathbf{x}_{2c}$; otherwise reject it and leave the point unchanged (that is, $\mathbf{x}_2 = \mathbf{x}_1$). The net result of this process is a transition probability,

$$p(\mathbf{x}_2|\mathbf{x}_1) = q(\mathbf{x}_2|\mathbf{x}_1)\, \alpha(\mathbf{x}_1, \mathbf{x}_2), \qquad (\mathbf{x}_2 \neq \mathbf{x}_1) \tag{15.8.6}$$

To see how this satisfies detailed balance, first multiply equation (15.8.5) by the denominator in the second argument of the min function. Then write down the identical equation, but exchange $\mathbf{x}_1$ and $\mathbf{x}_2$. From these pieces, one writes,

$$\begin{aligned} \pi(\mathbf{x}_1)\, q(\mathbf{x}_2|\mathbf{x}_1)\, \alpha(\mathbf{x}_1, \mathbf{x}_2) &= \min[\pi(\mathbf{x}_1)\, q(\mathbf{x}_2|\mathbf{x}_1),\ \pi(\mathbf{x}_2)\, q(\mathbf{x}_1|\mathbf{x}_2)] \\ &= \min[\pi(\mathbf{x}_2)\, q(\mathbf{x}_1|\mathbf{x}_2),\ \pi(\mathbf{x}_1)\, q(\mathbf{x}_2|\mathbf{x}_1)] \\ &= \pi(\mathbf{x}_2)\, q(\mathbf{x}_1|\mathbf{x}_2)\, \alpha(\mathbf{x}_2, \mathbf{x}_1) \end{aligned} \tag{15.8.7}$$

which, using equation (15.8.6), can be seen to be exactly the detailed balance equation (15.8.1).

It is often possible to choose the proposal distribution $q(\mathbf{x}_2|\mathbf{x}_1)$ in such a way as to simplify equation (15.8.5). For example, if $q(\mathbf{x}_2|\mathbf{x}_1)$ depends only on the absolute difference $|\mathbf{x}_1 - \mathbf{x}_2|$, as in the case of a normal distribution with fixed covariance, then the ratio $q(\mathbf{x}_1|\mathbf{x}_{2c})/q(\mathbf{x}_{2c}|\mathbf{x}_1)$ is just 1. Another case that occurs frequently is when, for some component x of $\mathbf{x}$, $q(x_{2c}|x_1)$ is lognormally distributed with a mode at x_1. In that case the ratio for this component is x_{2c}/x_1 (cf. equation 6.14.31).

15.8.2 Gibbs Sampler

An important special case of the Metropolis-Hastings algorithm is the *Gibbs sampler*. (Historically, the Gibbs sampler was developed independently of Metropolis-Hastings, see [2,5], but we discuss it here in a unified framework.) The Gibbs sampler is based on the fact that a multivariate distribution is uniquely determined by the set of all of its full conditional distributions; but if you don't know what this means, just read on anyway.

A *full conditional distribution* of $\pi(\mathbf{x})$ is obtained by holding all of the components of $\mathbf{x}$ constant *except one* (call it x), and then sampling as a function of x alone. In other words, it is the distribution that you see when you "drill through" $\pi(\mathbf{x})$ along a coordinate direction, and with fixed values of all the other coordinates. We'll denote a full conditional distribution by the notation $\pi(x \mid \mathbf{x}^-)$, where $\mathbf{x}^-$ means "values of all the coordinates except one." (To keep the notation readable, we are suppressing an index i that would tell which component of $\mathbf{x}$ is x.)

Suppose that we construct a Metropolis-Hastings chain that allows only the one coordinate x to vary. Then equation (15.8.5) would look like this:

$$\alpha(x_1, x_{2c}|\mathbf{x}^-) = \min\left(1, \frac{\pi(x_{2c}|\mathbf{x}^-)\, q(x_1|x_{2c}, \mathbf{x}^-)}{\pi(x_1|\mathbf{x}^-)\, q(x_{2c}|x_1, \mathbf{x}^-)}\right) \tag{15.8.8}$$

Now let's pick as our proposal distribution,

$$q(x_2|x_1, \mathbf{x}^-) = \pi(x_2|\mathbf{x}^-) \tag{15.8.9}$$

Look what happens: The second argument of the min function becomes 1, so the acceptance probability α is also 1. In other words, if we propose a value x_2 from the full conditional distribution $\pi(x_2|\mathbf{x}^-)$, we can always accept it. The advantage is obvious. The disadvantage is that the full conditional distribution *must* be properly normalized as a probability distribution — otherwise how could we use it as a transition probability? Thus, we will usually need to calculate (either analytically or by numerical integration) the normalizing constant

$$\int \pi(x|\mathbf{x}^-)dx \tag{15.8.10}$$

for every $\mathbf{x}^-$ of interest, and we will need to have a practical algorithm for drawing x_2 from the thus-normalized distribution. Note that these one-dimensional normalizing constants are *much* easier to compute than would be the multidimensional normalizing constant for the whole distribution $\pi(\mathbf{x})$.

The full Gibbs sampler operates as follows: Cycle through each component of $\mathbf{x}$ in turn. (A fixed cyclical order is usually used, but choosing a component randomly each time is also fine.) For each component, hold all the other components fixed and draw a new value x from the full conditional distribution $\pi(x \mid \mathbf{x}^-)$ of all possible values of that component. (This is where you might have to do a numerical integral at each step.) Set the component to the new value and go on to the next component.

You can see that the Gibbs sampler is "more global" than the regular Metropolis-Hastings algorithm. At each step, a component of $\mathbf{x}$ gets reset to a value completely independent of its previous value (independent, at least, in the conditional distribution). If we tried to get behavior like this with regular Metropolis-Hastings, by proposing really big multivariate normal steps, say, we would get nowhere, since the steps would be almost always rejected!

On the other hand, the need to draw from a normalized conditional distribution can be a real killer in terms of computational workload. Gibbs sampling can be recommended enthusiastically when the components of $\mathbf{x}$ have discrete, not continuous, values, and not too many possible values for each component. In that case the normalization is just a sum over not-too-many terms, and the Gibbs sampler can be very efficient. For the case of continuous variables, you are probably better off with regular Metropolis-Hastings, unless your particular problem admits to some fast, tricky way of getting the normalizations.

Don't confuse the Gibbs sampler with the tactic of doing regular Metropolis-Hastings steps along one component at a time. For the latter, we restrict the proposal distribution to proposing a change in a single component, either randomly chosen or else cycling through all the components in a regular order. This is sometimes useful if it lets us compute $\pi(\mathbf{x})$ more efficiently (e.g., using saved pieces from the previous calculation on components that have not changed). What makes this *not* Gibbs is that we calculate an acceptance probability in the regular way, with equation (15.8.5) and the full distribution $\pi(\mathbf{x})$, which need not be normalized.

15.8.3 MCMC: A Worked Example

A number of practical details regarding MCMC are best discussed in the context of a worked example:

> *At the beginning of an experiment, events occur Poisson randomly with a mean rate λ_1, but only every k_1th event is recorded. Then, at time t_c, the mean rate changes to λ_2, but now only every k_2th event is recorded. We are given the times $t_0, \ldots, t_{N-1}$ of the N recorded events. Oh, by the way, the values λ_1, λ_2, k_1, k_2, and t_c are all unknown. We want to find them.*

Let's decompose the separate parts of the calculation into separate objects. First we need an object that represents the point $\mathbf{x}$. Although we've been discussing $\mathbf{x}$ as if it were a vector, it can actually be a mixture of continuous, discrete, boolean, or any other kind of variable. In our example we have both continuous and discrete variables.

mcmc.h

```
struct State {
```
Worked MCMC example: Structure containing the components of $\mathbf{x}$.
```
    Doub lam1, lam2;          λ1 and λ2
    Doub tc;                  tc
    Int k1, k2;               k1 and k2
    Doub plog;                Set to log P by Plog, below.

    State(Doub la1, Doub la2, Doub t, Int kk1, Int kk2) :
        lam1(la1), lam2(la2), tc(t), k1(kk1), k2(kk2) {}
    State() {};
};
```

The constructor is used to set initial values. (The `plog` variable is not part of $\mathbf{x}$, but it will be used later.)

Next, we need an object for calculating $\pi(\mathbf{x}) = P(\mathbf{D}|\mathbf{x})$, the probability of the data given the parameters. For our example, we need to use a couple of facts about Poisson processes: If a Poisson process has a rate λ, then the waiting time to the kth event is distributed as Gamma(k, λ), that is,

$$p(\tau|k,\lambda) = \frac{\lambda^k}{(k-1)!}\tau^{k-1}e^{-\lambda\tau} \tag{15.8.11}$$

where $\tau = t_{i+k} - t_i$. (Compare equation 6.14.41, and also §7.3.10.) The exponential distribution is a special case with $k = 1$. Further, probabilities for non-overlapping intervals such as $t_{i+k} - t_i$ and $t_{i+2k} - t_{i+k}$ are independent. It follows that, for our example,

$$P(\mathbf{D}|\mathbf{x}) = \prod_{t_i \le t_c} p(t_{i+1} - t_i \,|\, k_1, \lambda_1) \times \prod_{t_i > t_c} p(t_{i+1} - t_i \,|\, k_2, \lambda_2) \tag{15.8.12}$$

where $p(\tau|k,\lambda)$ is as given in (15.8.11), and where t_i is now the ith *recorded* time. (In the words following equation 15.8.11, t_i was the ith event whether recorded or not.)

Actually, as the amount of data gets large, $P(\mathbf{D}|\mathbf{x})$ is likely to over- or underflow, so it is best to calculate $\log P$. It is important to make this calculation as

efficient as possible, because it will be done at every step. Particularly important is to minimize the amount of looping over all the data points. In our example, if you take the logarithm of equations (15.8.11) and (15.8.12), you'll see that the individual t_i's enter into log P only as a sum of intervals and sum of log of intervals, less than and greater than t_c. An efficient way to proceed is thus to digest the data once and store two cumulative sums. Then, given a value t_c, we can find our place in the table of sums by bisection and read off the left and right sums directly. There is thus no loop over the data at all! Life is rarely so good, but when it is, then *carpe diem*. The resulting object looks like this:

mcmc.h

```
struct Plog {
```
Functor that calculates log P of a `State`.
```
    VecDoub &dat;                                   Bind to data vector.
    Int ndat;
    VecDoub stau, slogtau;

    Plog(VecDoub &data) : dat(data), ndat(data.size()),
    stau(ndat), slogtau(ndat) {
```
    Constructor. Digest the data vector for subsequent fast calculation of log P. The data are assumed to be sorted in ascending order.
```
        Int i;
        stau[0] = slogtau[0] = 0.;
        for (i=1;i<ndat;i++) {
            stau[i] = dat[i]-dat[0];                Equal to sum of intervals.
            slogtau[i] = slogtau[i-1] + log(dat[i]-dat[i-1]);
        }
    }

    Doub operator() (State &s) {
```
    Return log P of s, and also set `s.plog`.
```
        Int i,ilo,ihi,n1,n2;
        Doub st1,st2,stl1,stl2, ans;
        ilo = 0;
        ihi = ndat-1;
        while (ihi-ilo>1) {                         Bisection to find where is t_c in the data.
            i = (ihi+ilo) >> 1;
            if (s.tc > dat[i]) ilo=i;
            else ihi=i;
        }
        n1 = ihi;
        n2 = ndat-1-ihi;
        st1 = stau[ihi];
        st2 = stau[ndat-1]-st1;
        stl1 = slogtau[ihi];
        stl2 = slogtau[ndat-1]-stl1;
```
        Equations (15.8.11) and (15.8.12):
```
        ans =  n1*(s.k1*log(s.lam1)-factln(s.k1-1))+(s.k1-1)*stl1-s.lam1*st1;
        ans += n2*(s.k2*log(s.lam2)-factln(s.k2-1))+(s.k2-1)*stl2-s.lam2*st2;
        return (s.plog = ans);
    }
};
```

The `Plog` object is the only place that the data enter, and they enter only through the constructor. All other parts of the calculation see the data only through the calculation of log P.

Next we come to the proposal generator, which we call `Proposal`. It doesn't have any contact with the data, or with log P. All it needs to know about is the domain of $\mathbf{x}$ (that is, `State`). It is worth thinking hard about the proposal gener-

ator. Although "almost any" generator will work in theory, a poor generator will take longer than the age of the universe to converge, while a good, *rapidly mixing* generator can go like lightning. This is where MCMC starts becoming an art.

Our example is designed to furnish an illustration of this in the interaction between the λ parameters and their corresponding k's. The mean rate of recorded counts is λ/k. Since λ is a continuous variable, we will be proposing relatively small changes in it at each step. Since k is discrete, there is no such thing as a small change, especially when k is small.

If we naively write a generator that proposes random independent changes in λ and k, then, after we have settled down to roughly the right value of λ/k, essentially all proposals for changing k will be rejected. The reason is that the acceptable step in λ required for a change in k from 1 to 2 (say) is so large (doubling λ) that our generator will pick it only, say, every billion years! If we are not smart enough to recognize this problem ahead of time, we can find it experimentally by inspecting the Markov chain as it evolves and noting the proposals to change k are never accepted.

A solution in our case is to have two kinds of steps. The first changes λ (by a small amount) and keeps k fixed. The second changes k and λ, keeping λ/k fixed. We choose randomly between the two kinds of steps, mostly choosing the first kind.

The general issue here is what to do when $\pi(\mathbf{x})$ defines some highly correlated directions among the components in $\mathbf{x}$. If you can recognize these directions, your proposal generator should, at least sometimes, generate proposals along them. Otherwise, it will have to propose very small steps, if they are ever to be accepted. In our example, this latter choice was made impossible by the discreteness in k, forcing us to diagnose and confront the issue directly. So, although `Proposal` doesn't directly have to know about log P, *you* may need a qualitative understanding of log P when you design `Proposal`.

Since only `Proposal` knows the algorithm by which a proposal is generated, this object must also calculate, and return, the ratio $q(\mathbf{x}_1|\mathbf{x}_{2c})/q(\mathbf{x}_{2c}|\mathbf{x}_1)$, which is needed in equation (15.8.5). Here is an example that proposes small lognormal steps for the variables λ_1, λ_2, and t_c, or else proposes incrementing k_1 and k_2 by 1, 0, or -1, with corresponding changes in the λ's as described above.

mcmc.h

```
struct Proposal {
```
Functor implementing the proposal distribution.
```
    Normaldev gau;
    Doub logstep;

    Proposal(Int ranseed, Doub lstep) : gau(0.,1.,ranseed), logstep(lstep) {}

    void operator() (const State &s1, State &s2, Doub &qratio) {
```
    Given state s1, set state s2 to a proposed candidate. Also set qratio to $q(\mathtt{s1}|\mathtt{s2})/q(\mathtt{s2}|\mathtt{s1})$.
```
        Doub r=gau.doub();
        if (r < 0.9) {
```
        Lognormal steps holding the k's constant.
```
            s2.lam1 = s1.lam1 * exp(logstep*gau.dev());
            s2.lam2 = s1.lam2 * exp(logstep*gau.dev());
            s2.tc = s1.tc * exp(logstep*gau.dev());
            s2.k1 = s1.k1;
            s2.k2 = s1.k2;
            qratio = (s2.lam1/s1.lam1)*(s2.lam2/s1.lam2)*(s2.tc/s1.tc);
```
            Factors for lognormal steps.
```
        } else {
```
        Steps that change k_1 and/or k_2.
```
            r=gau.doub();
```

```
            if (s1.k1>1) {
                if (r<0.5) s2.k1 = s1.k1;
                else if (r<0.75) s2.k1 = s1.k1 + 1;
                else s2.k1 = s1.k1 - 1;
            } else {                          k1 = 1 requires special treatment.
                if (r<0.75) s2.k1 = s1.k1;
                else s2.k1 = s1.k1 + 1;
            }
            s2.lam1 = s2.k1*s1.lam1/s1.k1;
            r=gau.doub();                     Now all the same for k2.
            if (s1.k2>1) {
                if (r<0.5) s2.k2 = s1.k2;
                else if (r<0.75) s2.k2 = s1.k2 + 1;
                else s2.k2 = s1.k2 - 1;
            } else {
                if (r<0.75) s2.k2 = s1.k2;
                else s2.k2 = s1.k2 + 1;
            }
            s2.lam2 = s2.k2*s1.lam2/s1.k2;
            s2.tc = s1.tc;
            qratio = 1.;
        }
    }
};
```

(We use the convenient fact that since `Normaldev` is derived from `Ran`, it contains both normal and uniform random number generators.)

How shall we set `logstep`, the size of the proposed lognormal step? A rule of thumb for proposals like this with an adjustable scale is that the average acceptance probability ought to be roughly between 0.1 and 0.4. If it is very much smaller, then decrease the step size parameter; if it is much larger, then increase the step size parameter. In our example, the value `logstep` $= 0.01$ (i.e., proposed changes on the order of $\pm 1\%$) gives good results.

Finally, there is a function that takes a specified number of steps, implementing equation (15.8.5). This short piece of code is about the only "universal" part of MCMC; it has no persistent state and gets all the information it needs via the `State`, `Plog`, and `Proposal` structures. As we have seen, these are all problem-dependent and benefit from cleverness and special tricks.

mcmc.h

```
Doub mcmcstep(Int m, State &s, Plog &plog, Proposal &propose) {
Take m MCMC steps, starting with (and updating) s.
    State sprop;                              Storage for candidate.
    Doub qratio,alph,ran;
    Int accept=0;
    plog(s);
    for (Int i=0;i<m;i++) {                   Loop over steps.
        propose(s,sprop,qratio);
        alph = min(1.,qratio*exp(plog(sprop)-s.plog));      Equation (15.8.5).
        ran = propose.gau.doub();
        if (ran < alph) {                     Accept the candidate.
            s = sprop;
            plog(s);
            accept++;
        }
    }
    return accept/Doub(m);
}
```

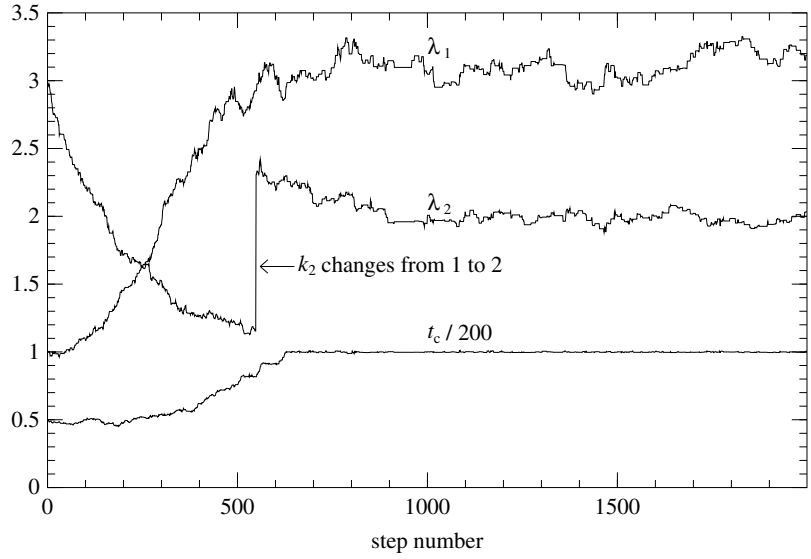


Figure 15.8.1. Evolution of model parameters λ_1, λ_2, and t_c as a function of Markov chain Monte Carlo step. In this example, the burn-in time is seen to be $\sim$ **1000** steps, after which the Markov chain explores the equilibrium distribution.

Let's try it all out. We'll assume $N = 1000$ data points t_i and start $\mathbf{x}$ with the values $\lambda_1 = 1$, $\lambda_2 = 3$, $t_c = 100$, and $k_1 = k_2 = 1$. (Secretly, we know that the data were generated using actual values $3, 2, 200, 1, 2$, respectively.) The random seed is 10102, and the lognormal stepsize is 0.01. We'll take 1000 steps of burn-in, and thereafter store values after every 10 steps. Driver code in `main` for this run is

```
VecDoub times(1000);
...                                   Fill the vector times here.
State s(1.,3.,100.,1,1);
Plog plog(times);
Proposal propose(10102,.01);
for (i=0;i<1000;i++) accept = mcmcstep(1,s,plog,propose);   Burn-in.
for (i=0;i<10000;i++) {                                     Production.
    accept = mcmcstep(10,s,plog,propose);
    ...                               Save values, increment averages, etc., here.
}
```

Figure 15.8.1 shows the evolution of the parameters λ_1, λ_2 and t_c. During burn-in, you can see the parameters heading toward equilibrium, mostly monotonically, but with the exception of λ_2, which goes rapidly toward the value 1, with the value $k = 1$. These values indeed replicate the mean rate of the recorded data. Only when it is near convergence (around step 560), does the model discover that the t_i's greater than t_c don't actually fit an exponential distribution ($k_2 = 1$) but *do* fit a gamma distribution with the same mean rate, but with $k_2 = 2$ (the correct answer). Had we not provided `Proposal` with a step that tests for this, we would likely have converged to a wrong answer. More precisely, we would have produced a model whose true burn-in time was, unknowably, a figurative billion years.

Figure 15.8.2 shows how λ_1 and λ_2 distribute themselves during 10^5 steps after step 1000. This is the payoff of MCMC: We learn not just about most likely parameter values, but also details about how well the parameters are determined by this

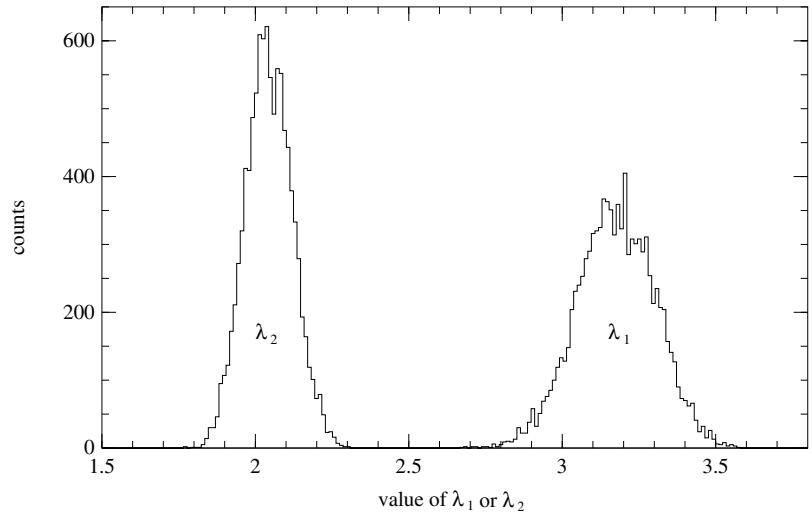


Figure 15.8.2. After burn-in, the MCMC model is run for an additional 10^5 steps. Parameter values are saved every 10 steps, giving the histograms shown for parameters λ_1 and λ_2. These histograms represent the inferred parameter values and their uncertainties. The model data were generated with parameters $\lambda_1 = 3$ and $\lambda_2 = 2$. The inferred values for this particular sample of 1000 data points are seen to be about as accurate as should be expected from their uncertainties.

particular data set. We could also have shown any joint distribution of interest, or computed any average quantity, for example

$$\langle \lambda_1 \rangle = \frac{1}{n-k} \sum_{i=k}^{n-1} (\lambda_1)_i \tag{15.8.13}$$

Here $k = 1000$ is the number of burn-in steps, which we reject; $n = k + 10^5$ is the number of steps that are averaged; and $(\lambda_1)_i$ denotes the value of λ_1 at the ith step. Sums like (15.8.13) are called *ergodic averages*.

Some remarks should be made about equation (15.8.13): One is allowed to average all steps, even though successive steps are not independent samples of $\pi(\mathbf{x})$. One would also be allowed to include in the average only every mth step, where you choose m to be greater than some empirically observed correlation time in the Markov chain. The latter is sometimes recommended as a means of estimating the standard error of $\langle \lambda_1 \rangle$, just as in Monte Carlo integration. (Compare equations 7.7.1 and 7.7.2.) Warning: Doing this in the context of a finite data set is often associated with conceptual error. While it is true that as $n \to \infty$, equation (15.8.13) does converge to a precise value, it is *not* true that this value has anything to do with the actual (population) value of λ_1. Rather, it is just the best apparent (sample) value of λ_1 for this particular data set. The relation of this apparent value to the actual value has nothing to do with the standard error of $\langle \lambda_1 \rangle$, but is instead indicated by the width of the distribution of all the $(\lambda_1)_i$'s.

Figure 15.8.2 illustrates this point well. By running the model for a long time, we could achieve beautifully precise distributions that have extremely well-converged means. But they would not be centered on the (here secretly known) true values 3 and 2. You should run an MCMC model only (i) long enough to be sure (or sure

enough) that there was sufficient burn-in, and (ii) long enough to characterize distributions well enough that the error in mean quantities of interest is reasonably small compared with the observed dispersion of those quantities in the Markov chain.

15.8.4 Other Aspects of MCMC

We have said little about how to determine the necessary length of burn-in, other than to advise you to *look at the output* (which is always a good idea). There is in fact a large literature on this subject [2-4]. A number of so-called *convergence diagnostic* tools have been developed. The problem is that, even when you use these tools, you really must *look at the output* anyway; so their added value is often not large. It is always a good idea to have the length of burn-in be *at least* 1 or 2% of the total length of your run, which will be determined by the accuracy that you need in estimating model parameters. Keep in mind that it is easy to construct scary examples of distributions $\pi(\mathbf{x})$ full of false convergence traps. The more you know about your distribution, the better off you will be.

Multiple, independent Markov chains can be run to explore a single distribution $\pi(\mathbf{x})$. On a single processor the only reason to do this would be to meet some unusual need for independent samples of the distribution. However, on machines with multiple processors, this is a natural way of achieving efficient parallelization.

We have assumed that the number of dimensions in $\mathbf{x}$ is fixed. It is possible to have models, however, in which the number of fitted parameters is itself a variable. These *variable dimension models* require special care in the design of proposal distributions that can step between different numbers of dimensions. See the paper by Phillips and Smith in [2] for an introduction.

15.8.5 Importance Sampling and MCMC

In §7.9 we noted that the error in Monte Carlo integration could be reduced by importance sampling, where we write (in the notation of this section)

$$I \equiv \int f(\mathbf{x})\, d\mathbf{x} = \int \frac{f(\mathbf{x})}{p(\mathbf{x})}\, p(\mathbf{x})\, d\mathbf{x} \tag{15.8.14}$$

for an aptly chosen p. We saw that the ideal p would (i) resemble f in functional form, cf. equation (7.9.6), and (ii) admit to a good method for sampling uniformly over $p(\mathbf{x})\, d\mathbf{x}$.

You might think that MCMC provides a great general-purpose way to sample over any p and thus make importance sampling easy to implement in all cases. Unfortunately, no. The problem, once again (as for the Gibbs sampler), is the normalizing constant. MCMC's great virtue is that it samples over a distribution $\pi(\mathbf{x})$ without requiring that it be normalized. If you ask what normalized probability distribution $p(\mathbf{x})$ is actually being sampled over, it is of course

$$p(\mathbf{x}) = \frac{\pi(\mathbf{x})}{\int \pi(\mathbf{x}) d\mathbf{x}} \tag{15.8.15}$$

Equation (15.8.14) then becomes

$$I \equiv \int f(\mathbf{x})\, d\mathbf{x} = \int \pi(\mathbf{x})\, d\mathbf{x} \;\times\; \int \frac{f(\mathbf{x})}{\pi(\mathbf{x})}\, p(\mathbf{x})\, d\mathbf{x} \tag{15.8.16}$$

The differential $p(\mathbf{x})\,d\mathbf{x}$ can be sampled over by MCMC, knowing only $\pi(\mathbf{x})$; no problem. The $\pi(\mathbf{x})$ in the denominator of the integrand can also be readily computed. But we have, in general, no easy way to calculate that pesky normalizing constant, $\int \pi(\mathbf{x})\,d\mathbf{x}$.

Sometimes, though not often, you can construct a function $\pi(\mathbf{x})$ that both resembles f and also can be integrated analytically, so that the normalizing constant is knowable. Then, yes, by all means use MCMC to sample $\pi(\mathbf{x})$. In this case the idea of recording only every mth step, after choosing m large enough so that the points thus chosen are independent samples, is not a bad idea after all. In fact you'll have to do this if you expect to use the error estimate in equation (7.9.3) as written.

Finally, if the integral that you really want is

$$J \equiv \frac{\int f(\mathbf{x})\pi(\mathbf{x})\,d\mathbf{x}}{\int \pi(\mathbf{x})\,d\mathbf{x}} = \int f(\mathbf{x})\,p(\mathbf{x})\,d\mathbf{x} \tag{15.8.17}$$

with $f(\mathbf{x})$ and $\pi(\mathbf{x})$ both known (and $p(\mathbf{x})$ only implied), then MCMC is exactly what you need. It provides uniform samples over $p(\mathbf{x})d\mathbf{x}$, and no calculation of a normalizing constant is needed.

CITED REFERENCES AND FURTHER READING:

Hastings, W.K. 1970, "Monte Carlo Sampling Methods Using Markov Chains and Their Applications," *Biometrika*, vol. 57, pp. 97–109.[1]

Gilks, W.R., Richardson, S., and Spiegelhalter, D.J., eds. 1996, *Markov Chain Monte Carlo in Practice* (Boca Raton, FL: Chapman & Hall/CRC), especially Chapter 1.[2]

Gamerman, D. 1997, *Markov Chain Monte Carlo: Stochastic Simulation for Bayesian Inference* (London: Chapman & Hall). [3]

Neal, R.M. 1993, "Probabilistic Inference Using Markov Chain Monte Carlo Methods," *Technical Report CRG-TR-93-1*, Department of Computer Science, University of Toronto. Available at `http://www.cs.toronto.edu/~radford/ftp/review.pdf`. [4]

Casella, G., and George, E.I. 1992, "Explaining the Gibbs Sampler," *American Statistician*, vol. 46, no. 3, pp. 167–174.[5]

Tanner, M.A. 2005, *Tools for Statistical Inference: Methods for the Exploration of Posterior Distributions and Likelihood Functions*, 3rd ed. (New York: Springer).

Liu, J.S. 2002, *Monte Carlo Strategies in Scientific Computing* (New York: Springer).

Beichl, I., and Sullivan, F. (eds.) 2006, *Computing in Science and Engineering*, special issue on Monte Carlo Methods, vol. 8, no. 2 (March/April), pp. 7–47.

15.9 Gaussian Process Regression

Some types of statistical models do not depend on knowing (or guessing) parameterized functional forms, and thus lie outside of the parameter-fitting paradigm that has thus far occupied our attention. As an alternative to assuming that our data have some functional form, we can assume that they have some statistical property. A common example is to assume that the data, viewed as an entire set, is drawn from some multivariate normal (Gaussian) distribution in a high-dimensional space. That distribution is allowed to have a complicated correlation structure: The individual data points are *not* assumed to be independent. We can then ask, *given* the

data points that we observe, what are the most probable values for other quantities of interest, for example the values of variables at points other than the ones measured. Of course, as previously, we are also encouraged to ask not just about the most probable values, but about the whole distribution around the most probable values. This general scheme is called *Gaussian process regression*.

We have already met examples of Gaussian process regression twice before in this book, though under different names. In §3.7 we discussed *kriging* as a multidimensional interpolation technique. Later, in §13.6, we discussed *linear prediction*, mostly in the context of one-dimensional data such as time series. Here we can usefully merge some of the ideas in those two sections.

As we presented it in §3.7, kriging was an interpolation, not a fitting, technique. This was evident from the facts that (i) the interpolated function output by the `Krig` object went exactly through the measured data points, and (ii) we never discussed how to input measurement errors. However, the `Krig` object's constructor did have an argument `err`, introduced with the mysterious remark that you should leave it set to `NULL` until you read §15.9. Well, here we are!

We did incorporate measurement errors in §13.6, although they were there called *noise*. In particular, equations (13.6.6) and (13.6.7) can be used (after some change of notation and algebraic manipulation) to derive the appropriate generalization of equations (3.7.14) and (3.7.15) to the case where the measurements y_i, $i = 0, \ldots, N-1$, have errors characterized by some covariance matrix $\mathbf{\Sigma}$. In most cases $\mathbf{\Sigma}$ will be simply a diagonal matrix with elements σ_i^2, the squares of the individual errors. The answers are

$$\hat{y}_* = \mathbf{V}_* \cdot (\mathbf{V} - \mathbf{\Sigma}')^{-1} \cdot \mathbf{Y} \tag{15.9.1}$$

and

$$\mathrm{Var}(\hat{y}_*) = \mathbf{V}_* \cdot (\mathbf{V} - \mathbf{\Sigma}')^{-1} \cdot \mathbf{V}_* \tag{15.9.2}$$

where

$$\mathbf{\Sigma}' \equiv \begin{pmatrix} \mathbf{\Sigma} & 0 \\ 0 & 0 \end{pmatrix} \tag{15.9.3}$$

That is, we simply subtract $\mathbf{\Sigma}$ (suitably augmented by bordering zeros) from $\mathbf{V}$ (equation 3.7.13) before inverting the matrix. The argument `err`, input as the σ_i's (not squared), does this for the case of diagonal measurement errors. Note that `err` has type `Doub*`. If your errors are stored in a `VecDoub`, then you'll send `&err[0]` to the `Krig` constructor. (Sorry about this hack. The purpose was to make `NULL` a possible default value.)

So, no new code is needed in this section. In `Krig`, you already have a serviceable multidimensional Gaussian process regression fitting routine, all ready to go.

When you are fitting, rather than interpolating, it is a good idea to pay more attention to the choice of variogram model than we did in §3.7. While for simple applications there is nothing wrong with the power-law model implemented in the `Powvargram` object

$$v(r) = \alpha r^{\beta} \tag{15.9.4}$$

several other models are widely used. These include the *exponential model*,

$$v(r) = b[1 - \exp(-r/a)] \tag{15.9.5}$$

the *spherical model*,

$$v(r) = \begin{cases} b\left(\frac{3}{2}\frac{r}{a} - \frac{1}{2}\frac{r^3}{a^3}\right) & 0 \le r \le a \\ b & a \le r \end{cases} \tag{15.9.6}$$

and various anisotropic models for which $v(\mathbf{r})$ is not just a function of the magnitude r. See [1,2] for derivations and examples.

We should also mention the so-called *nugget effect*, though, in our opinion, its name vastly outshines its utility. If $v(\mathbf{r})$ does not go to zero as $\mathbf{r} \to 0$, but instead goes to some constant v_0, then the resulting variogram describes a distribution that decorrelates by some finite amount in an infinitesimal distance. That is, if you find a gold nugget at location $\mathbf{x}$, there is no certainty that you'll find another one at location $\mathbf{x} + \delta\mathbf{x}$, no matter how small you make $\delta\mathbf{x}$. Some practitioners deem it desirable to allow for a nonzero nugget effect, allowing nonzero values of v_0 when they empirically fit $v(r)$ from a data set. That seems debatable to us; but in deference to such opinion we have given the `Powvargram` constructor an otherwise undocumented argument, `nug`, for feeding in the value v_0 of your choice. (We draw the line at actually fitting for such a parameter!)

Beyond debatable, and actually incorrect, however, is to confuse the nugget effect with the effect of measurement error. They seem superficially similar: Measurement error also decorrelates measured values, even at arbitrarily small distances (even zero). Conceptually, and mathematically, however, they are different. Referring to equation (3.7.13), a nugget effect adds a constant positive value to all the *off-diagonal* v_{ij}'s. Measurement errors, on the other hand, subtract (not necessarily constant) negative values from the *diagonal* v_{ii}'s. These actions do not have equivalent effects on equations (3.7.14) and (3.7.15). This can readily be seen in Figure 15.9.1, which may also help elucidate the difference between kriging interpolation and kriging fitting. Only panel (d) in the figure shows a correct use of kriging for data with measurement errors, that is, kriging fitting with errors σ_i. Panels (b) and (c) show the results of kriging interpolation with and without a nugget effect. One sees that even with a positive nugget, the interpolated curve goes exactly through the data points, which is incorrect when measurement errors are significant. The legitimate use of kriging interpolation (as in §3.7) is for smooth functions that are "exactly" known at scattered points. Kriging fits using σ_i's (this section) are for data with errors.

CITED REFERENCES AND FURTHER READING:

Cressie, N. 1991, *Statistics for Spatial Data* (New York: Wiley).[1]

Wackernagel, H. 1998, *Multivariate Geostatistics*, 2nd ed. (Berlin: Springer).[2]

Isaaks, E.H., and Srivastava, R.M. 1989, *Applied Geostatistics* (New York: Oxford University Press).

Rasmussen, C.E., and Williams, C.K.I. 2006, *Gaussian Processes for Machine Learning* (Cambridge, MA: MIT Press).

Rybicki, G.B., and Press, W.H. 1992, "Interpolation, Realization, and Reconstruction of Noisy, Irregularly Sampled Data," *Astrophys. J.*, vol. 398, pp. 169–176.

Deutsch, C.V., and Journel, A.G. 1992, *GSLIB: Geostatistical Software Library and User's Guide* (New York: Oxford University Press).

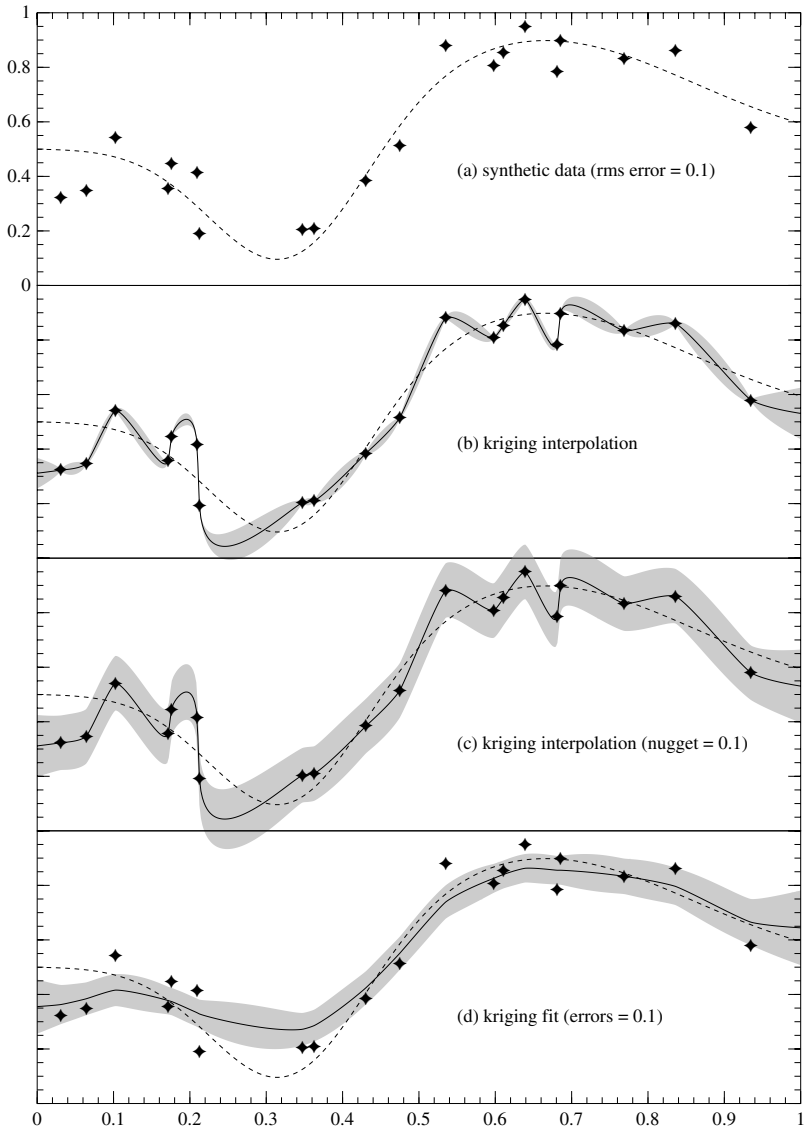


Figure 15.9.1. One-dimensional examples of interpolation and fitting by kriging. (a) Synthetic data points generated from the known curve (dashed line) with Gaussian errors (r.m.s. magnitude 0.1). (b) Result of kriging interpolation. Equation (3.7.14) is plotted as the solid line, while the 1-σ estimated interpolation errors (3.7.15) are shown as the shaded band. The interpolation error is seen to be meaningless for data with measurement errors. (c) Same as (b), but with a nugget effect of $\mathbf{0.1}$. (d) Result of kriging fit (equations 15.9.1 and 15.9.2) using the actual measurement errors. This is the correct use of kriging for data with errors.

CHAPTER 16

Classification and Inference

16.0 Introduction

This chapter groups together a selection of computational techniques whose common feature is that they treat problems of classification and inference on complex models. Given substantially more space, the chapter might have been expanded to be a more complete survey of *machine learning*; but at its present length, it cannot pretend to be such. (A few general references are given below.)

Classification and inference, in a loose sense, are also the goals of many of the purely statistical methods that we already discussed in Chapters 14 and 15, and the line between such techniques and machine learning is a fuzzy one. This chapter's topics tend to have one or both of two characteristics: the underlying model (i) has discrete or combinatorial aspects that distinguish it from "classical" statistical methods, and/or (ii) has empirical or heuristic aspects that make exact statistical treatments unattainable.

There is a list of important topics, related to those in this chapter, that we would have wanted to include if only they could have been reduced to suitable length. *Bayesian networks* is at the top of this list. Section 16.0.1 gives an example of the kind of problem that a Bayesian network can solve. Other significant topics that we must omit include

- genetic algorithms
- neural nets
- kernel methods more general than those discussed in §16.5

16.0.1 Bayesian networks

These are sometimes called *Bayes nets*, *Bayesian learning networks*, or *belief networks*. Here we want only to give a flavor of the method, so that you will know when to consult the references below.

A Bayesian network consists of nodes, each of which can have a value. The values can be {true,false}, or a set of possibilities like {low, medium, high}, or an integer. Figure 16.0.1 shows an example where all the nodes have true/false values.

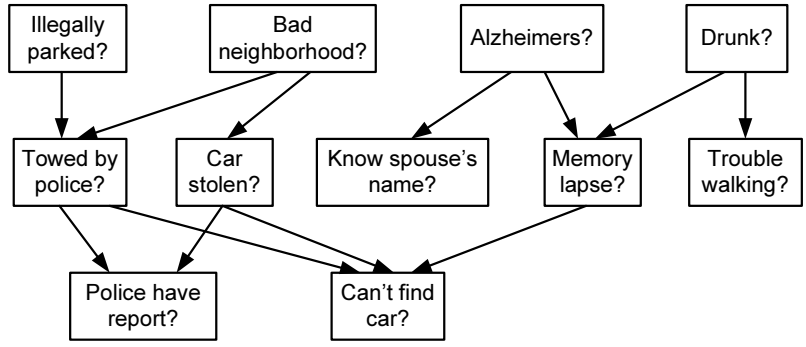


Figure 16.0.1. Example of a Bayesian network. Evidence about any node can be propagated to give probabilistic conclusions about any other node.

Each node in a network has a set of *prior probabilities* or *priors* that give the likelihood of its values absent any additional evidence. If a node has one or more *parents*, then its priors are conditioned on the values of the parents. For example, referring to the Figure, we might have

P(Illegally-Parked = true)
0.20

Bad-N'hood	P(Car-Stolen = true | Bad-N'hood)
true	0.05
false	0.001

Alzheimers	Drunk	P(Memory-Lapse = true | Alzheimers, Drunk)
true	true	0.999
true	false	0.95
false	true	0.50
false	false	0.01

And so on, for all the other nodes.

Things get interesting when we have some evidence to assimilate into the network. For example, you might be coming out of a bar in a bad neighborhood, walking with some difficulty, and be unable to find your car. Is it stolen? The Bayesian network theory gives algorithms for propagating information both up (from "Can't find car?") and down (from "Bad neighborhood?") to get new *posterior* estimates for the probabilities at all nodes, including, here, "Car stolen?" You can also compute in advance the value of new evidence. For example, how much would it help to call the police and see if there is a police report of a towed or recovered, stolen, car?

For more than this brief taste, see [1-3].

CITED REFERENCES AND FURTHER READING:

Hastie, T., Tibshirani, R., and Friedman, J.H. 2003, *The Elements of Statistical Learning* (Berlin: Springer).